

TUGAS AKHIR - ES234849

**PERBANDINGAN PERFORMA TEKNOLOGI *DATA LAKE*
DAN *DATA WAREHOUSE* DALAM PENGELOLAAN
DATA STACK OVERFLOW MENGGUNAKAN
ARSITEKTUR *HYBRID HUB-AND-SPOKE* DAN *KIMBALL
METHOD***

IHSAN KAMIL AL GHOZI

NRP 5026211117

Dosen Pembimbing

Radityo Prasetyanto Wibowo, S.Kom, M.Kom.

NIP 198402032010121003

Program Studi S1 Sistem Informasi

Departemen Sistem Informasi

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2025



TUGAS AKHIR - ES234849

**PERBANDINGAN PERFORMA TEKNOLOGI *DATA LAKE*
DAN *DATA WAREHOUSE* DALAM PENGELOLAAN
DATA STACK OVERFLOW MENGGUNAKAN
ARSITEKTUR *HYBRID HUB-AND-SPOKE* DAN *KIMBALL
METHOD***

IHSAN KAMIL AL GHOZI

NRP 5026211117

Dosen Pembimbing

Radityo Prasetyanto Wibowo, S.Kom, M.Kom.

NIP 198402032010121003

Program Studi S1 Sistem Informasi

Departemen Sistem Informasi

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2025



FINAL PROJECT - ES234849

***PERFORMANCE COMPARISON OF DATA LAKE AND
DATA WAREHOUSE TECHNOLOGY IN DATA STACK
OVERFLOW MANAGEMENT USING HYBRID HUB-
AND-SPOKE ARCHITECTURE AND KIMBALL METHOD***

IHSAN KAMIL AL GHOZI

NRP 5026211117

Advisor

Radityo Prasetianto Wibowo, S.Kom, M.Kom.

NIP 198402032010121003

Study Program S1 Information System

Department of Information System

Faculty of Intelligent Electrical and Informatics Technology

Institut Teknologi Sepuluh Nopember

Surabaya

2025

LEMBAR PENGESAHAN

PERBANDINGAN PERFORMA TEKNOLOGI *DATA LAKE* DAN *DATA WAREHOUSE* DALAM PENGELOLAAN DATA STACK OVERFLOW MENGGUNAKAN ARSITEKTUR *HYBRID HUB-AND-SPOKE* DAN *KIMBALL METHOD*

TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat
memperoleh gelar Sarjana pada
Program Studi S-1 Sistem Informasi
Departemen Sistem Informasi
Fakultas Teknologi Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember

Oleh : **IHSAN KAMIL AL GHOZI**

NRP. 5026211117

Disetujui oleh Tim Penguji Tugas Akhir :

- | | |
|--|------------|
| 1. Radityo Prasetyanto Wibowo, S.Kom, M.Kom. | Pembimbing |
| 2. Faizal Johan Atletiko, S.Kom, MT. | Penguji |
| 3. Irmasari Hafidz, S.Kom., M.Sc. | Penguji |

SURABAYA

Juni, 2025

PERNYATAAN ORISINALITAS

Yang bertanda tangan di bawah ini:

Nama mahasiswa / NRP : Ihsan Kamil Al Ghazi
Program studi : S-1 Sistem Informasi
Dosen Pembimbing / NIP : Radityo Prasetyanto Wibowo, S.Kom, M.Kom. /
198402032010121003

dengan ini menyatakan bahwa Tugas Akhir dengan judul “Perbandingan Performa Teknologi *Data Lake* dan *Data Warehouse* Dalam Pengelolaan Data Stack Overflow Menggunakan Arsitektur *Hybrid Hub-and-Spoke* dan *Kimball Method*” adalah hasil karya sendiri, bersifat orisinal, dan ditulis dengan mengikuti kaidah penulisan ilmiah.

Bilamana di kemudian hari ditemukan ketidaksesuaian dengan pernyataan ini, maka saya bersedia menerima sanksi sesuai dengan ketentuan yang berlaku di Institut Teknologi Sepuluh Nopember.

Surabaya, _____

Mengetahui
Dosen Pembimbing

Mahasiswa

Radityo Prasetyanto Wibowo, S.Kom, M.Kom.
NIP. 198402032010121003

Ihsan Kamil Al Ghazi
NRP. 5026211117

ABSTRAK

PERBANDINGAN PERFORMA TEKNOLOGI *DATA LAKE* DAN *DATA WAREHOUSE* DALAM PENGELOLAAN *DATA STACK OVERFLOW* MENGGUNAKAN *ARSITEKTUR HYBRID HUB-AND-SPOKE* DAN *KIMBALL METHOD*

Nama Mahasiswa / NRP : Ihsan Kamil Al Khozi / 5026211117
Departemen : Sistem Informasi FTEIC - ITS
Dosen Pembimbing : Radityo Prasetyanto Wibowo, S.Kom, M.Kom.

Abstrak

Kemajuan teknologi informasi menuntut organisasi mengelola data skala besar secara efisien untuk mendukung keputusan berbasis data. Dua arsitektur dominan, Data Warehouse yang terstruktur dan Data Lake yang fleksibel, kerap membingungkan organisasi karena minimnya studi perbandingan langsung terkait performa keduanya.

Penelitian ini membandingkan performa Data Warehouse berbasis Kimball Method dengan Data Lake berarsitektur Hybrid Hub-and-Spoke. Evaluasi dilakukan dengan empat metrik utama: waktu pembangunan sistem, durasi ETL, Query per Second (QPS), dan efisiensi penyimpanan.

Implementasi dilakukan menggunakan dataset Stack Overflow. Data Warehouse dibangun dengan SQL Server 2019 dan ETL melalui Pentaho, sedangkan Data Lake memanfaatkan MinIO, ETL Python, dan DuckDB sebagai engine analitik. Pengujian QPS dilakukan secara sistematis menggunakan test harness Python.

Hasil menunjukkan Data Lake unggul signifikan dalam QPS dan efisiensi penyimpanan berkat Parquet dan DuckDB yang ter-vektorisasi. Sebaliknya, Data Warehouse lebih cepat dalam proses ETL jika sumber data sudah terstruktur. Namun, ETL Data Lake menjadi lebih lama saat tahap awal ingestion (konversi dari XML). Penelitian ini menyimpulkan bahwa untuk beban kerja analitis modern, Data Lake menawarkan solusi lebih unggul dan menjadi panduan praktis bagi organisasi dalam memilih arsitektur sesuai kebutuhan dan karakteristik datanya.

Kata kunci: *Data Lake, Data Warehouse, Performa ETL, QPS, 4-step Kimball Method, Hub-and-Spoke, DuckDB*

ABSTRACT

PERFORMANCE COMPARISON OF DATA LAKE AND DATA WAREHOUSE TECHNOLOGY IN DATA STACK OVERFLOW MANAGEMENT USING HYBRID HUB-AND-SPOKE ARCHITECTURE AND KIMBALL METHOD

Student Name / NRP : Ihsan Kamil Al Khozi / 5026211117
Department : Information System FTEIC - ITS
Advisor : Radityo Prasetyanto Wibowo, S.Kom, M.Kom.

Abstract

The advancement of information technology demands organizations to manage large-scale data efficiently to support data-driven decision-making. Two dominant architectures—structured Data Warehouses and flexible Data Lakes—often pose challenges for organizations due to the lack of direct performance comparison studies.

This study compares the performance of a Kimball Method-based Data Warehouse with a Data Lake built on Hybrid Hub-and-Spoke architecture. The evaluation focuses on four key performance metrics: system build time, ETL duration, Query per Second (QPS), and storage efficiency.

The implementation uses the Stack Overflow dataset. The Data Warehouse is built with SQL Server 2019 and ETL using Pentaho, while the Data Lake leverages MinIO, Python-based ETL, and DuckDB as the analytical engine. QPS testing is conducted systematically using a Python-based test harness.

Results show that the Data Lake significantly outperforms in QPS and storage efficiency, thanks to the columnar Parquet format and vectorized DuckDB engine. On the other hand, the Data Warehouse performs faster ETL when data is already structured. However, Data Lake ETL takes longer due to the initial ingestion phase (XML to Parquet conversion). The study concludes that for modern analytical workloads, the Data Lake offers a more performant solution and provides practical guidance for organizations in selecting the architecture that best fits their needs and data characteristics.

Keywords: *Data Lake, Data Warehouse, DuckDB, ETL Performance, Hub-and-Spoke , QPS, 4-step Kimball Method*

DAFTAR ISI

LEMBAR PENGESAHAN	i
PERNYATAAN ORISINALITAS	ii
ABSTRAK	iii
ABSTRACT	iv
DAFTAR ISI	v
DAFTAR GAMBAR	viii
DAFTAR TABEL	ix
DAFTAR KODE	x
BAB 1 PENDAHULUAN	11
1.1 Latar Belakang	11
1.2 Rumusan Masalah	12
1.3 Batasan Masalah	12
1.4 Tujuan	13
1.5 Manfaat	13
BAB 2 TINJAUAN PUSTAKA	14
2.1 Hasil Penelitian Terdahulu	14
2.2 Dasar Teori	18
2.2.1 <i>Data Lake</i>	18
2.2.2 <i>Data Warehouse</i>	19
2.2.3 <i>Hub-and-Spoke</i>	19
2.2.4 Kimball Method	19
2.2.5 <i>Object Storage</i>	20
2.2.6 <i>Extract, Transform, Load (ETL)</i>	20
2.2.7 Pentaho Data Integration	20
2.2.8 <i>Manual Testing</i>	20
BAB 3 METODOLOGI	21
3.1 Metode yang digunakan	21
3.1.1 Penentuan Rumusan Masalah	21
3.1.2 Melakukan Studi Literatur	22
3.1.3 Identifikasi Kebutuhan	22
3.1.4 Merancang Solusi <i>Data Lake</i>	23
3.1.5 Merancang Solusi Data Warehouse	24

3.1.6	Mendefinisikan Proses <i>Dimensional Modelling</i>	25
3.1.7	Merancang Alur ETL.....	25
3.1.8	Melakukan Pengujian Sistem	26
3.1.9	Menyusun Buku Tugas Akhir.....	27
3.2	Urutan pelaksanaan penelitian.....	27
BAB 4	HASIL DAN PEMBAHASAN	29
4.1	Identifikasi Kebutuhan	29
4.2	Pengumpulan Data.....	30
4.3	Data Warehouse.....	33
4.3.1	Pemilihan Proses Bisnis	33
4.3.2	Penentuan Granularitas.....	33
4.3.3	Identifikasi Dimensi	33
4.3.4	Identifikasi Fakta	35
4.4	Data Lake.....	36
4.4.1	Arsitektur Hybrid Hub and Spoke	37
4.4.2	Implementasi Spoke 1: Enterprise Data Warehouse (3NF).....	37
4.4.3	Implementasi Spoke 2: Dimensional Model	41
4.5	Extract, Transform, Load (ETL).....	42
4.5.1	High Level ETL Plan.....	42
4.5.2	Pemuatan Data Historis (<i>One-Time Historical Load</i>)	44
4.5.3	Spesifikasi Pemetaan Data (Source-to-Target Mapping)	45
4.6	Pengujian Sistem	47
4.6.1	Pengujian Waktu Proses ETL.....	47
4.6.2	Pengujian Performa Kueri (QPS)	55
4.6.3	Pengujian Efisiensi Penyimpanan	57
4.6.4	Analisis Kualitatif Waktu Pembangunan	58
4.7	Analisis Hasil dan Pembahasan.....	59
4.7.1	Analisis Performa ETL	59
4.7.2	Analisis Performa Kueri (QPS).....	60
4.7.3	Analisis Efisiensi Penyimpanan	61
4.7.4	Pembahasan Kualitatif.....	63
BAB 5	KESIMPULAN DAN SARAN	65
5.1	Kesimpulan.....	65
5.2	Saran	66

DAFTAR PUSTAKA	67
LAMPIRAN	69
BIODATA PENULIS	99

DAFTAR GAMBAR

Gambar 2.1. Arsitektur Hub-and-Spoke	19
Gambar 3.1. Diagram Alir Tahapan Pengerjaan Tugas Akhir.....	21
Gambar 3.2. Rancangan Arsitektur Data Lake	23
Gambar 3.3. Rancangan Arsitektur Data Warehouse	24
Gambar 4.1. Diagram arsitektur umum Hub-and-Spoke	37
Gambar 4.2. Diagram ERD untuk Spoke 1 (3NF).....	41
Gambar 4.3. Diagram star schema untuk Spoke 2.....	42
Gambar 4.4. High Level ETL Plan	43
Gambar 4.5. Proses ETL untuk tabel dim_activity_type	48
Gambar 4.6. Proses ETL untuk tabel dim_post	48
Gambar 4.7. Proses ETL untuk tabel dim_time.....	49
Gambar 4.8. Proses ETL untuk tabel dim_user	50
Gambar 4.9. Proses ETL untuk tabel fact_user_activity (Comments)	50
Gambar 4.10. Proses ETL untuk tabel fact_user_activity (Posts)	51
Gambar 4.11. Proses ETL untuk tabel fact_user_activity (Votes)	52

DAFTAR TABEL

Tabel 2.1. Penelitian Terdahulu oleh Harby, A. A., & Zulkernine, F. (2022).....	14
Tabel 2.2. Penelitian Terdahulu oleh Nambiar, A. & Mundra, D. (2022)	15
Tabel 2.3. Penelitian Terdahulu oleh Katari, A & Ankam, M. (2020)	15
Tabel 2.4. Penelitian Terdahulu oleh El Aissi et al., (2022)	16
Tabel 2.5. Penelitian Terdahulu oleh Farhan, M. S., Youssef, A., & Abdelhamid, L. (2024)	17
Tabel 4.1. Spesifikasi perangkat keras.....	29
Tabel 4.2. Spesifikasi perangkat lunak	29
Tabel 4.3. Struktur data dbo.Comments	31
Tabel 4.4. Struktur data dbo.Posts	31
Tabel 4.5. Struktur data dbo.Users	32
Tabel 4.6. Struktur data dbo.Votes	32
Tabel 4.7. Struktur tabel dim_time	34
Tabel 4.8. Struktur tabel dim_user.....	34
Tabel 4.9. Struktur tabel dim_post.....	35
Tabel 4.10. Struktur tabel dim_activity_type	35
Tabel 4.11. Struktur tabel fact_user_activity	36
Tabel 4.12. Proses normalisasi pada tabel Post	38
Tabel 4.13. Proses normalisasi pada tabel Comments.....	39
Tabel 4.14. Proses normalisasi pada tabel Users	39
Tabel 4.15. Proses normalisasi pada tabel Votes.....	40
Tabel 4.16. Pemetaan tabel dim_activity_type	45
Tabel 4.17. Pemetaan tabel dim_time.....	45
Tabel 4.18. Pemetaan tabel dim_user	46
Tabel 4.19. Pemetaan tabel dim_post	46
Tabel 4.20. Pemetaan tabel fact_user_activity	46
Tabel 4.21. Rekapitulasi proses ETL pada arsitektur data warehouse	52
Tabel 4.22. ETL pertama: konversi XML ke parquet.....	53
Tabel 4.23. Proses ETL pada data lake Spoke 1	54
Tabel 4.24. Proses ETL pada data lake Spoke 2.....	54
Tabel 4.25. Perbandingan total waktu eksekusi ETL	59
Tabel 4.26. Perbandingan hasil QPS.....	60
Tabel 4.27. Perbandingan ukuran penyimpanan data sumber	61
Tabel 4.28. Hasil konversi XML ke parquet pada data lake.....	62
Tabel 4.29. Perbandingan ukuran penyimpanan data warehouse dan data lake	62

DAFTAR KODE

Kode 4.1. Jumlah total setiap jenis aktivitas (Q1)	55
Kode 4.2. Total skor yang dihasilkan dari vote per tahun (Q2).....	55
Kode 4.3. 10 Pengguna paling aktif di lokasi tertentu (Q3)	56
Kode 4.4. 10 Jumlah vote pada postingan dengan judul 'python' (Q4)	56
Kode 4.5. Rasio komentar per jawaban untuk pengguna paling reputabel (Q5)	57

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Dalam beberapa tahun terakhir, perkembangan teknologi informasi telah mendorong organisasi untuk mengelola dan menganalisis data secara lebih efisien guna mendukung pengambilan keputusan strategis. *Data-driven decision-making* kini menjadi kunci keberhasilan organisasi dalam menghadapi tantangan pasar yang dinamis. Berdasarkan survei, perusahaan dengan CEO yang menjadi ujung tombak dalam pengambilan keputusan berdasarkan data memiliki peluang 77% lebih besar untuk mencapai tujuan bisnisnya, serta 59% lebih mungkin untuk memperoleh wawasan yang dapat ditindaklanjuti dari hasil analisis (Anthony, 2024). Fakta ini menunjukkan bahwa pengambilan keputusan berbasis data merupakan pendekatan yang terbukti efektif dalam menciptakan keunggulan kompetitif.

Untuk mendukung proses ini, dua pendekatan utama dalam pengelolaan data skala besar telah muncul, yaitu *Data Warehouse* dan *Data Lake*. *Data Warehouse* merupakan solusi yang telah lama digunakan oleh berbagai organisasi untuk menyimpan dan mengelola data terstruktur dalam skala besar secara terorganisir. Dengan pendekatan seperti *Kimball Method*, *Data Warehouse* dirancang menggunakan *dimensional modeling* untuk mengintegrasikan berbagai sumber data dan menyajikan laporan yang relevan melalui teknologi *Online Analytical Processing* (OLAP) (Kovacic et al., 2022). Kemampuan ini membuat *Data Warehouse* menjadi pilihan utama bagi organisasi yang membutuhkan analisis yang cepat dan konsisten.

Di sisi lain, *Data Lake* muncul sebagai solusi yang lebih fleksibel untuk menangani data dalam berbagai format—terstruktur, semi-terstruktur, dan tidak terstruktur. Berbeda dengan *Data Warehouse*, *Data Lake* memungkinkan penyimpanan data dalam bentuk mentah (*raw*) dan mengolahnya sesuai kebutuhan di masa depan. Dengan arsitektur seperti *Hybrid Hub-and-Spoke*, *Data Lake* dapat mengintegrasikan pengelolaan data terpusat dengan kemampuan distribusi untuk analisis yang spesifik dan beragam. Fleksibilitas ini menjadi sangat relevan di era *big data*, di mana volume dan keragaman data terus berkembang pesat.

Meskipun *Data Warehouse* dan *Data Lake* memiliki keunggulannya masing-masing, banyak organisasi yang kesulitan dalam memilih teknologi yang tepat sesuai dengan kebutuhan mereka. Tantangan utamanya adalah membandingkan performa kedua arsitektur secara objektif dalam skenario yang sama dengan memperhatikan aspek waktu pembangunan arsitektur, kecepatan proses ETL, *Query per Second* (QPS), serta penggunaan ruang penyimpanan. Metrik-metrik ini sangat berpengaruh terhadap efisiensi dan performa sistem dalam pengelolaan dan analisis data skala besar. Berdasarkan statistik, terdapat sekitar 2,5 triliun *byte* data dibuat setiap harinya yang akan terus meningkat hingga lebih dari 463 *exabyte* pada tahun 2025 (Branka, 2024). Sayangnya, penelitian yang membandingkan kedua pendekatan ini secara langsung masih terbatas, dengan sebagian besar studi hanya fokus pada satu pendekatan tanpa analisis yang lebih mendalam. Hal ini menciptakan kebutuhan untuk penelitian yang lebih komprehensif, yang tidak hanya menguji performa masing-masing sistem, tetapi juga memberikan wawasan praktis tentang bagaimana keduanya dapat diterapkan dalam skenario yang sama.

Penelitian ini bertujuan untuk mengisi kekosongan tersebut dengan menguji performa dua arsitektur pengelolaan data utama: *Data Warehouse* yang menerapkan *Kimball Method* dan *Data Lake* yang menggunakan pendekatan *Hybrid Hub-and-Spoke*. Kedua arsitektur ini akan

diuji untuk menilai berbagai aspek performa, seperti waktu pembangunan, efektivitas penyimpanan data, kecepatan proses ETL, QPS, dan penggunaan ruang penyimpanan dalam pengelolaan dan analisis data skala besar. Dataset yang digunakan dalam penelitian ini adalah Stack Overflow Brent Ozar 50GB, sebuah dataset besar yang relevan untuk pengujian sistem berbasis OLAP dan memberikan gambaran realistis tentang bagaimana sistem ini bekerja ketika menangani volume data besar dengan berbagai kompleksitas. Penelitian ini juga akan menggunakan SQL Server 2019 sebagai alat dalam menangani *Relational Database Management System* (RDBMS) untuk memastikan bahwa pengujian dilakukan dalam lingkungan yang terstandarisasi dan dapat diandalkan. SQL Server 2019 dipilih karena kemampuannya dalam menangani beban data besar dan mendukung pengolahan data yang cepat, yang sangat penting untuk pengujian performa *Data Warehouse* berbasis OLAP.

Melalui penelitian ini, diharapkan dapat tercipta pemahaman yang lebih komprehensif mengenai keunggulan dan keterbatasan masing-masing arsitektur pengelolaan data dalam konteks volume data yang besar dan beragam. Hasil penelitian ini diharapkan dapat memberikan panduan yang jelas bagi organisasi dalam memilih solusi pengelolaan data yang paling sesuai dengan kebutuhan mereka, serta kontribusi signifikan bagi pengembangan studi teknologi pengelolaan data. Selain itu, penelitian ini diharapkan juga memberikan wawasan baru dalam pengoptimalan arsitektur data di masa depan, khususnya dalam menghadapi tantangan yang dihadapi oleh organisasi dalam era *big data*.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah yang diangkat pada tugas akhir ini adalah sebagai berikut:

1. Seberapa signifikkah perbedaan performa antara arsitektur *Data Warehouse* (*Kimball Method*) dengan arsitektur *Data Lake* (*Hybrid Hub-and-Spoke*) jika diukur dari aspek waktu pembangunan, kecepatan proses ETL, dan efektivitas penyimpanan data?
2. Bagaimana kedua arsitektur tersebut berbeda dalam menangani beban kerja analitik, yang diukur melalui metrik *Query per Second* (QPS), dalam skenario untuk mendukung pengambilan keputusan?
3. Bagaimana implikasi dari penggunaan ruang penyimpanan pada masing-masing arsitektur terhadap efisiensi biaya dan skalabilitas pengelolaan data dalam jangka panjang?

1.3 Batasan Masalah

Beberapa batasan masalah yang ditetapkan dalam tugas akhir ini adalah sebagai berikut:

1. Penelitian ini hanya menggunakan dataset Stack Overflow Brent Ozar sebesar 50GB untuk arsitektur *Data Warehouse* dan data Stack Overflow berformat XML untuk *Data Lake* sebagai objek penelitian.
2. Arsitektur dan teknologi yang digunakan terbatas pada:
 - *Data Warehouse* dengan *Kimball Method* yang diimplementasikan pada platform *Relational Database Management System* (RDBMS) SQL Server 2019.
 - *Data Lake* dengan arsitektur *Hybrid Hub-and-Spoke* yang menggunakan MinIO sebagai *object storage*.
 - Proses *Extract, Transform, Load* (ETL) untuk arsitektur *Data Warehouse* akan diimplementasikan menggunakan Pentaho Data Integration, sedangkan untuk arsitektur *Data Lake* akan diimplementasikan menggunakan Python dan DuckDB.

3. Metrik performa yang dievaluasi terbatas pada:
 - Waktu dan kompleksitas Pembangunan arsitektur, yang akan dinilai secara kualitatif.
 - Kecepatan proses ETL.
 - *Query per Second* (QPS)
 - Efisiensi penggunaan ruang penyimpanan.

1.4 Tujuan

Tujuan dari penulisan tugas akhir ini adalah sebagai berikut:

1. Menganalisis dan membandingkan secara kuantitatif performa arsitektur *Data Warehouse* (*Kimball Method*) dengan *Data Lake* (*Hybrid Hub-and-Spoke*) berdasarkan metrik kecepatan proses ETL, *throughput query* (QPS), dan efisiensi penggunaan ruang penyimpanan.
2. Mengevaluasi implikasi dari hasil perbandingan performa (kecepatan, skalabilitas, dan fleksibilitas) terhadap potensi peningkatan kualitas dan kecepatan pengambilan keputusan berbasis data yang difasilitasi oleh masing-masing arsitektur.
3. Memberikan kontribusi pada pengembangan studi teknologi pengelolaan data, khususnya dalam konteks perbandingan performa *Data Warehouse* dan *Data Lake* di era *big data*.

1.5 Manfaat

Berdasarkan latar belakang, rumusan masalah, dan tujuan yang telah diuraikan, manfaat yang diharapkan dari penulisan tugas akhir ini adalah sebagai berikut:

Bagi Organisasi dan Pengambil Keputusan:

1. Memberikan rekomendasi berbasis data tentang pemilihan arsitektur pengelolaan data yang paling sesuai dengan kebutuhan operasional dan strategis organisasi.

Bagi Praktisi Teknis:

1. Menyediakan analisis mendalam mengenai efisiensi waktu pembangunan, kecepatan proses ETL, dan *Query per Second* (QPS) pada kedua arsitektur dalam skenario yang sama.
2. Menyediakan informasi teknis terkait efisiensi arsitektur pengelolaan data yang dapat membantu pengembang dalam memilih teknologi yang optimal untuk implementasi sistem berbasis OLAP.

Bagi Industri Teknologi dan Komunitas Akademik:

1. Menyediakan wawasan yang dapat membantu industri teknologi data menciptakan solusi yang lebih efisien, *scalable*, dan relevan dengan kebutuhan analisis data di era *big data*.

Bagi Penulis:

1. Menjadi sarana untuk menerapkan dan memperdalam pengetahuan teoretis maupun praktis mengenai teknologi pengelolaan data.

BAB 2 TINJAUAN PUSTAKA

2.1 Hasil Penelitian Terdahulu

Penelitian terdahulu berfungsi untuk menyediakan dasar teori dan referensi dalam suatu studi, memberikan wawasan tentang metodologi yang telah digunakan, temuan yang telah dicapai, serta kesenjangan yang masih ada dalam literatur. Berikut merupakan beberapa penelitian yang dijadikan dasar dalam penyusunan tugas akhir ini:

Tabel 2.1. Penelitian Terdahulu oleh Harby, A. A., & Zulkernine, F. (2022)

PENELITIAN 1	
Judul	<i>From Data Warehouse to Lakehouse: A Comparative Review</i>
Nama Peneliti	Ahmed A. Harby, Farhana Zulkernine
Tahun Penelitian	2022
Deskripsi	Penelitian ini membahas evolusi sistem manajemen data dari <i>Data Warehouse</i> ke <i>Data Lake</i> , serta memperkenalkan konsep <i>Data Lakehouse</i> sebagai penggabungan keunggulan dari kedua arsitektur tersebut. Penelitian ini juga memberikan tinjauan komparatif yang mendalam serta rekomendasi desain untuk arsitektur <i>Data Lakehouse</i> yang optimal (Harby & Zulkernine, 2022).
Kelebihan	Penelitian ini menguraikan secara rinci keunggulan dan keterbatasan dari <i>Data Warehouse</i> dan <i>Data Lake</i> dalam konteks <i>big data</i> . Penelitian ini juga menyediakan arahan yang jelas untuk pengembangan arsitektur <i>Lakehouse</i> sebagai solusi integratif serta menjelaskan tantangan utama pengelolaan data besar, seperti kecepatan <i>ingestion</i> , fleksibilitas format, dan efisiensi analitik.
Kekurangan	Penelitian ini tidak menyajikan hasil eksperimen langsung atau simulasi performa antara arsitektur yang dibandingkan. Selain itu, penelitian ini lebih bersifat konseptual dan tidak membahas detail teknis implementasi sistem <i>Lakehouse</i> dan juga tidak mencakup kasus penggunaan dataset.
Keterikatan dengan Tugas Akhir	Penelitian ini memberikan landasan teoretis yang relevan untuk memahami kelebihan dan kekurangan <i>Data Warehouse</i> serta <i>Data Lake</i> , yang mendukung tujuan tugas akhir ini dalam membandingkan keduanya. Namun, tugas akhir ini melengkapi kekurangan studi ini dengan menyediakan pengujian empiris menggunakan dataset nyata serta evaluasi performa berbasis metrik kuantitatif, sehingga memberikan kontribusi praktis yang lebih terukur dan spesifik.

Tabel 2.2. Penelitian Terdahulu oleh Nambiar, A. & Mundra, D. (2022)

PENELITIAN 2	
Judul	<i>An Overview of Data Warehouse and Data Lake in Modern Enterprise Data Management</i>
Nama Peneliti	Athira Nambiar, Divyansh Mundra
Tahun Penelitian	2022
Deskripsi	Penelitian ini mengkaji peran <i>Data Warehouse</i> dan <i>Data Lake</i> dalam pengelolaan data modern pada perusahaan. Penelitian ini juga membahas definisi, karakteristik, dan kerangka kerja terkait kedua sistem, termasuk arsitektur dan pertimbangan desainnya. Artikel ini juga mencakup tantangan implementasi serta peluang penelitian di masa depan dalam pengelolaan <i>big data</i> (Nambiar & Mundra, 2022).
Kelebihan	Penelitian ini memberikan gambaran menyeluruh tentang <i>Data Warehouse</i> dan <i>Data Lake</i> , termasuk definisi dan karakteristik masing-masing. Penelitian ini juga membahas tantangan implementasi serta peluang penelitian, yang dapat membantu mengidentifikasi celah dalam penerapan saat ini, serta menyediakan pandangan masa depan terkait pengembangan teknologi manajemen data.
Kekurangan	Penelitian ini bersifat deskriptif tanpa pengujian empiris atau evaluasi performa sistem. Penelitian ini juga kurang menyertakan studi kasus nyata untuk mendukung analisis yang disajikan, sehingga tidak mencakup perbandingan berbasis metrik yang relevan untuk menentukan efisiensi atau efektivitas kedua sistem.
Keterikatan dengan Tugas Akhir	Penelitian ini menjadi landasan teoritis yang membantu memahami peran, karakteristik, dan tantangan <i>Data Warehouse</i> serta <i>Data Lake</i> . Tugas akhir ini melengkapi celah penelitian ini dengan melakukan pengujian berbasis metrik performa, sehingga memberikan kontribusi lebih konkret dan aplikatif dalam konteks manajemen data besar.

Tabel 2.3. Penelitian Terdahulu oleh Katari, A & Ankam, M. (2020)

PENELITIAN 3

Judul	<i>Data lakes and Optimizing Query</i>
Nama Peneliti	Abhilash Katari, Madhu Ankam
Tahun Penelitian	2020
Deskripsi	Penelitian ini berfokus pada peran <i>Data Lake</i> dalam manajemen data modern, khususnya dalam menyimpan data terstruktur dan tidak terstruktur dengan skala besar. Penelitian ini menyoroti pentingnya optimasi <i>query</i> dalam meningkatkan performa dan efisiensi pengambilan data. Selain itu, dibahas pula praktik terbaik, strategi, serta alat dan teknologi yang dapat digunakan untuk mengoptimalkan <i>query</i> di dalam <i>Data Lake</i> (Katari, 2020).
Kelebihan	Penelitian ini menyediakan panduan praktis tentang optimasi <i>query</i> yang sangat relevan untuk meningkatkan efisiensi sistem <i>Data Lake</i> . Penelitian ini juga menyoroti peran teknologi pendukung dalam mengurangi beban komputasi dan mempercepat proses analitik, serta membahas dampak signifikan optimasi <i>query</i> terhadap pengambilan keputusan bisnis yang lebih cepat dan akurat.
Kekurangan	Penelitian lebih terfokus pada optimasi <i>query</i> tanpa pembahasan rinci tentang integrasi <i>Data Lake</i> dengan sistem lainnya seperti <i>Data Warehouse</i> . Penelitian ini juga tidak memberikan studi kasus konkret yang menunjukkan implementasi nyata dari teknik optimasi <i>query</i> , sehingga ini lebih ke pendekatan teoritis dan kurang disertai validasi empiris atau simulasi performa.
Keterikatan dengan Tugas Akhir	Kedua penelitian ini membahas pentingnya optimasi dalam sistem <i>Data Lake</i> , yang berkontribusi pada efisiensi pengambilan data. Tugas akhir ini akan memanfaatkan beberapa teknik optimisasi <i>query</i> , namun dengan membandingkan performa <i>Data Lake</i> dan <i>Data Warehouse</i> dalam konteks dua arsitektur pengelolaan data yang berbeda.

Tabel 2.4. Penelitian Terdahulu oleh El Aissi et al., (2022)

PENELITIAN 4	
Judul	<i>Data Lake Versus Data Warehouse Architecture: A Comparative Study</i>

Nama Peneliti	Mohamed El Mehdi El Aissi, Sarah Benjelloun, Yassine Loukili, Younes Lakhri, Abdessamad El Boushaki, Hiba Chougrad, Safae Elhaj Ben Ali
Tahun Penelitian	2022
Deskripsi	Penelitian ini membandingkan arsitektur <i>Data Lake</i> dan <i>Data Warehouse</i> , dengan fokus pada perbedaan utama antara keduanya serta faktor kunci yang menentukan aplikasi dan keunggulan masing-masing. Penelitian ini bertujuan untuk memberikan wawasan tentang bagaimana kedua arsitektur tersebut menangani data, terutama dalam konteks volume besar dan data tidak terstruktur (El Aissi et al., 2022).
Kelebihan	Penelitian ini memberikan perbandingan langsung antara <i>Data Lake</i> dan <i>Data Warehouse</i> , mempermudah pemahaman konsep keduanya. Penelitian ini juga menyediakan deskripsi faktor-faktor kunci yang relevan untuk memilih arsitektur berdasarkan kebutuhan organisasi, serta mengidentifikasi keterbatasan tradisional <i>Data Warehouse</i> dan bagaimana <i>Data Lake</i> menawarkan solusi alternatif.
Kekurangan	Penelitian ini berfokus lebih pada definisi dan perbandingan, dengan sedikit perhatian pada aspek teknis atau performa sistem. Penelitian ini juga tidak membahas secara mendalam integrasi antara <i>Data Lake</i> dan <i>Data Warehouse</i> dalam kerangka arsitektur hybrid sehingga kurang menyoroti implementasi praktis atau studi kasus nyata yang mendukung klaimnya.
Keterikatan dengan Tugas Akhir	Keduanya mendalami perbedaan dan manfaat masing-masing arsitektur. Namun tugas akhir ini dapat memperluas studi ini dengan mengeksplorasi bagaimana performa dari <i>Data Lake</i> dan <i>Data Warehouse</i> dapat diukur dari berbagai metrik pengujian, sehingga tidak hanya menjelaskan tentang definisi semata.

Tabel 2.5. Penelitian Terdahulu oleh Farhan, M. S., Youssef, A., & Abdelhamid, L. (2024)

PENELITIAN 5	
Judul	<i>A Model for Enhancing Unstructured Big Data Warehouse Execution Time</i>
Nama Peneliti	Marwa Salah Farhan, Amira Youssef, Laila Abdelhamid

Tahun Penelitian	2024
Deskripsi	Penelitian ini membahas model baru bernama <i>Extract-Clean-Load-Transform</i> (ECLT) yang dirancang untuk meningkatkan efisiensi waktu eksekusi pada <i>data warehouse</i> untuk <i>big data</i> yang tidak terstruktur, terutama teks. Model ini diimplementasikan menggunakan <i>Spark</i> dan menunjukkan performa yang lebih baik dibandingkan pendekatan tradisional seperti ETL, ELT, dan variasi lainnya dalam pengolahan dataset besar hingga 1 TB (Farhan et al., 2024).
Kelebihan	Penelitian ini memperkenalkan model baru yang dapat mengolah data tidak terstruktur dengan lebih cepat. Selain itu, penelitian ini juga berfokus pada efisiensi waktu eksekusi, yang dibuktikan melalui eksperimen dengan dataset besar, serta penggunaan teknologi <i>Spark</i> memungkinkan pengolahan paralel yang mendukung skala data besar.
Kekurangan	Fokus penelitian hanya pada data tidak terstruktur, terutama teks, sehingga penerapannya untuk jenis data lain mungkin terbatas. Selain itu, penelitian ini hanya berfokus pada pengurangan waktu eksekusi tanpa mengelaborasi potensi dampaknya pada analisis data yang lebih kompleks.
Keterikatan dengan Tugas Akhir	Penelitian ini memperkenalkan pendekatan baru untuk pengolahan data besar dengan waktu eksekusi yang optimal. Namun, pada tugas akhir ini akan membahas kinerja data warehouse dan data lake dalam pengelolaan data besar, namun dengan fokus yang lebih luas pada perbandingan antara dua teknologi ini dalam konteks dataset besar dan dua arsitektur pengelolaan data yang berbeda.

2.2 Dasar Teori

Dasar teori merupakan bagian dalam sebuah penelitian atau tulisan ilmiah yang menjelaskan konsep-konsep, prinsip-prinsip, serta teori-teori yang menjadi dasar bagi penelitian yang dilakukan. Tujuan dari bagian ini adalah untuk memberikan dasar ilmiah yang kokoh bagi penelitian dan menguraikan kerangka pemikiran yang digunakan. Berikut ini adalah penjelasan lebih lanjut mengenai beberapa konsep dasar teori yang berkaitan dengan tugas akhir ini.

2.2.1 Data Lake

Data lake adalah *platform* penyimpanan terpusat yang memungkinkan pengguna untuk menyimpan data mentah yang belum diproses dalam format aslinya, termasuk data yang tidak terstruktur, semi-terstruktur, atau terstruktur, dalam skala besar. *Data lake* ini membantu perusahaan untuk membuat keputusan bisnis yang lebih baik melalui visualisasi atau dasbor yang berasal dari analisis *big data*, pembelajaran mesin, dan analitik waktu nyata (Nambiar &

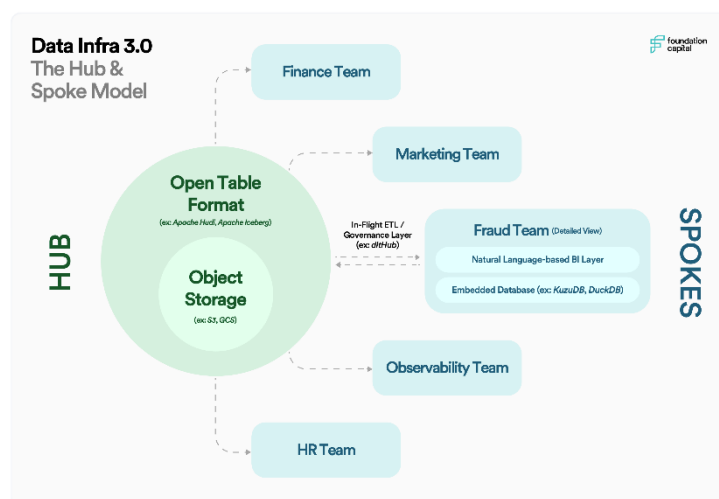
Mundra, 2022). Berbeda dengan *data warehouse*, yang biasanya menyimpan data terstruktur, *data lake* memungkinkan organisasi untuk menyimpan data dalam berbagai format seperti file log, video, atau dokumen teks yang dapat diakses secara *real-time* oleh berbagai alat analitik (Gorelik & Safari, 2019). *Data lake* penting dalam ekosistem *big data* karena menawarkan fleksibilitas dalam mengelola data dari berbagai sumber.

2.2.2 Data Warehouse

Data warehouse adalah repositori terpusat yang dirancang untuk menyimpan volume data besar yang terstruktur dari berbagai sumber. Data warehouse ini mengikuti prinsip OLAP (Online Analytical Processing), yang menekankan pandangan historis yang terpadu dan mengoptimalkan data untuk wawasan analitis. Pengaturan ini memastikan kinerja kueri yang efisien dan memfasilitasi antarmuka yang ramah pengguna untuk mengeksplorasi dan mengekstrak informasi berharga yang mendukung kecerdasan bisnis dan proses pengambilan keputusan (Jaiswar, 2024). Dengan mengorganisir data ke dalam struktur yang terpadu, data warehouse memungkinkan analisis yang efisien terhadap tren historis, memungkinkan bisnis untuk membuat keputusan yang lebih baik berdasarkan wawasan data yang komprehensif.

2.2.3 Hub-and-Spoke

Hybrid *Hub-and-spoke* adalah arsitektur data yang menggabungkan dua model pengelolaan data: *hub-and-spoke* dan model direct flow. Arsitektur *hub-and-spoke* terdiri dari repositori terpusat yang menyimpan seluruh metadata dari setiap alat yang menghasilkan metadata. Repositori pusat ini disebut *hub*, sementara aplikasi seperti alat pemodelan data, aplikasi BI, dan perangkat lunak integrasi data berfungsi sebagai *spoke* (Orogat, 2021). Model *hybrid* ini memungkinkan fleksibilitas karena mengintegrasikan alur data terpusat dengan koneksi langsung ke spokes untuk akses data yang lebih cepat sesuai kebutuhan lokal. Dalam konteks *data lake*, arsitektur ini memungkinkan pengelolaan data mentah secara efisien di *hub*, sementara *spokes* menyediakan akses ke data terstruktur untuk analisis yang lebih spesifik.



Gambar 2.1. Arsitektur Hub-and-Spoke

Sumber: (Iyengar, 2023)

2.2.4 Kimball Method

Kimball Method adalah pendekatan *data warehouse* yang menggunakan *dimensional modeling* untuk mengorganisir data. Menurut metode ini, data diatur dalam bentuk *star schema*

atau *snowflake schema*, dengan fokus pada kemudahan akses untuk keperluan analisis dan pelaporan. Pendekatan ini menekankan pada penggunaan *fact tables* (data transaksi) dan *dimension tables* (atribut terkait) untuk memungkinkan pengguna bisnis melakukan analisis yang lebih fleksibel (Kimball & Ross, 2016). Kimball lebih menekankan pada pelaporan yang terdesentralisasi dan mendukung BI berbasis analisis historis serta visualisasi data yang mudah diakses (Himami et al., 2021). Metodologi ini sangat populer di kalangan organisasi yang ingin memaksimalkan aksesibilitas data untuk analisis yang cepat.

2.2.5 Object Storage

Object Storage adalah metode penyimpanan data di mana data dikelola sebagai objek, bukan file atau blok data tradisional seperti dalam *file sistem* atau *block storage*. Setiap objek biasanya terdiri dari data itu sendiri, metadata, dan ID unik yang memungkinkannya untuk diakses dan diidentifikasi. Salah satu keunggulan *object storage* adalah skalabilitasnya, sehingga cocok untuk menyimpan data dalam jumlah besar yang heterogen seperti di *data lake* (Kovalev et al., 2019). Layanan *cloud* seperti *Amazon S3* dan *Google Cloud Storage* menggunakan model ini untuk menyimpan data besar, yang memfasilitasi akses data yang cepat dan terorganisir.

2.2.6 Extract, Transform, Load (ETL)

Proses *Extract, Transform, Load* (ETL) merupakan dasar yang penting dalam mempersiapkan data untuk analisis *Business Intelligence* (BI). Proses ini membangun saluran terstruktur untuk mengambil data dari berbagai sumber, mengubahnya menjadi format yang dapat digunakan, dan memuatnya ke dalam penyimpanan data target, biasanya berupa *data warehouse* atau *data lake*. ETL berfungsi sebagai tulang punggung rekayasa data, memastikan bahwa aplikasi BI memiliki akses ke data yang bersih, konsisten, dan siap untuk dianalisis (Pattyam, n.d.). ETL merupakan komponen utama dalam alur kerja BI karena proses ini mempersiapkan data untuk dianalisis lebih lanjut.

2.2.7 Pentaho Data Integration

Pentaho adalah platform perangkat lunak sumber terbuka untuk integrasi data dan solusi *Business Intelligence*. Pentaho menyediakan berbagai alat yang mendukung proses ETL, data mining, reporting, dan dashboarding. Sebagai salah satu alat yang sering digunakan dalam pengembangan sistem BI, Pentaho memiliki keunggulan dalam fleksibilitas, kemampuan untuk mengintegrasikan data dari berbagai sumber, serta kemudahan dalam pemrosesan dan visualisasi data (Sulistyo et al., 2020). Pentaho Data Integration dipilih karena mendukung berbagai format data yang disimpan di *data lake* dan mampu beradaptasi dengan arsitektur hybrid seperti *Hub-and-spoke*.

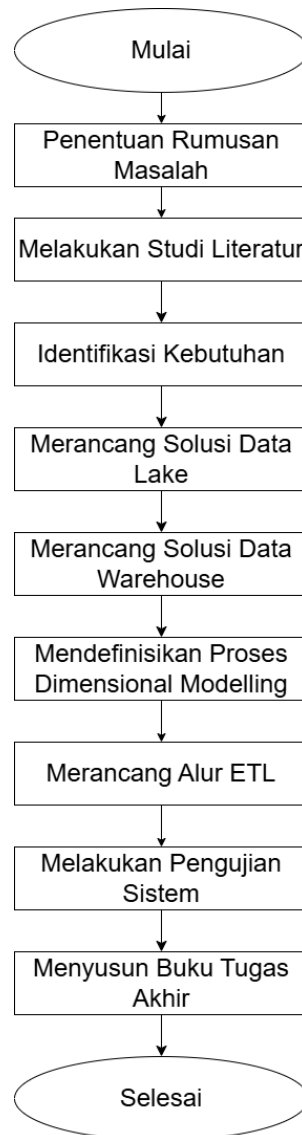
2.2.8 Manual Testing

Manual testing adalah metode pengujian perangkat lunak di mana proses eksekusi tes dilakukan oleh manusia tanpa menggunakan alat otomatisasi. Pengujian ini bertujuan untuk mengidentifikasi bug atau masalah dalam aplikasi berdasarkan spesifikasi yang diberikan. *Manual testing* memainkan peran penting dalam memahami bagaimana suatu aplikasi berinteraksi dengan pengguna akhir dan membantu mendeteksi masalah pada antarmuka, logika bisnis, dan fungsionalitas (Halani et al., 2021).

BAB 3 METODOLOGI

3.1 Metode yang digunakan

Berikut merupakan diagram alir dari tahapan pengerjaan tugas akhir ini:



Gambar 3.1. Diagram Alir Tahapan Pengerjaan Tugas Akhir
Sumber: Dokumentasi Pribadi

Penjelasan lebih rinci untuk masing-masing tahapan pengerjaan dijelaskan pada sub bab di bawah ini.

3.1.1 Penentuan Rumusan Masalah

Tahap awal penelitian ini adalah mendefinisikan masalah utama yang menjadi fokus utama. Latar belakang penelitian diangkat dari kebutuhan industri untuk mengelola data berskala besar secara efisien, serta tantangan yang muncul dalam memilih antara arsitektur *Data Lake* dan

Data Warehouse. Berdasarkan identifikasi kesenjangan pada penelitian terdahulu, di mana perbandingan performa secara langsung antara kedua arsitektur ini masih terbatas, maka dirumuskan tujuan untuk menganalisis dan membandingkan performa keduanya secara kuantitatif. Sebagai objek penelitian, digunakan dataset Stack Overflow, yang diasumsikan representatif untuk mensimulasikan beban kerja analisis data di dunia nyata. Tahapan ini memastikan penelitian memiliki landasan masalah yang kuat, arah yang jelas, dan kontribusi praktis yang signifikan.

3.1.2 Melakukan Studi Literatur

Studi literatur adalah metode penelitian yang dilakukan dengan mengumpulkan, menganalisis, dan mensintesis hasil-hasil penelitian terdahulu yang relevan dengan topik atau bidang tertentu. Studi literatur yang dilakukan secara efektif mampu menjadi landasan kokoh untuk mengembangkan wawasan dan teori baru. Melalui pengintegrasian temuan dan sudut pandang dari berbagai penelitian empiris, metode ini dapat memberikan jawaban atas pertanyaan penelitian dengan cakupan yang lebih luas dibandingkan penelitian tunggal. Selain itu, studi literatur juga berfungsi untuk memberikan gambaran menyeluruh tentang bidang penelitian yang beragam dan lintas disiplin, memperkuat validitas bukti yang ada, serta mengidentifikasi celah penelitian yang masih perlu dieksplorasi lebih lanjut (Snyder, 2019).

Tahap awal dalam studi literatur melibatkan penentuan topik dan cakupan penelitian secara spesifik dan terperinci. Setelah itu, pencarian literatur yang relevan dilakukan menggunakan berbagai sumber, seperti basis data universitas, situs web, serta platform akademik seperti Google Scholar, IEEE Xplore, dan Science Direct. Hasil temuan dari penelitian sebelumnya yang dianggap penting kemudian digabungkan untuk menyusun kerangka teoritis yang kuat sekaligus mengidentifikasi celah penelitian atau area yang belum banyak dikaji

3.1.3 Identifikasi Kebutuhan

Pada tahap ini, dilakukan identifikasi kebutuhan sistem dan data secara mendalam untuk memastikan seluruh proses penelitian berjalan sesuai tujuan. Dataset Stack Overflow dianalisis untuk memahami struktur, volume, dan tipe data yang relevan dalam konteks perbandingan performa. Selanjutnya, dilakukan identifikasi dan pemilihan perangkat lunak utama yang akan membentuk dua teknologi yang akan dibandingkan:

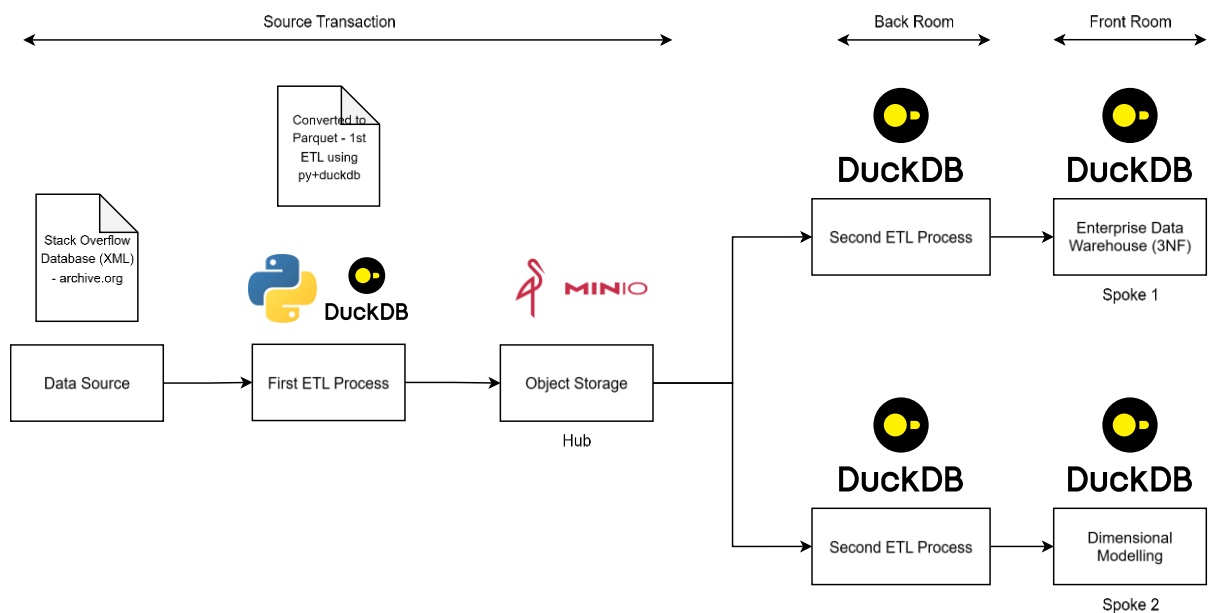
- Untuk Arsitektur *Data Lake*: Dipilih MinIO sebagai *object storage* yang kompatibel dengan ekosistem S3, serta DuckDB sebagai *in-process analytical engine* yang memiliki performa tinggi dalam mengelola format data modern seperti Parquet. Proses ETL dirancang menggunakan bahasa pemrograman Python karena fleksibilitas dan kekayaan ekosistem library-nya.
- Untuk Arsitektur *Data Warehouse*: Dipilih Pentaho Data Integration (PDI) sebagai platform ETL berbasis GUI yang umum digunakan di industri, dan Microsoft SQL Server 2019 sebagai RDBMS target.

Langkah ini memastikan seluruh kebutuhan teknis dan data telah dipersiapkan secara matang agar implementasi dan pengujian sistem dapat dilakukan secara optimal dan komparatif.

3.1.4 Merancang Solusi *Data Lake*

Pada tahap ini, dilakukan perancangan arsitektur sistem untuk solusi *Data Lake*. Desain ini dikembangkan untuk mendukung proses pengolahan data modern dengan menggunakan arsitektur *Hybrid Hub-and-Spoke*, yang dipilih karena kemampuannya untuk menyeimbangkan antara fleksibilitas penyimpanan data mentah dan kebutuhan akan penyajian data yang terstruktur untuk analisis. Tujuan utama tahap ini adalah menyusun kerangka teknis yang memungkinkan pemrosesan data dari format semi-terstruktur hingga menjadi data *insight* yang optimal.

Rancangan arsitektur *Data Lake* yang diusulkan divisualisasikan pada Gambar 3.2.



Gambar 3.2. Rancangan Arsitektur *Data Lake*

Sumber: Dokumentasi Pribadi

Arsitektur ini terdiri dari beberapa komponen dan alur kerja utama:

1. Sumber Data dan *Hub* Penyimpanan

Sumber data utama adalah dataset Stack Overflow dalam format mentah XML. Sebagai langkah pertama, data XML ini akan melalui proses ETL awal yang dibangun menggunakan skrip Python dan DuckDB. Proses ini bertugas untuk mem-parsing dan mengonversi data dari format XML menjadi format Apache Parquet. File-file Parquet yang efisien ini kemudian disimpan di MinIO, yang berfungsi sebagai *Object Storage* terpusat atau *Hub*. *Hub* ini menjadi satu-satunya sumber kebenaran (*single source of truth*) untuk data yang sudah bersih dan siap diolah lebih lanjut.

2. *Spokes* (Penyajian Data Analitis)

Dari *Hub*, data kemudian diproses lebih lanjut untuk menghasilkan dua jenis *spoke* atau cabang analitis yang berbeda, yang masing-masing melayani tujuan yang spesifik:

- *Spoke 1 - Enterprise Data Warehouse (3NF)*

Spoke ini dirancang untuk menghasilkan model data relasional yang ternormalisasi hingga Bentuk Normal Ketiga (3NF). Data dari *Hub* akan diproses melalui ETL kedua untuk memastikan integritas data dan menghilangkan redundansi. Hasilnya adalah sekumpulan file Parquet yang merepresentasikan tabel-tabel dalam skema 3NF.

- *Spoke 2 - Dimensional Modelling*

Spoke ini dirancang untuk menghasilkan skema bintang (*star schema*) yang secara struktur identik dengan yang ada pada arsitektur *Data Warehouse*. Tujuannya adalah untuk menciptakan skenario perbandingan performa kueri yang adil (*apple-to-apple*) dengan arsitektur pembanding.

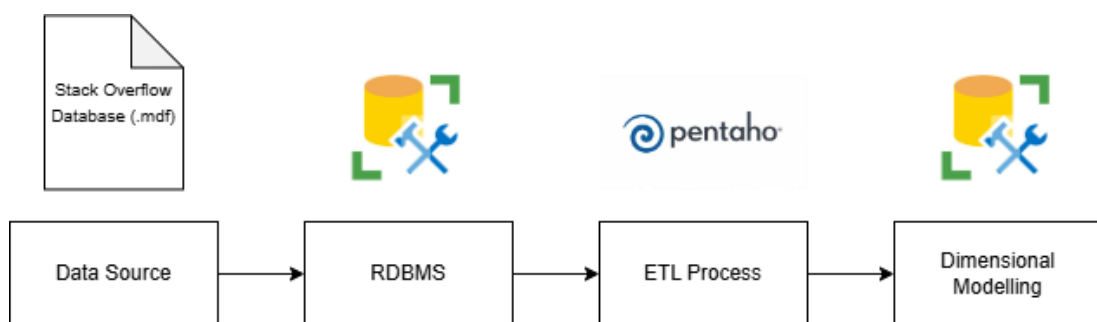
Dengan arsitektur ini, penelitian dapat secara komprehensif menguji dan membandingkan tidak hanya performa penyajian data dalam format yang berbeda (3NF vs. Dimensional), tetapi juga keseluruhan proses ETL dalam ekosistem *Data Lake* modern.

3.1.5 Merancang Solusi Data Warehouse

Tahap ini berfokus pada perancangan arsitektur *Data Warehouse* tradisional yang akan menjadi tolok ukur perbandingan. Arsitektur ini dirancang untuk mengikuti praktik terbaik industri dengan menggunakan Microsoft SQL Server 2019 sebagai *Relational Database Management System* (RDBMS) dan Pentaho Data Integration (PDI) sebagai platform ETL.

Tujuan utamanya adalah untuk mengubah data sumber yang semi-terstruktur menjadi sebuah model dimensional (*star schema*) yang dioptimalkan untuk kueri analitis, sesuai dengan Kimball Method.

Alur kerja yang dirancang untuk arsitektur ini divisualisasikan pada Gambar 3.3.



Gambar 3.3. Rancangan Arsitektur Data Warehouse

Sumber: Dokumentasi Pribadi

Proses perancangan ini mengikuti alur sebagai berikut:

1. Sumber Data

Dataset Stack Overflow berukuran 50GB dalam format database (.mdf) digunakan sebagai sumber data. File ini di-attach langsung ke dalam instance SQL Server 2019, mensimulasikan kondisi di mana data berasal dari sistem database operasional yang ada.

2. Proses ETL

Seluruh proses *Extract, Transform, Load* dirancang dan akan diimplementasikan menggunakan Pentaho Data Integration (PDI). PDI akan bertanggung jawab untuk membaca data dari database sumber, melakukan semua transformasi yang diperlukan seperti membuat *surrogate key*, membersihkan data, dan melakukan *lookup*, serta membentuk struktur tabel dimensi dan fakta.

3. Target (Dimensional Model)

Hasil akhir dari proses ETL adalah sekumpulan tabel dimensi dan fakta yang membentuk *star schema*. Model dimensional ini kemudian dimuat kembali ke dalam

sebuah database baru di SQL Server yang sama, yang secara spesifik berfungsi sebagai *Data Warehouse* untuk kebutuhan analisis.

3.1.6 Mendefinisikan Proses *Dimensional Modelling*

Pada tahap ini, dilakukan perancangan skema logis untuk *Data Warehouse* dengan mengadopsi *4-step Kimball Method*. Pendekatan ini dipilih karena fokusnya pada kemudahan penggunaan dan performa kueri untuk kebutuhan analisis bisnis. Proses perancangan ini dilakukan secara sistematis melalui empat langkah sebagai berikut:

- a. Pemilihan Proses Bisnis
Langkah pertama adalah mengidentifikasi proses bisnis utama yang akan diukur dan dianalisis. Proses yang dipilih harus merupakan aktivitas operasional inti yang menghasilkan data kuantitatif. Untuk penelitian ini, proses yang diidentifikasi adalah Analisis Aktivitas Pengguna di platform Stack Overflow.
- b. Menyatakan Granularitas:
Langkah kedua adalah menentukan grain atau tingkat kedetailan terendah dari data yang akan disimpan di dalam tabel fakta. Granularitas ini harus didefinisikan secara eksplisit untuk memastikan setiap baris fakta memiliki arti yang konsisten. Untuk proses yang telah dipilih, granularitasnya adalah satu baris untuk setiap peristiwa aktivitas tunggal.
- c. Mengidentifikasi Dimensi
Langkah ketiga adalah mengidentifikasi dan merancang dimensi-dimensi yang memberikan konteks deskriptif ("siapa, apa, kapan, di mana") pada fakta. Berdasarkan proses bisnis dan granularitas, dimensi yang relevan diidentifikasi, yang dalam penelitian ini mencakup *dim_time*, *dim_user*, *dim_post*, dan *dim_activity_type*.
- d. Mengidentifikasi Fakta
Langkah terakhir adalah mengidentifikasi facts atau ukuran numerik yang berasal dari proses bisnis. Fakta-fakta ini adalah metrik kuantitatif yang akan diagregasi dan dianalisis. Untuk penelitian ini, fakta yang diidentifikasi mencakup ukuran seperti *ActivityCount* (hitungan peristiwa), *ScoreValue*, *PostLength*, dan lain-lain.

3.1.7 Merancang Alur ETL

Pada tahap ini, dirancang alur kerja *Extract, Transform, Load* (ETL) secara rinci untuk kedua arsitektur yang diuji. Perancangan ini berfungsi sebagai cetak biru teknis untuk implementasi, yang mendefinisikan bagaimana data dipindahkan dari sumber, diubah menjadi model data yang diinginkan, dan dimuat ke sistem target.

- a. Tahap Ekstraksi (*Extract*)
Langkah pertama adalah mengekstrak data dari sistem sumber. Karena kedua arsitektur menggunakan format sumber yang berbeda, pendekatan ekstraksinya pun berbeda:
 - Untuk *Data Warehouse*: Ekstraksi dilakukan dengan *Table Input* di Pentaho yang menjalankan kueri SQL langsung ke database sumber *StackOverflow2013* di SQL Server.
 - Untuk *Data Lake*: Ekstraksi dilakukan dalam dua tahap. Pertama, data mentah format XML di-*parsing* dan dikonversi menjadi format Apache Parquet. Kedua, file-file Parquet ini kemudian dibaca oleh DuckDB dari sistem file lokal yang telah di-*mount* dari MinIO.
- b. Tahap Transformasi (*Transform*)

Data yang telah diekstrak kemudian diproses untuk transformasi. Tahap ini mencakup pembersihan data (penanganan nilai NULL), pengayaan data (pembuatan *surrogate key*), dan penerapan logika bisnis untuk membentuk tabel dimensi dan fakta.

- Untuk *Data Warehouse*: Transformasi dirancang menggunakan serangkaian *step* di Pentaho. Logika *lookup* untuk mendapatkan kunci dimensi dan memisahkan alur *INSERT/UPDATE* dirancang secara manual untuk menangani SCD Tipe 1.
- Untuk *Data Lake*: Transformasi dirancang menggunakan skrip Python. Untuk optimasi performa, diterapkan strategi *hybrid*. Dimensi kecil digabungkan menggunakan *Stream Lookup* di dalam memori, sementara dimensi besar (*dim_post*) digabungkan menggunakan metode *Sort-Merge Join* yang lebih hemat memori.

c. Tahap Pemuatan (Load)

Setelah data ditransformasikan, langkah terakhir adalah memuatnya ke sistem target.

- Untuk *Data Warehouse*: Data hasil transformasi dimuat ke dalam tabel dimensi dan fakta yang sesuai di dalam database ETL_SO di SQL Server menggunakan *step Table Output* dan *Update*.
- Untuk *Data Lake*: Data hasil transformasi ditulis kembali sebagai file Parquet baru dan diunggah ke *spoke* yang sesuai di MinIO (*spoke1_edw3nf* atau *spoke2_dm*).

3.1.8 Melakukan Pengujian Sistem

Tahap pengujian sistem merupakan fase krusial dalam penelitian ini yang bertujuan untuk mengevaluasi dan memvalidasi kedua arsitektur yang telah diimplementasikan. Pengujian dirancang secara sistematis untuk mengumpulkan data kuantitatif dan kualitatif yang akan menjadi dasar perbandingan performa. Metodologi pengujian mencakup beberapa skenario utama yang dijelaskan di bawah ini.

a. Pengujian Kinerja (*Performance Testing*)

Pengujian ini berfokus pada pengukuran metrik-metrik utama yang berkaitan dengan kecepatan dan *throughput* sistem.

- Kecepatan Proses ETL: Mengukur total waktu yang dibutuhkan oleh setiap arsitektur untuk menyelesaikan keseluruhan proses full refresh, dari ekstraksi data sumber hingga data siap saji di sistem target. Ini akan mengevaluasi efisiensi pipeline data secara end-to-end.
- Performa Kueri (QPS): Menjalankan serangkaian kueri analitis yang identik pada kedua sistem secara berulang dalam interval waktu yang ditentukan (300 detik) untuk mengukur jumlah kueri yang dapat diproses per detik. Pengujian ini mensimulasikan beban kerja analitis untuk mengevaluasi *throughput* sistem.

b. Pengujian Efisiensi Sumber Daya

Pengujian ini bertujuan untuk menganalisis efisiensi penggunaan sumber daya infrastruktur. Untuk *Data Warehouse* akan ditotal ukuran penggunaan penyimpanan di masing-masing tabel, sedangkan untuk *Data Lake* akan ditotal semua ukuran file parquet di *spoke 1* dan *spoke 2*.

c. Analisis Kualitatif

Selain pengukuran kuantitatif, dilakukan juga analisis kualitatif terhadap aspek-aspek pembangunan. Pengukuran diuji dari aspek kompleksitas dan waktu Pembangunan, yakni mengevaluasi secara subjektif kemudahan penggunaan tools, tantangan dalam implementasi, fleksibilitas arsitektur, dan waktu yang dihabiskan untuk proses *debugging* pada kedua platform.

d. Validasi Fungsional Alur ETL

Pengujian ini bertujuan untuk memastikan proses ETL berjalan sesuai rancangan tanpa menyebabkan kehilangan atau kerusakan data. Validasi ini mencakup pemeriksaan konsistensi jumlah baris dan keakuratan data antara sumber dan target setelah proses ETL selesai. Ini menjamin bahwa perbandingan performa dilakukan pada data yang valid dan reliabel.

3.1.9 Menyusun Buku Tugas Akhir

Tahap penutup dalam proyek ini adalah pembuatan laporan tugas akhir yang mendokumentasikan seluruh proses penelitian dan implementasi sistem. Laporan ini mencakup berbagai aspek seperti latar belakang permasalahan, analisis kebutuhan pengguna, proses bisnis yang diterapkan, metode yang digunakan dalam pengembangan *data lake* dan *data warehouse*, serta arsitektur sistem yang diimplementasikan beserta hasil pengujiannya. Pembahasan hasil dan saran pengembangan di masa mendatang juga disertakan untuk memberikan wawasan lebih lanjut. Dengan adanya laporan ini, dapat menjadi referensi yang komprehensif dan berguna untuk penelitian atau pengembangan sistem serupa di masa depan.

3.2 Urutan pelaksanaan penelitian

Berikut ini adalah jadwal pelaksanaan penelitian yang dirancang dalam kurun waktu 4 bulan.

Tabel 3.1. Jadwal Pelaksanaan Penelitian

No	Jenis Kegiatan	Bulan															
		I				II				III				IV			
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1	Penentuan Rumusan Masalah																
2	Melakukan studi literatur																
3	Identifikasi kebutuhan																
4	Merancang solusi <i>data lake</i>																
5	Merancang solusi <i>data warehouse</i>																
6	Mendefinisikan proses dimensional modelling																
7	Merancang alur ETL																

8	Melakukan pengujian sistem																
9	Menyusun buku tugas akhir																

BAB 4 HASIL DAN PEMBAHASAN

4.1 Identifikasi Kebutuhan

Sebagai landasan fundamental dalam penelitian ini, dilakukan identifikasi kebutuhan secara sistematis untuk memastikan seluruh proses implementasi dan pengujian berjalan sesuai dengan tujuan yang telah ditetapkan. Identifikasi ini mencakup empat pilar utama: kebutuhan fungsional sistem, kebutuhan non-fungsional untuk pengukuran performa, kebutuhan data sebagai objek penelitian, serta kebutuhan perangkat keras dan lunak sebagai lingkungan implementasi.

Untuk mendukung proses pengembangan dan perbandingan performa arsitektur Data Lake dan Data Warehouse, digunakan lingkungan teknis yang terdiri dari perangkat keras dan lunak yang telah terdefinisi. Penggunaan perangkat ini disesuaikan untuk menangani keseluruhan proses *Extract, Transform, Load* (ETL), pengelolaan data berskala besar, serta pengujian performa yang komprehensif. Spesifikasi lengkap perangkat keras yang digunakan disajikan pada Tabel 4.1

Tabel 4.1. Spesifikasi perangkat keras

Laptop Pribadi	
Merek/Model	Lenovo IdeaPad Slim 5 14ARE05
Prosesor	AMD Ryzen 5 4500U with Radeon Graphics 2.38 GHz
RAM	8 GB
Sistem Operasi	Windows 11 Home Single Language 64-bit (Build 22631)
Penyimpanan	512GB + 1TB External Hard Disk

Selain perangkat keras, sistem ini juga dikembangkan dengan menggunakan perangkat lunak dengan spesifikasi seperti pada Tabel 4.2.

Tabel 4.2. Spesifikasi perangkat lunak

Proses ETL	Pentaho Data Integration, DuckDB, Python
Object Storage	MinIO
RDBMS	Microsoft SQL Server 2019

Sebagai langkah awal dalam fase implementasi, dilakukan identifikasi kebutuhan secara menyeluruh untuk memastikan penelitian berjalan sesuai dengan tujuan dan ruang lingkup yang telah ditetapkan. Kebutuhan ini mencakup aspek fungsional, non-fungsional, data, dan teknologi. Kebutuhan fungsional sistem didefinisikan sebagai kemampuan untuk menjalankan proses ETL secara *end-to-end*. Sistem harus mampu mengekstrak data dari berbagai sumber

(database relasional dan file Parquet), melakukan transformasi seperti pembersihan data, konversi tipe, dan pengayaan, hingga memuat hasilnya ke dalam model data target yang semantik dan akurat. Model data akhir ini dirancang untuk dapat mendukung eksekusi kueri analitis yang kompleks.

Kebutuhan non-Fungsional menjadi inti dari penelitian ini, di mana performa sistem menjadi tolok ukur utama. Beberapa metrik performa utama yang diidentifikasi untuk diukur dan dibandingkan antara kedua arsitektur adalah:

- Waktu Proses ETL: Mengukur efisiensi dan kecepatan keseluruhan pipeline data, mulai dari pembacaan data sumber hingga data siap di dalam sistem target. Metrik ini krusial untuk mengevaluasi seberapa cepat sistem dapat menyerap dan mengolah data baru.
- Performa Kueri (QPS - *Query Per Second*): Mengukur kecepatan sistem dalam menyajikan data untuk kebutuhan analisis. Metrik ini mensimulasikan beban kerja dari pengguna atau aplikasi *Business Intelligence* (BI) yang melakukan kueri ad-hoc.
- Efisiensi Penyimpanan: Menganalisis jumlah ruang penyimpanan fisik yang dibutuhkan oleh masing-masing arsitektur untuk menyimpan volume data yang sama. Metrik ini berkaitan erat dengan biaya operasional dan infrastruktur.

Kebutuhan Data dan Teknologi menetapkan bahwa penelitian akan menggunakan dataset publik berskala besar, yaitu tumpukan data (*data dump*) Stack Overflow berformat .mdf dengan ukuran awal sekitar 50GB untuk *Data Warehouse*, dan data (*data dump*) Stack Overflow berformat XML untuk *Data Lake*. Untuk mengimplementasikan kedua arsitektur secara terpisah, digunakan dua tumpukan teknologi yang berbeda. Arsitektur *Data Warehouse* diimplementasikan menggunakan Microsoft SQL Server sebagai RDBMS dan Pentaho Data Integration (PDI) sebagai tools ETL. Sementara itu, arsitektur *Data Lake* diimplementasikan menggunakan MinIO sebagai *object storage* (Hub) dengan proses ETL yang dibangun menggunakan Python dan DuckDB sebagai mesin pemrosesan datanya.

4.2 Pengumpulan Data

Tahap pengumpulan data merupakan langkah esensial yang berfokus pada pemerolehan dataset yang representatif untuk pengujian dan perbandingan performa kedua arsitektur. Dataset Stack Overflow dipilih sebagai objek utama penelitian karena skalanya yang besar dan kompleksitasnya yang mencerminkan skenario data dunia nyata.

Untuk menciptakan perbandingan yang adil namun tetap menyoroti perbedaan penanganan data, penelitian ini mengadopsi pendekatan dua sumber dengan konten yang identik namun format yang berbeda:

1. Dataset untuk *Data Warehouse*

Data untuk arsitektur ini diperoleh dari blog pribadi Brent Ozar, yang menyediakan data Stack Overflow dalam format database Microsoft SQL Server (.mdf) dengan total ukuran sekitar 50GB. Format ini dipilih karena merepresentasikan skenario umum di mana *Data Warehouse* mengekstrak data dari sistem database operasional yang sudah terstruktur. File .mdf ini dapat langsung di-attach ke dalam RDBMS SQL Server 2019.

2. Dataset untuk *Data Lake*

Data untuk arsitektur ini diperoleh dari *Internet Archive* (archive.org), yang menyediakan data serupa namun dalam format mentah XML. Format semi-terstruktur ini dipilih untuk

mensimulasikan tantangan yang sering dihadapi dalam lingkungan *Data Lake*, yaitu kebutuhan untuk menangani data dari berbagai format non-relasional.

Penting untuk dicatat bahwa isi konten dari kedua dataset ini adalah serupa (identik), namun perbedaan utama terletak pada format berkasnya. Hal ini memungkinkan untuk secara valid membandingkan efisiensi dan performa *Data Lake* dan *Data Warehouse* dalam mengelola jenis data yang sama namun dengan representasi format yang berbeda. Struktur data yang digunakan disajikan dalam Tabel 4.3.

Tabel 4.3. Struktur data dbo.Comments

No.	Column_Name	Data_Type	Max_Character	Is_Nullable	Primary_Key
1	Id	INT	NULL	NO	YES
2	CreationDate	DATETIME	NULL	NO	NO
3	PostId	INT	NULL	NO	NO
4	Score	INT	NULL	YES	NO
5	Text	NVARCHAR	700	NO	NO
6	UserId	INT	NULL	YES	NO

Tabel Comments mempunyai kolom Id, CreationDate, PostId, Score, Text, dan UserId yang berisi sebanyak 24534730 baris data. Selanjutnya ada tabel Posts dengan struktur data yang disajikan dalam Tabel 4.4.

Tabel 4.4. Struktur data dbo.Posts

No.	Column_Name	Data_Type	Max_Character	Is_Nullable	Primary_Key
1	Id	INT	NULL	NO	YES
2	AcceptedAnswerId	INT	NULL	YES	NO
3	AnswerCount	INT	NULL	YES	NO
4	Body	NVARCHAR	-1	NO	NO
5	ClosedDate	DATETIME	NULL	YES	NO
6	CommentCount	INT	NULL	YES	NO
7	CommunityOwnedDate	DATETIME	NULL	YES	NO
8	CreationDate	DATETIME	NULL	NO	NO
9	FavoriteCount	INT	NULL	YES	NO
10	LastActivityDate	DATETIME	NULL	NO	NO
11	LastEditDate	DATETIME	NULL	YES	NO
12	LastEditorDisplayName	NVARCHAR	40	YES	NO
13	LastEditorUserId	INT	NULL	YES	NO
14	OwnerUserId	INT	NULL	YES	NO
15	ParentId	INT	NULL	YES	NO
16	PostTypeId	INT	NULL	NO	NO
17	Score	INT	NULL	NO	NO
18	Tags	NVARCHAR	150	YES	NO
19	Title	NVARCHAR	250	YES	NO
20	ViewCount	INT	NULL	NO	NO

Tabel Posts mempunyai kolom Id yang berperan sebagai *Primary Key* pada tabel ini. Tabel Posts memiliki 20 kolom dan mempunyai sebanyak 17142169 baris data. Selanjutnya terdapat tabel Users dengan struktur data yang disajikan dalam Tabel 4.5.

Tabel 4.5. Struktur data dbo.Users

No.	Column_Name	Data_Type	Max_Character	Is_Nullable	Primary_Key
1	Id	INT	NULL	NO	YES
2	AboutMe	NVARCHAR	-1	YES	NO
3	Age	INT	NULL	YES	NO
4	CreationDate	DATETIME	NULL	NO	NO
5	DisplayName	NVARCHAR	40	NO	NO
6	DownVotes	INT	NULL	NO	NO
7	EmailHash	NVARCHAR	40	YES	NO
8	LastAccessDate	DATETIME	NULL	NO	NO
9	Location	NVARCHAR	100	YES	NO
10	Reputation	INT	NULL	NO	NO
11	UpVotes	INT	NULL	NO	NO
12	Views	INT	NULL	NO	NO
13	WebsiteUrl	NVARCHAR	200	YES	NO
14	AccountId	INT	NULL	YES	NO

Tabel Users mempunyai kolom Id yang berperan sebagai *Primary Key* pada tabel ini. Tabel Users memiliki 14 kolom dan mempunyai sebanyak 2465713 baris data. Selanjutnya terdapat tabel Votes dengan struktur data yang disajikan dalam Tabel 4.6.

Tabel 4.6. Struktur data dbo.Votes

No.	Column_Name	Data_Type	Max_Character	Is_Nullable	Primary_Key
1	Id	INT	NULL	NO	YES
2	PostId	INT	NULL	NO	NO
3	UserId	INT	NULL	YES	NO
4	BountyAmount	INT	NULL	YES	NO
5	VoteTypeId	INT	NULL	NO	NO
6	CreationDate	DATETIME	NULL	NO	NO

Tabel Votes mempunyai kolom Id, PostId, UserId, BountyAmount, VoteTypeId, dan CreationDate yang berisi sebanyak 52928720 baris data.

Perbedaan fundamental dalam format sumber ini memungkinkan penelitian untuk mengevaluasi tidak hanya performa pemrosesan data, tetapi juga kompleksitas pada tahap ekstraksi awal untuk masing-masing arsitektur. Dari keseluruhan dataset tersebut, empat entitas utama diekstrak untuk membangun model data. Pemilihan ini didasarkan pada relevansinya dalam merepresentasikan aktivitas inti platform:

- Users: Data master pengguna yang berisi lebih dari 2.4 juta record.
- Posts: Data transaksi utama yang berisi lebih dari 17 juta record pertanyaan dan jawaban.
- Comments: Data transaksi interaksi yang berisi lebih dari 24 juta record.
- Votes: Data transaksi interaksi dengan volume tertinggi, berisi lebih dari 52 juta record.

Untuk menjaga agar proses eksperimen dapat dikelola dalam batasan waktu penelitian, dilakukan pemfilteran data berdasarkan rentang waktu. Hanya data dengan kolom CreationDate antara tanggal 31 Juli 2008 hingga 31 Desember 2013 yang digunakan. Sebagai langkah pra-pemrosesan akhir, khususnya untuk arsitektur *Data Lake*, data sumber XML dikonversi menjadi format file Apache Parquet. Format berbasis kolom ini dipilih karena efisiensi

penyimpanan dan kecepatan aksesnya yang superior, yang kemudian diakses melalui *mount Rclone* untuk proses ETL selanjutnya.

4.3 Data Warehouse

Setelah dilakukan identifikasi kebutuhan, selanjutnya akan dilakukan pembangunan *Data Warehouse* sesuai kebutuhan. Implementasi arsitektur pertama difokuskan pada pembangunan *Data Warehouse* tradisional dengan Microsoft SQL Server. Pembangunan ini dilakukan dengan mengikuti prinsip 4-langkah dari Metodologi Kimball untuk merancang sebuah skema bintang (*star schema*) yang dioptimalkan untuk kueri analitis. Setiap langkah mulai dari pemilihan proses bisnis hingga identifikasi fakta dijelaskan secara detail pada sub-bab berikut.

4.3.1 Pemilihan Proses Bisnis

Langkah pertama dan paling fundamental dalam metodologi Kimball adalah pemilihan proses bisnis yang akan dimodelkan. Berdasarkan sifat data Stack Overflow yang merupakan rekaman interaksi, proses bisnis yang dipilih adalah Analisis Aktivitas Pengguna. Pemilihan proses ini didasarkan pada tujuan untuk dapat menjawab pertanyaan-pertanyaan analitis yang krusial bagi pemahaman platform, seperti mengidentifikasi pengguna paling berpengaruh, memahami tren keterlibatan pengguna dari waktu ke waktu, serta menganalisis topik atau jenis postingan yang paling banyak menarik interaksi. Dengan fokus pada aktivitas pengguna, model data yang dihasilkan dapat secara langsung mendukung kebutuhan analisis tersebut.

4.3.2 Penentuan Granularitas

Setelah proses bisnis ditentukan, langkah krusial selanjutnya adalah mendefinisikan granularitas atau *grain* dari tabel fakta. *Grain* menentukan tingkat kedetailan terendah yang direpresentasikan oleh satu baris tunggal di dalam tabel fakta. Untuk proses Analisis Aktivitas Pengguna, granularitas yang ditetapkan adalah tingkat transaksional, di mana:

Satu baris di dalam tabel fakta merepresentasikan satu peristiwa (event) aktivitas tunggal yang terjadi

Sebagai contoh, jika seorang pengguna membuat satu postingan dan memberikan tiga upvote, maka akan tercatat sebanyak empat baris terpisah di dalam tabel *fact_user_activity*. Pemilihan grain pada level atomik ini memberikan fleksibilitas tertinggi untuk analisis. Dengan grain ini, agregasi untuk menjawab hampir semua pertanyaan bisnis—seperti menghitung jumlah vote per hari, jumlah komentar per pengguna, atau metrik transaksional lainnya—dapat dilakukan dengan COUNT atau SUM sederhana pada tabel fakta, tanpa perlu mendesain ulang tabel di masa depan.

4.3.3 Identifikasi Dimensi

Setelah granularitas ditetapkan, langkah selanjutnya adalah mengidentifikasi dimensi-dimensi yang akan digunakan untuk menganalisis fakta. Dimensi menyediakan konteks deskriptif "siapa, apa, kapan, dan bagaimana" dari sebuah peristiwa bisnis yang tercatat di tabel fakta. Sebelum merancang setiap dimensi, penting untuk menentukan strategi penanganan perubahan data, yang dikenal sebagai *Slowly Changing Dimension* (SCD). Dalam penelitian ini, digunakan SCD Tipe 1, di mana setiap perubahan data pada sumber akan langsung menimpa (*overwrite*) data lama di tabel dimensi. Pendekatan ini dipilih karena kesederhanaannya dan fokus penelitian yang lebih menitikberatkan pada perbandingan performa daripada pelacakan histori data.

Berdasarkan proses bisnis Analisis Aktivitas Pengguna, diidentifikasi empat dimensi utama sebagai berikut:

1. **dim_time**

Dimensi ini menjawab konteks "kapan" sebuah aktivitas terjadi dan memungkinkan analisis data secara temporal. Dimensi ini tidak berasal dari tabel sumber, melainkan dibuat secara prosedural oleh proses ETL untuk mencakup seluruh rentang waktu data penelitian

Tabel 4.7. Struktur tabel dim_time

Column	Description	Datatype	Size	SCD
TimeKey (PK)	Surrogate key	INTEGER	-	SCD 0
FullDate	Tanggal lengkap (YYYY-MM-DD)	DATE	-	SCD 1
Year	Tahun	INTEGER	-	SCD 1
Month	Bulan dalam angka (1-12)	INTEGER	-	SCD 1
Day	Hari dalam bulan (1-31)	INTEGER	-	SCD 1
Weekday	Hari dalam seminggu (angka)	INTEGER	-	SCD 1

2. **dim_user**

Dimensi ini menjawab konteks "siapa" yang melakukan aktivitas. Dimensi ini berisi atribut-atribut yang mendeskripsikan pengguna, seperti nama, lokasi, dan reputasi. Untuk menjaga integritas dan performa, UserKey digunakan sebagai *surrogate key* yang unik dan tidak bergantung pada sistem sumber.

Tabel 4.8. Struktur tabel dim_user

Column	Description	Datatype	Size	SCD
UserKey (PK)	Surrogate key	INTEGER	-	SCD 0
UserId_Original	Natural key dari sistem sumber	INTEGER	-	SCD 0
DisplayName	Nama tampilan pengguna	NVARCHAR	100	SCD 1
Location	Lokasi yang ditulis pengguna	NVARCHAR	100	SCD 1
CreationDate	Tanggal akun pengguna dibuat	DATETIME	-	SCD 1
Reputation	Nilai reputasi pengguna	INTEGER	-	SCD 1

3. dim_post

Dimensi ini memberikan konteks "apa" yang menjadi objek dari aktivitas, khususnya untuk postingan. Atribut utamanya mencakup judul dan tipe postingan untuk membedakan antara pertanyaan dan jawaban.

Tabel 4.9. Struktur tabel dim_post

Column	Description	Datatype	Size	SCD
PostKey (PK)	Surrogate key	INTEGER	-	SCD 0
PostId_Original	Natural key dari sistem sumber	INTEGER	-	SCD 0
Title	Judul postingan	NVARCHAR	255	SCD 1
PostType	Tipe postingan (e.g., 'Question', 'Answer')	NVARCHAR	50	SCD 1

4. dim_activity_type

Dimensi ini menjawab konteks "jenis aktivitas apa" yang terjadi. Dimensi ini merupakan sebuah tabel lookup buatan yang secara eksplisit mengklasifikasikan setiap peristiwa ke dalam kategori yang mudah dipahami (contoh: 'Create Post', 'Upvote'), sehingga mempermudah proses analisis dan pelaporan.

Tabel 4.10. Struktur tabel dim_activity_type

Column	Description	Datatype	Size	SCD
ActivityTypeKey (PK)	Surrogate key	INTEGER	-	SCD 0
ActivityName	Nama deskriptif dari aktivitas	VARCHAR	50	SCD 1

4.3.4 Identifikasi Fakta

Tabel fakta merupakan pusat dari model bintang yang menyimpan ukuran-ukuran kuantitatif dari proses bisnis. Sesuai dengan granularitas yang telah ditentukan, tabel fact_user_activity dirancang sebagai Tabel Fakta Transaksional, yang berarti setiap barisnya menyimpan data dari satu peristiwa tunggal. Strukturnya terdiri dari dua jenis kolom utama:

1. *Foreign Keys*: Kumpulan kunci yang merujuk pada *primary key* di setiap tabel dimensi. Kolom-kolom ini adalah "lem" yang mengikat data kuantitatif di tabel fakta dengan data kontekstual di tabel dimensi. Kolom-kolom tersebut adalah:
 - TimeKey (merujuk ke dim_time)
 - UserKey (merujuk ke dim_user)
 - PostKey (merujuk ke dim_post)

- ActivityTypeKey (merujuk ke dim_activity_type)
2. *Measures*: Kolom-kolom numerik yang dapat diagregasi dan menjadi fokus dari analisis. *Measure* yang diidentifikasi untuk proses bisnis ini adalah:
- ActivityCount: Sebuah *fact* yang nilainya selalu 1, digunakan untuk menghitung total semua jenis peristiwa.
 - ScoreValue: Ukuran nilai atau skor yang dihasilkan dari sebuah aktivitas, terutama dari vote.
 - PostLength dan CommentLength: Ukuran panjang teks, hanya diisi saat aktivitasnya adalah membuat post atau komentar.
 - BountyAmount: Ukuran jumlah bounty yang diberikan pada aktivitas vote tertentu.

Tabel 4.11. Struktur tabel fact_user_activity

Column	Description	Datatype	Size	SCD
ActivityId (PK)	Surrogate key untuk tabel fakta	BIGINT	-	SCD 0
UserKey (FK)	Kunci yang terhubung ke dim_user	INTEGER	-	N/A
PostKey (FK)	Kunci yang terhubung ke dim_post	INTEGER	-	N/A
TimeKey (FK)	Kunci yang terhubung ke dim_time	INTEGER	-	N/A
ActivityTypeKey (FK)	Kunci yang terhubung ke dim_activity_type	INTEGER	-	N/A
ActivityCount	Measure untuk menghitung kejadian (selalu 1)	INTEGER	-	N/A
ScoreValue	Nilai skor dari sebuah aktivitas (e.g., vote)	INTEGER	-	N/A
PostLength	Panjang karakter dari sebuah postingan	INTEGER	-	N/A
CommentLength	Panjang karakter dari sebuah komentar	INTEGER	-	N/A
BountyAmount	Jumlah bounty pada sebuah aktivitas vote	INTEGER	-	N/A

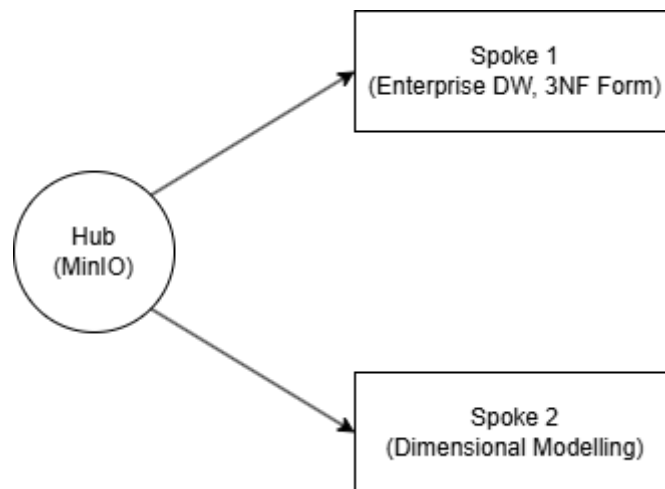
4.4 Data Lake

Arsitektur kedua yang dibangun sebagai pembanding adalah *Data Lake* modern. Pendekatan ini secara fundamental berbeda dari *Data Warehouse* tradisional. Jika *Data*

Warehouse menerapkan prinsip *schema-on-write*, di mana skema data yang kaku harus didefinisikan terlebih dahulu, *Data Lake* mengadopsi prinsip *schema-on-read*. Pada *Data Lake*, data disimpan dalam format mentah atau semi-terstruktur, dan skema baru diterapkan saat data tersebut akan dibaca atau dianalisis. Hal ini memberikan tingkat fleksibilitas dan skalabilitas yang superior, meskipun datang dengan tantangan tata kelola data tersendiri untuk menghindari fenomena *data swamp* (rawa data)

4.4.1 Arsitektur Hybrid Hub and Spoke

Untuk penelitian ini, arsitektur yang diimplementasikan adalah *Hybrid Hub-and-Spoke*. Arsitektur ini dipilih karena memberikan keseimbangan antara fleksibilitas penyimpanan data mentah dan kebutuhan akan data yang terkurasi dan terpercaya untuk analisis.



Gambar 4.1. Diagram arsitektur umum *Hybrid Hub-and-Spoke*

Sumber: Dokumentasi Pribadi

Arsitektur ini terdiri dari dua komponen utama:

1. *Hub* (Pusat Data)
Berfungsi sebagai repositori sentral dan *single source of truth*. Dalam implementasi ini, Hub adalah sebuah *bucket* di dalam server MinIO Object Storage. Semua data sumber yang telah difilter berdasarkan rentang waktu disimpan di sini dalam format Apache Parquet. Hub ini menjadi zona pendaratan (*landing zone*) dan arsip historis, menjaga keaslian dan detail data sumber.
2. *Spokes* (Cabang Analitis)
Berfungsi sebagai lapisan data terkurasi yang berasal dari *Hub*, dirancang khusus untuk kebutuhan analisis yang berbeda. Dengan memisahkan data olahan ke dalam *Spokes*, *Hub* tetap bersih dari data turunan dan tata kelola menjadi lebih mudah. Dua spoke diimplementasikan dalam penelitian ini:

4.4.2 Implementasi Spoke 1: Enterprise Data Warehouse (3NF)

Tujuan dari Spoke 1 adalah untuk menyediakan data yang bersih dan ternormalisasi hingga Bentuk Normal Ketiga (3NF). Model ini ideal untuk kueri operasional atau analisis yang membutuhkan integritas data tinggi dan minimalisasi redundansi. Proses ETL untuk *Spoke 1* difokuskan pada transformasi skema dari file sumber menjadi struktur 3NF. Langkah normalisasi utama adalah menghilangkan ketergantungan transitif, di mana sebuah atribut non-kunci bergantung pada atribut non-kunci lainnya. Proses ini diimplementasikan dengan skrip

DuckDB yang membaca file sumber dari *Hub*, lalu memilih hanya kolom-kolom yang relevan sambil membuang kolom deskriptif yang redundan.

Tabel 4.12. Proses normalisasi pada tabel Post

Kolom di Posts_final.parquet (Sumber)	Keterangan	Kolom di Posts_3nf.parquet (Target)
Id	Disimpan. Kunci utama postingan.	Id
AcceptedAnswerId	Disimpan. Kunci ke postingan lain (jawaban).	AcceptedAnswerId
AnswerCount	Disimpan. Atribut transaksional postingan.	AnswerCount
Body	Disimpan. Atribut deskriptif utama postingan.	Body
ClosedDate	Disimpan. Atribut status postingan.	ClosedDate
CommentCount	Disimpan. Atribut transaksional postingan.	CommentCount
CommunityOwnedDate	Disimpan. Atribut status postingan.	CommunityOwnedDate
ContentLicense	Disimpan. Atribut metadata postingan.	ContentLicense
CreationDate	Disimpan. Atribut tanggal postingan.	CreationDate
FavoriteCount	Disimpan. Atribut transaksional postingan.	FavoriteCount
LastActivityDate	Disimpan. Atribut tanggal postingan.	LastActivityDate
LastEditDate	Disimpan. Atribut tanggal postingan.	LastEditDate
LastEditorDisplayName	Redundan (Dihilangkan). Nama bisa didapat dari Users via LastEditorUserId.	-
LastEditorUserId	Disimpan. Kunci ke pengguna (editor terakhir).	LastEditorUserId
OwnerDisplayName	Redundan (Dihilangkan). Nama bisa didapat dari Users via OwnerUserId.	-
OwnerUserId	Disimpan. Kunci ke pengguna (pemilik).	OwnerUserId
ParentId	Disimpan. Kunci ke postingan lain (pertanyaan).	ParentId
PostTypeId	Disimpan. Kunci ke tipe postingan.	PostTypeId
Score	Disimpan. Atribut transaksional postingan.	Score
Tags	Disimpan. Atribut deskriptif postingan.	Tags
Title	Disimpan. Atribut deskriptif utama postingan.	Title
ViewCount	Disimpan. Atribut transaksional postingan.	ViewCount

Tabel Posts masih belum memenuhi kaidah 3NF, sehingga perlu dilakukan proses normalisasi. Proses normalisasi ini bertujuan agar atribut yang bukan Primary Key tidak memiliki ketergantungan transitif pada Primary Key. Pada tabel Posts ini, atribut LastEditorDisplayName dan OwnerDisplayName akan dihilangkan. Penghapusan kolom ini karena kedua atribut tersebut bisa didapat dari LastEditorUserId dan OwnerUserId. Selanjutnya terdapat tabel Votes dengan struktur data yang disajikan dalam Tabel 4.13.

Tabel 4.13. Proses normalisasi pada tabel Comments

Kolom di Comments_final.parquet (Sumber)	Keterangan	Kolom di Comments_3nf.parquet (Target)
Id	Disimpan. Kunci utama komentar.	Id
PostId	Disimpan. Kunci ke postingan.	PostId
Score	Disimpan. Atribut transaksional komentar.	Score
Text	Disimpan. Atribut deskriptif utama komentar.	Text
CreationDate	Disimpan. Atribut tanggal komentar.	CreationDate
UserId	Disimpan. Kunci ke pengguna.	UserId
UserDisplayName	Redundan (Dihilangkan). Nama bisa didapat dari Users via UserId.	-

Serupa dengan tabel Posts sebelumnya, pada tabel Comments ini masih belum memenuhi kaidah 3NF dimana atribut UserDisplayName bisa didapatkan dari UserId. Oleh karena itu, kolom UserDisplayName akan dihilangkan. Selanjutnya terdapat tabel Users dengan struktur data yang disajikan dalam Tabel 4.14.

Tabel 4.14. Proses normalisasi pada tabel Users

Kolom di Users_final.parquet (Sumber)	Keterangan	Kolom di Users_3nf.parquet (Target)
Id	Disimpan. Kunci utama pengguna.	Id
Reputation	Disimpan. Atribut pengguna.	Reputation
CreationDate	Disimpan. Atribut pengguna.	CreationDate
DisplayName	Disimpan. Atribut pengguna.	DisplayName
LastAccessDate	Disimpan. Atribut pengguna.	LastAccessDate
WebsiteUrl	Disimpan. Atribut pengguna.	WebsiteUrl
Location	Disimpan. Atribut pengguna.	Location
AboutMe	Disimpan. Atribut pengguna.	AboutMe
Views	Disimpan. Atribut pengguna.	Views
UpVotes	Disimpan. Atribut pengguna.	UpVotes
DownVotes	Disimpan. Atribut pengguna.	DownVotes
AccountId	Disimpan. Atribut pengguna.	AccountId

Tabel Users memenuhi kaidah 3NF dimana semua atributnya tidak mempunyai ketergantungan transitif dengan Primary Key. Oleh karena itu, tidak perlu dilakukan proses

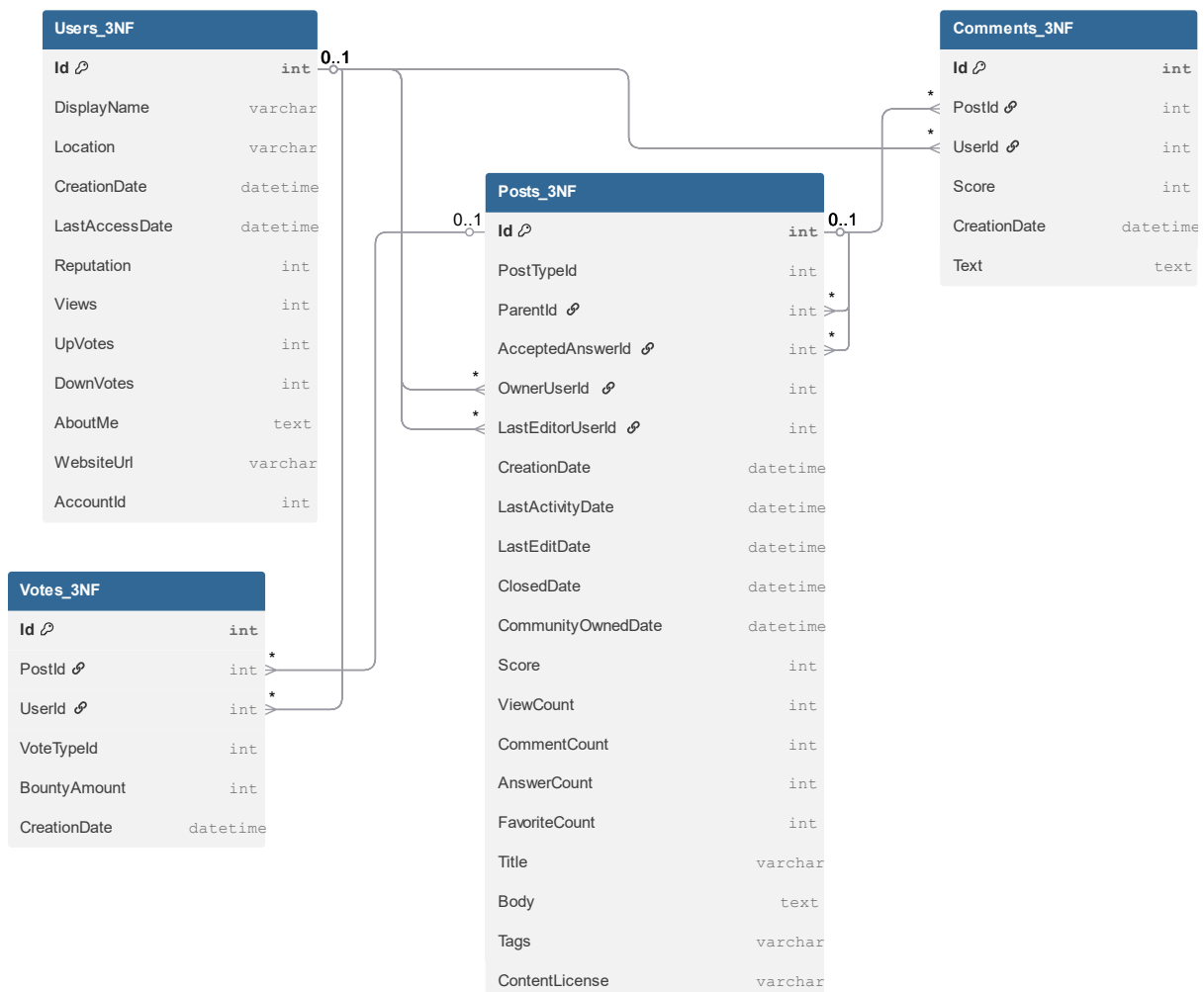
normalisasi pada tabel ini. Selanjutnya terdapat tabel Votes dengan struktur data yang disajikan dalam Tabel 4.15.

Tabel 4.15. Proses normalisasi pada tabel Votes

Kolom di Votes_final.parquet (Sumber)	Keterangan	Kolom di Votes_3nf.parquet (Target)
Id	Disimpan. Kunci utama vote.	Id
PostId	Disimpan. Kunci ke postingan.	PostId
UserId	Disimpan. Kunci ke pengguna.	UserId
BountyAmount	Disimpan. Atribut transaksional vote.	BountyAmount
VoteTypeId	Disimpan. Kunci ke tipe vote.	VoteTypeId
CreationDate	Disimpan. Atribut tanggal vote.	CreationDate

Serupa dengan tabel Users sebelumnya, tabel Votes ini memenuhi kaidah 3NF dimana semua atribut di tabel ini tidak mempunyai ketergantungan transitif dengan Primary Key. Oleh karena itu, tidak perlu dilakukan proses normalisasi pada tabel ini.

Hasil akhir dari proses ini adalah satu set file Parquet baru (Users_3nf.parquet, Posts_3nf.parquet, dll.) yang secara kolektif membentuk sebuah model relasional 3NF. File-file ini kemudian disimpan di dalam folder spoke1_edw3nf di MinIO.



Gambar 4.2. Diagram ERD untuk Spoke 1 (3NF)
Sumber: Dokumentasi Pribadi

4.4.3 Implementasi Spoke 2: Dimensional Model

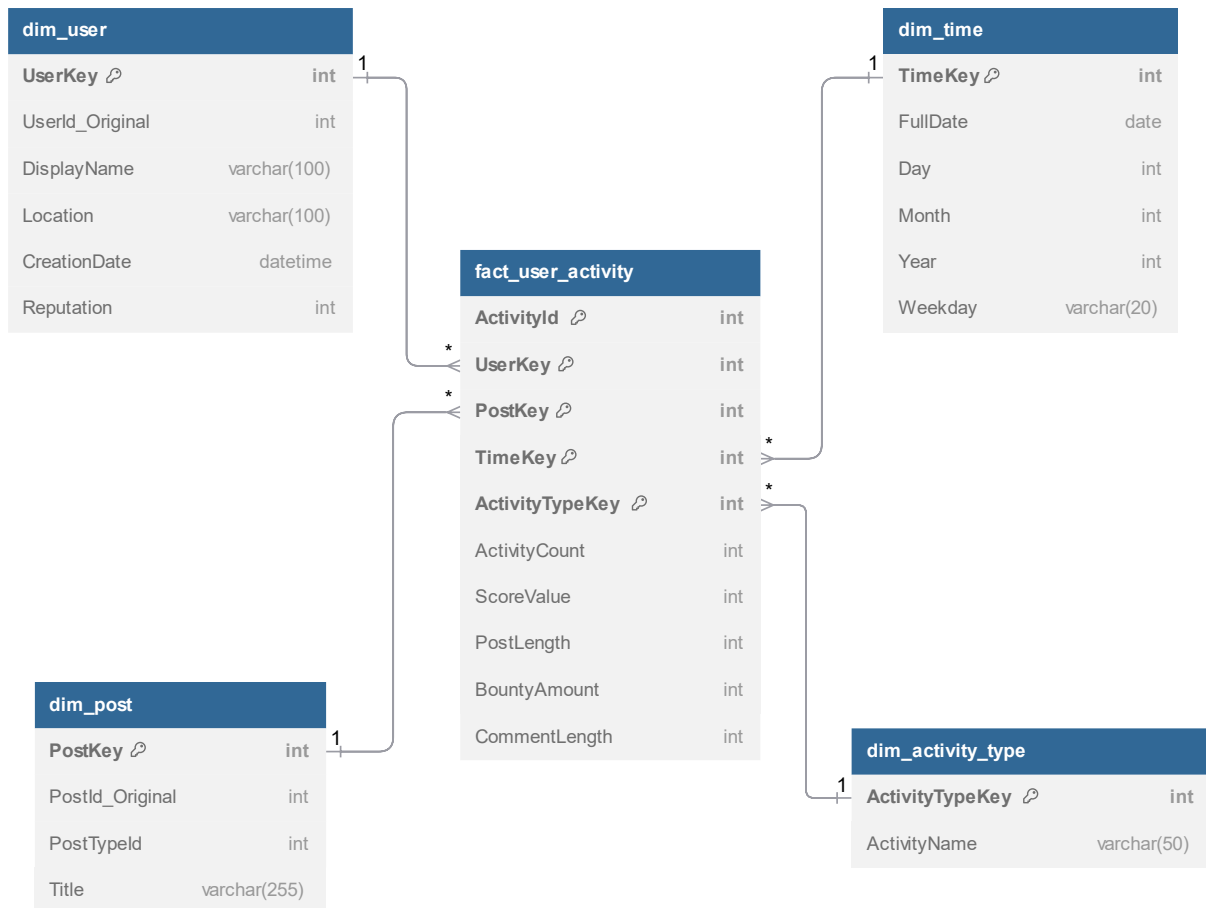
Tujuan utama dari Spoke 2 adalah untuk membangun sebuah star schema yang secara struktur identik dengan yang dibangun di Data Warehouse SQL Server. Hal ini krusial untuk menciptakan skenario perbandingan performa kueri yang adil dan seimbang (*apple-to-apple*) antara kedua ekosistem teknologi.

Berbeda dengan pendekatan lain, proses ETL untuk spoke ini sepenuhnya dijalankan secara native di dalam DuckDB. Seluruh proses, mulai dari membaca berbagai file Parquet sumber, melakukan transformasi, hingga menyimpan hasilnya, dieksekusi menggunakan satu atau serangkaian perintah SQL yang kuat. Pendekatan ini menyoroti kapabilitas DuckDB sebagai mesin pemrosesan analitis dan ETL yang mandiri.

Alur kerjanya adalah sebagai berikut:

1. Ekstraksi dan Transformasi Terintegrasi
Menggunakan kueri SQL tunggal yang kompleks, DuckDB diperintahkan untuk membaca beberapa file Parquet secara bersamaan (misalnya, `Posts.parquet`, `Users.parquet`, `Comments.parquet`, dll.).
2. Logika Transformasi dalam SQL
Semua logika transformasi termasuk JOIN antar tabel, CAST tipe data, GROUP BY, dan pembuatan kolom baru didefinisikan langsung di dalam kueri SQL tersebut. Ini memaksimalkan efisiensi karena seluruh proses terjadi di dalam engine DuckDB yang teroptimasi.
3. Pemuatan (*Load*)
Hasil akhir dari kueri yang telah ditransformasi tersebut kemudian diekspor kembali menjadi satu set file Parquet baru yang merepresentasikan tabel fakta dan dimensi. Proses ini dilakukan dengan membungkus kueri SELECT di dalam perintah `COPY ... TO ...`, yang secara efisien menulis hasilnya ke object storage MinIO (melalui mount Rclone).

Pendekatan ini secara efektif menggunakan DuckDB tidak hanya sebagai *query engine*, tetapi juga sebagai ETL tool yang mampu menangani transformasi data skala besar secara mandiri.



Gambar 4.3. Diagram star schema untuk Spoke 2

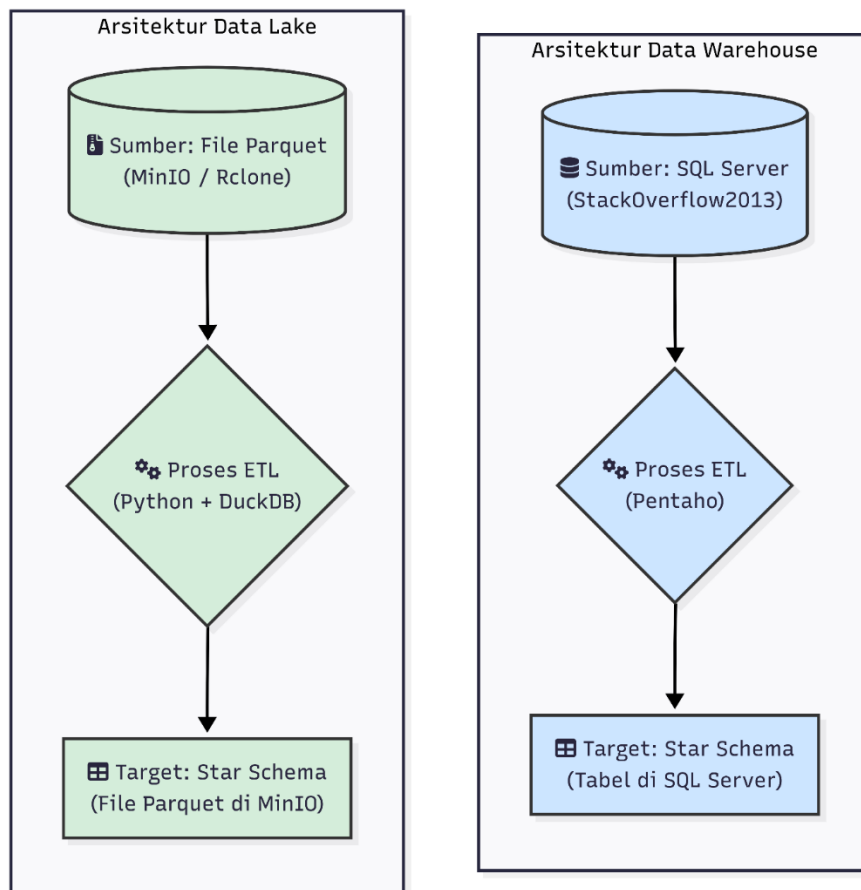
Sumber: Dokumentasi Pribadi

4.5 Extract, Transform, Load (ETL)

Proses *Extract, Transform, Load* (ETL) merupakan jantung dari implementasi teknis dalam penelitian ini. Tahapan ini berfungsi sebagai *pipeline* yang bertanggung jawab untuk memindahkan data dari sistem sumber, mengubahnya menjadi format dan struktur yang sesuai dengan model data yang telah dirancang, dan memuatnya ke dalam sistem target untuk dianalisis. Meskipun tujuan akhirnya serupa, strategi dan implementasi ETL untuk arsitektur *Data Warehouse* dan *Data Lake* menunjukkan perbedaan fundamental yang akan diuraikan pada bagian berikut.

4.5.1 High Level ETL Plan

Untuk memberikan gambaran yang jelas mengenai keseluruhan alur kerja, disusun sebuah rencana ETL tingkat tinggi (*High Level ETL Plan*). Rencana ini menguraikan tahapan-tahapan utama dalam proses ETL dan membandingkan bagaimana setiap tahapan diimplementasikan pada arsitektur *Data Warehouse* dan *Data Lake*. Berikut adalah gambar mengenai *High Level ETL Plan*.



Gambar 4.4. *High Level ETL Plan*

Sumber: Dokumentasi Pribadi

Rencana ETL dibagi menjadi empat tahapan utama sebagai berikut:

1. Tahap Ekstraksi (*Extract*)

- Data Warehouse (Pentaho)

Proses ekstraksi dilakukan menggunakan step *Table Input* yang terhubung ke database sumber melalui JDBC. Kueri SQL dieksekusi langsung pada server, memungkinkan pemfilteran dan penggabungan awal terjadi di sisi database.

- Data Lake (DuckDB)

Proses ekstraksi dilakukan langsung oleh DuckDB dengan menjalankan fungsi `read_parquet()` pada file Parquet yang tersimpan di sistem file lokal (yang di-mount dari MinIO). Pendekatan ini menunjukkan kemampuan query engine untuk mengakses data berbasis file secara langsung tanpa perantara.

2. Tahap Transformasi Dimensi (*Dimension Transformation*)

- Data Warehouse (Pentaho)

Transformasi untuk memuat tabel dimensi diimplementasikan menggunakan pola manual SCD Tipe 1 yang melibatkan serangkaian step seperti *Database Lookup* dan *Filter Rows*. Proses ini bersifat sangat interaktif dengan database, di mana Pentaho mengirimkan banyak kueri individual untuk setiap baris data.

- Data Lake (DuckDB)

Seluruh proses transformasi dimensi dilakukan secara *native* di dalam DuckDB menggunakan kueri SQL. Proses *lookup* dan penggabungan data dilakukan dengan operasi JOIN langsung antar file Parquet, yang sangat efisien karena terjadi sepenuhnya di dalam engine DuckDB.

3. Tahap Transformasi Fakta (*Fact Transformation*)

- Data Warehouse (Pentaho)

Untuk mengatasi keterbatasan sumber daya sistem saat menangani beberapa alur data besar secara bersamaan, arsitektur yang berbeda diterapkan. Setiap jenis aktivitas fakta diolah dalam *transformation* terpisah (ETL_fact_posts.ktr, ETL_fact_comments.ktr, dll.). Pendekatan ini memecah satu pekerjaan besar menjadi beberapa pekerjaan lebih kecil yang kemudian dijalankan secara berurutan oleh Job Utama, sehingga lebih hemat memori.

- Data Lake (DuckDB)

Serupa dengan pendekatan *Data Warehouse*, setiap jenis aktivitas fakta juga diolah sebagai proses terpisah. Setiap proses dieksekusi oleh sebuah skrip SQL yang didedikasikan untuk satu jenis aktivitas (misalnya, satu skrip untuk memproses Posts, satu skrip untuk Comments), yang dijalankan secara berurutan untuk menjaga efisiensi memori.

4. Tahap Pemuatan dan Orkestrasi (Load and Orchestration)

- Data Warehouse (Pentaho)

Orkestrasi atau urutan eksekusi proses ETL dilakukan secara manual. Setiap file *transformation* (.ktr) dijalankan satu per satu langsung dari antarmuka Pentaho Data Integration. Proses dimulai dengan menjalankan *transformation* untuk mengosongkan tabel fakta, diikuti oleh eksekusi berurutan dari setiap *transformation* untuk memuat tabel dimensi (ETL_dim_time, ETL_dim_user, dst.). Setelah semua dimensi berhasil dimuat, setiap *transformation* untuk alur fakta (ETL_fact_posts, ETL_fact_comments, ETL_fact_votes) juga dijalankan secara terpisah hingga semua data berhasil dimuat. Pendekatan ini memastikan dependensi data tetap terpenuhi meskipun tidak menggunakan file Job (.kjb) terpusat.

- Data Lake (DuckDB)

Orkestrasi juga dilakukan secara manual dengan menjalankan setiap skrip SQL satu per satu langsung di dalam *command-line interface* (CLI) DuckDB. Urutan eksekusi dijaga secara manual, dimulai dari skrip untuk membuat tabel dimensi, diikuti oleh skrip untuk setiap tabel fakta, untuk memastikan dependensi data terpenuhi.

4.5.2 Pemuatan Data Historis (*One-Time Historical Load*)

Proses ETL yang diimplementasikan dalam penelitian ini adalah jenis pemuatan data historis satu kali (*One-Time Historical Load*). Secara teknis, pendekatan ini juga dikenal sebagai *Full Load* atau *Full Refresh*, di mana keseluruhan *data warehouse* dibangun ulang dari awal pada setiap eksekusi. Pendekatan ini secara fundamental berbeda dengan ETL Inkremental yang hanya memuat data baru atau data yang berubah sejak eksekusi terakhir. Implementasi *one-time historical load* ini terlihat jelas dari beberapa karakteristik utama dalam alur kerja yang telah dibangun, baik pada arsitektur *Data Warehouse* maupun *Data Lake*:

1. Pembersihan Data Target di Awal

Alur kerja utama yang diorkestrasi oleh Job Pentaho selalu diawali dengan langkah untuk mengosongkan tabel fakta (DELETE FROM fact_user_activity). Demikian pula, pada setiap proses pemuatan tabel dimensi, opsi *Truncate table* diaktifkan untuk memastikan tabel dimensi selalu dimulai dari keadaan kosong. Ini adalah ciri khas dari proses *full refresh*.

2. Pembacaan Data Sumber Secara Penuh

Setiap transformation membaca keseluruhan data sumber yang berada dalam ruang lingkup yang telah ditentukan (data dari tahun 2008 hingga 2013). Tidak ada logika yang diimplementasikan untuk hanya memilih data yang berubah atau data yang baru ditambahkan sejak eksekusi sebelumnya.

3. Tidak Adanya Mekanisme Pelacakan (*Tracking*)

Desain ETL ini tidak menggunakan teknik *incremental* seperti *high watermarking* (menyimpan tanggal eksekusi terakhir) atau *Change Data Capture* (CDC). Seluruh proses dirancang untuk bersifat independen dari eksekusi sebelumnya.

Pemilihan metode *one-time historical load* ini sangat sesuai dengan tujuan utama penelitian. Dengan memuat puluhan juta baris data historis dalam satu kali proses, kita dapat menciptakan beban kerja (*workload*) yang signifikan dan realistis. Beban kerja inilah yang digunakan untuk menguji dan mengukur metrik-metrik performa utama seperti total waktu eksekusi ETL dan penggunaan sumber daya sistem (memori, disk) secara *end-to-end*. Pendekatan ini menciptakan sebuah *baseline* pengujian yang adil, di mana kedua arsitektur (*Data Warehouse* dan *Data Lake*) dihadapkan pada tantangan yang sama persis untuk dibandingkan secara objektif.

4.5.3 Spesifikasi Pemetaan Data (*Source-to-Target Mapping*)

Sub-bab ini menyajikan spesifikasi pemetaan data secara rinci dari sistem sumber ke sistem target (*Data Warehouse*). Pemetaan ini, sering disebut sebagai *Source-to-Target Mapping* (STM), berfungsi sebagai "cetak biru" bagi proses ETL, yang mendokumentasikan asal-usul setiap kolom di dalam *Data Warehouse* dan aturan transformasi yang diterapkan padanya.

Tabel 4.16. Pemetaan tabel dim_activity_type

Source table	Source column	Target table	Target column
Proses ETL	Didefinisikan di step Data Grid	dim_activity_type	ActivityTypeKey (PK)
Proses ETL	Didefinisikan di step Data Grid	dim_activity_type	ActivityName

Pada tabel dim_time, seluruh kolomnya dihasilkan melalui proses ETL. Step Data Grid memainkan peran penting dalam pengisian kolom dari tabel ini. Selanjutnya terdapat pemetaan tabel dim_time yang disajikan dalam Tabel 4.17

Tabel 4.17. Pemetaan tabel dim_time

Source table	Source column	Target table	Target column
Proses ETL	Dihasilkan oleh step Add sequence	dim_time	Time Key (PK)
Proses ETL	Dihasilkan oleh step Calculator	dim_time	FullDate
Proses ETL	Dihasilkan dari FullDate	dim_time	Year

Proses ETL	Dihasilkan dari FullDate	dim_time	Month
Proses ETL	Dihasilkan dari FullDate	dim_time	Day
Proses ETL	Dihasilkan dari FullDate	dim_time	Weekday

Pada tabel dim_time, seluruh kolomnya dihasilkan melalui proses ETL. Step add sequence dan calculator memainkan peran penting dalam pengisian kolom dari tabel ini. Selanjutnya terdapat pemetaan tabel dim_user yang disajikan dalam Tabel 4.18

Tabel 4.18. Pemetaan tabel dim_user

Source table	Source column	Target table	Target column
Proses ETL	Dihasilkan oleh proses SCD	dim_user	UserKey (PK)
dbo.Users	Id	dim_user	UserId_Original
dbo.Users	DisplayName	dim_user	DisplayName
dbo.Users	Location	dim_user	Location
dbo.Users	CreationDate	dim_user	CreationDate
dbo.Users	Reputation	dim_user	Reputation

Pada tabel dim_user, UserKey merupakan surrogate key yang dihasilkan oleh proses SCD di ETL. Untuk kolom lain dihasilkan dari tabel Users yang ada di database StackOverflow2013. Selanjutnya terdapat pemetaan tabel dim_post yang disajikan dalam Tabel 4.19

Tabel 4.19. Pemetaan tabel dim_post

Source table	Source column	Target table	Target column
Proses ETL	Dihasilkan oleh proses SCD	dim_post	PostKey (PK)
dbo.Posts	Id	dim_post	PostId_Original
dbo.Posts	Title	dim_post	Title
dbo.PostTypes	Type	dim_post	PostType

Pada tabel dim_post, PostKey merupakan surrogate key yang dihasilkan oleh proses SCD di ETL. Untuk kolom lain dihasilkan dari tabel Posts yang ada di database StackOverflow2013. Selanjutnya terdapat pemetaan tabel fact_user_activity yang disajikan dalam Tabel 4.20

Tabel 4.20. Pemetaan tabel fact_user_activity

Source table	Source column	Target table	Target column
SQL Server	IDENTITY(1,1)	fact_user_activity	ActivityId (PK)
Hasil Lookup	dim_user.UserKey	fact_user_activity	UserKey (FK)
Hasil Lookup	dim_post.PostKey	fact_user_activity	PostKey (FK)
Hasil Lookup	dim_time.TimeKey	fact_user_activity	TimeKey (FK)
Hasil Lookup	dim_activity_type.ActivityTypeKey	fact_user_activity	ActivityTypeKey (FK)
Proses ETL	Diberi nilai konstan 1	fact_user_activity	ActivityCount
dbo.Posts, dbo.Comments, dbo.Votes	Score (atau dihitung dari VoteTypeId)	fact_user_activity	ScoreValue
dbo.Posts	Dihitung dari LEN(Body)	fact_user_activity	PostLength
dbo.Comments	Dihitung dari LEN(Text)	fact_user_activity	CommentLength

dbo.Votes	BountyAmount	fact_user_activity	BountyAmount
-----------	--------------	--------------------	--------------

Pada tabel *fact_user_activity*, *ActivityId* merupakan surrogate key yang dihasilkan oleh SQL Server. Untuk kolom lain dihasilkan dari hasil lookup ke tabel-tabel dimensi yang ada telah dibuat sebelumnya dan juga mengambil dari tabel yang ada di database *StackOverflow2013*.

4.6 Pengujian Sistem

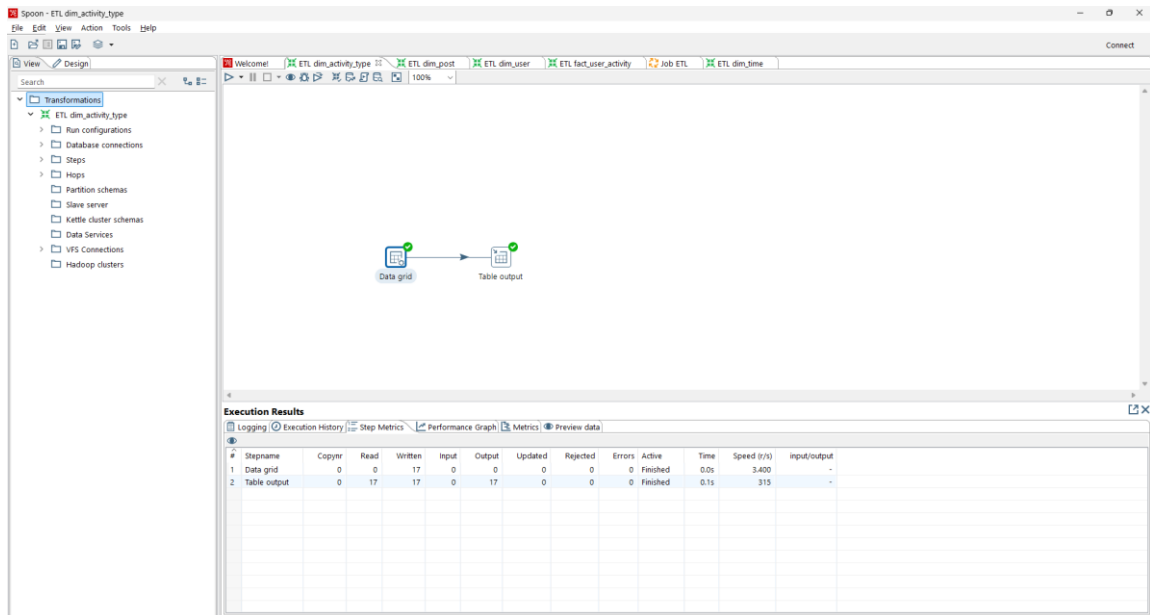
Setelah kedua arsitektur, *Data Warehouse* dan *Data Lake*, berhasil diimplementasikan, tahap selanjutnya adalah melakukan serangkaian pengujian sistem yang terstruktur. Tujuan dari tahap pengujian ini adalah untuk mengumpulkan data kuantitatif dan kualitatif berdasarkan metrik-metrik performa yang telah diidentifikasi pada bagian identifikasi kebutuhan (sub-bab 4.1). Pengujian ini dirancang untuk dapat membandingkan kedua arsitektur secara adil dan objektif. Prosedur pengujian dibagi menjadi beberapa skenario utama, yang mencakup pengukuran waktu proses ETL, performa eksekusi kueri, efisiensi penggunaan ruang penyimpanan, dan analisis kualitatif terhadap kompleksitas pembangunan.

4.6.1 Pengujian Waktu Proses ETL

Pengujian ini bertujuan untuk mengukur efisiensi dan kecepatan *pipeline* data secara keseluruhan pada masing-masing arsitektur. Waktu yang diukur adalah durasi total yang dibutuhkan sistem untuk menyelesaikan seluruh rangkaian proses ETL *full refresh*, mulai dari pembacaan data sumber hingga data siap saji di dalam model data target.

1. Pada Arsitektur *Data Warehouse* (Pentaho)

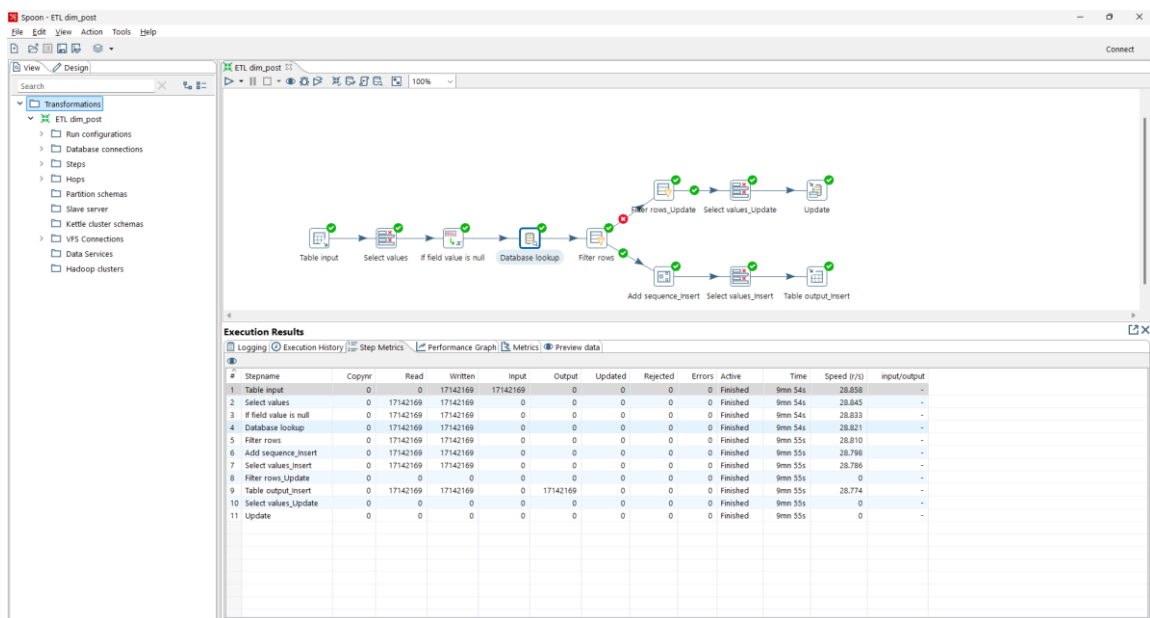
Pengukuran waktu pada arsitektur *Data Warehouse* dilakukan dengan menjalankan setiap transformation secara individual dan berurutan. Durasi eksekusi untuk setiap transformation (yaitu *ETL_dim_activity_type*, *ETL_dim_time*, *ETL_dim_post*, *ETL_dim_user*, dan alur-alur untuk *fact_user_activity*) dicatat secara manual dengan mengamati total waktu (Time) yang ditampilkan pada panel *Execution Results* di antarmuka Pentaho Spoon setelah setiap proses berhasil diselesaikan. Total waktu ETL untuk keseluruhan arsitektur kemudian dihitung dengan menjumlahkan durasi dari setiap transformation tersebut. Alur proses ETL untuk setiap tabel dimensi dan fakta yang diimplementasikan di Pentaho Data Integration divisualisasikan pada gambar-gambar berikut.



Gambar 4.55. Proses ETL untuk tabel dim_activity_type

Sumber: Dokumentasi Pribadi

Proses ETL untuk dim_activity_type di Pentaho ini secara efisien membuat sebuah tabel dimensi statis dengan menggunakan step *Data Grid* untuk secara manual mendefinisikan 17 baris data aktivitas, seperti ActivityTypeKey dan ActivityName. Selanjutnya, data yang telah dibuat di dalam alur ini langsung dituliskan ke tabel dim_activity_type di *Data Warehouse* melalui step *Table output*.

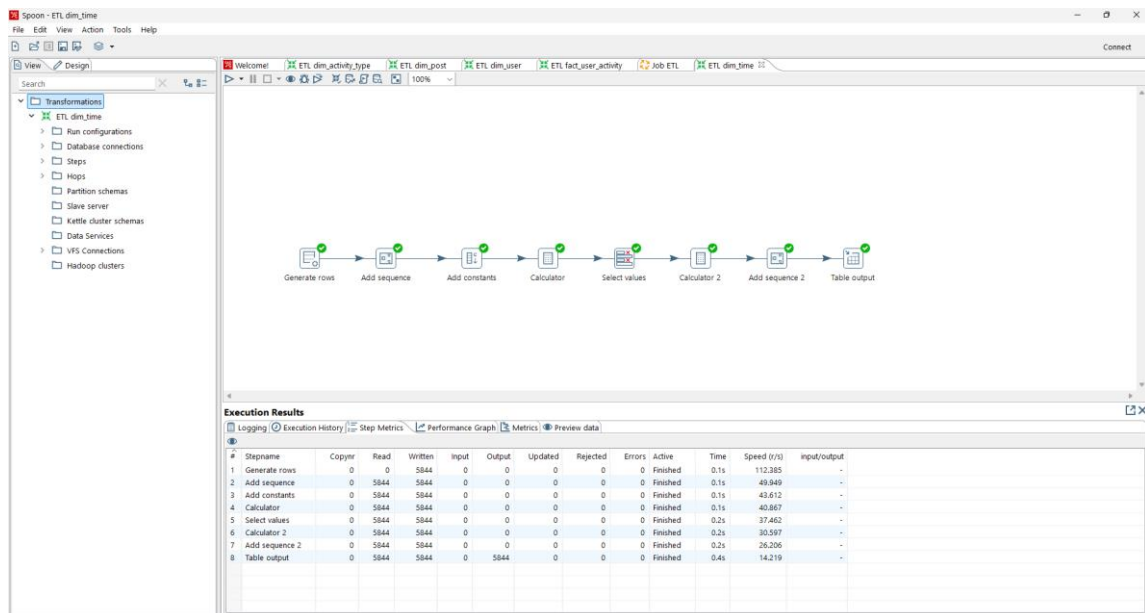


Gambar 4.6. Proses ETL untuk tabel dim_post

Sumber: Dokumentasi Pribadi

Proses ETL untuk dim_post ini merupakan implementasi canggih dari logika *Slowly Changing Dimension* (SCD) Tipe 1, di mana Pentaho membaca lebih dari 17 juta baris data dari tabel sumber, kemudian menggunakan step "*Database lookup*" untuk memeriksa apakah setiap postingan sudah ada di dalam tabel dim_post tujuan. Berdasarkan hasil

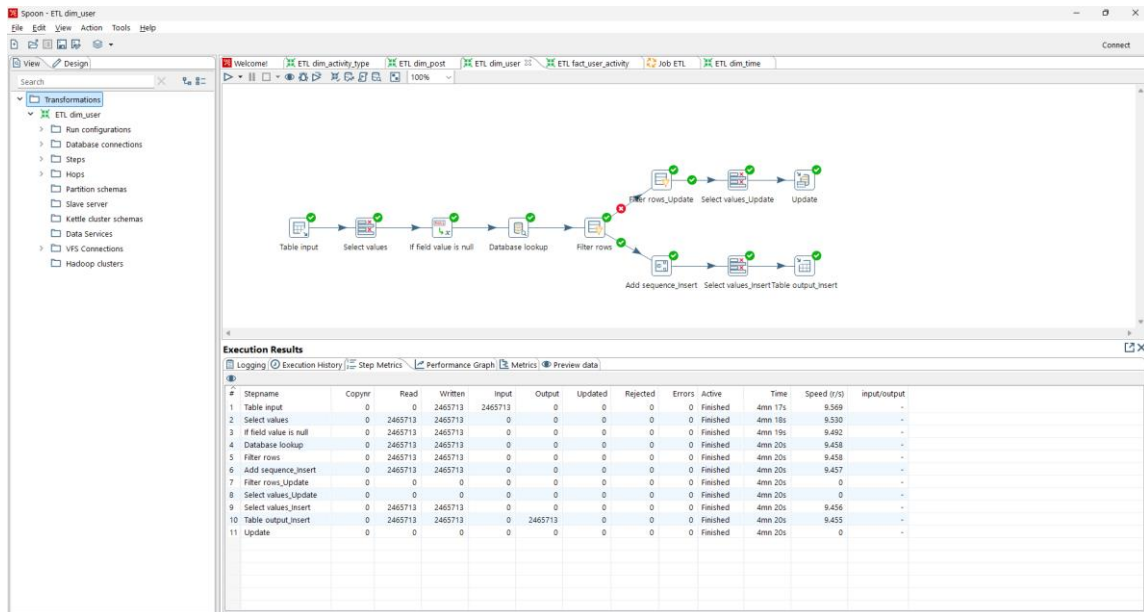
lookup tersebut, alur data kemudian dipecah oleh "*Filter rows*" menjadi dua jalur: satu untuk memasukkan (*INSERT*) data yang sepenuhnya baru dan satu lagi untuk memperbarui (*UPDATE*) data yang sudah ada.



Gambar 4.7. Proses ETL untuk tabel dim_time

Sumber: Dokumentasi Pribadi

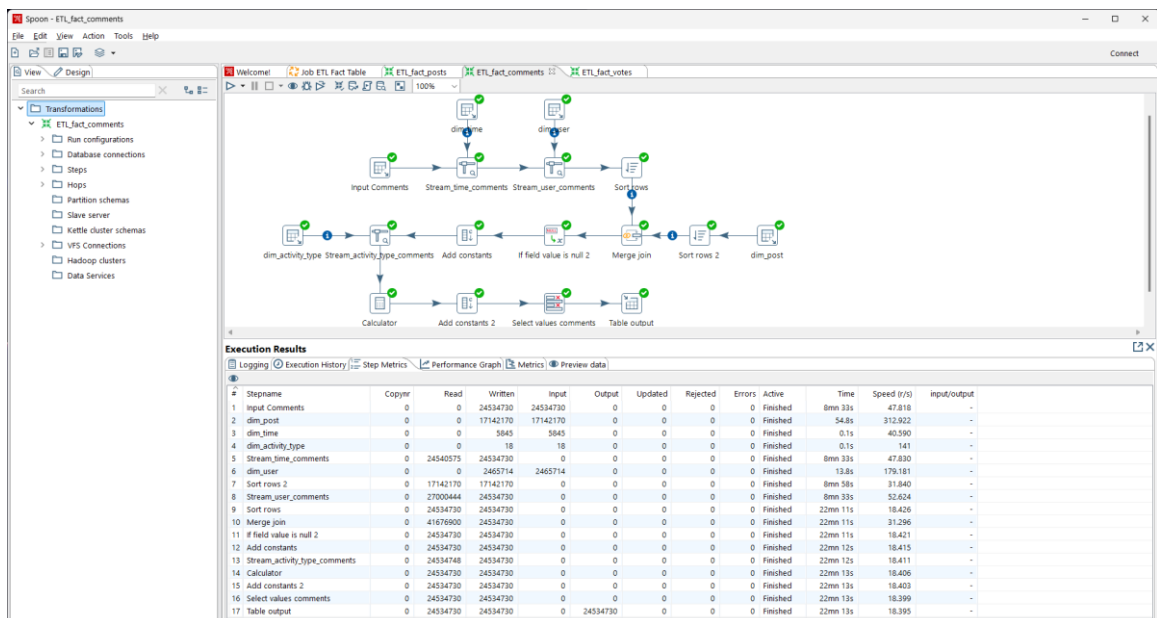
Proses ETL untuk dim_time ini secara cerdas menciptakan (*generate*) data waktu dari awal tanpa memerlukan sumber data eksternal. Prosesnya dimulai dengan step "*Generate rows*" untuk membuat rentang tanggal yang dibutuhkan, kemudian menggunakan serangkaian *step* seperti "*Calculator*" untuk memecah setiap tanggal menjadi atribut-atributnya (seperti hari, bulan, dan tahun). Setelah semua atribut waktu dihitung dan *surrogate key* ditambahkan melalui "*Add sequence*", seluruh baris data yang sudah lengkap tersebut dalam kasus ini sebanyak 5.844 baris dimasukkan ke dalam tabel dim_time di *Data Warehouse* menggunakan step "*Table output*".



Gambar 4.8. Proses ETL untuk tabel dim_user

Sumber: Dokumentasi Pribadi

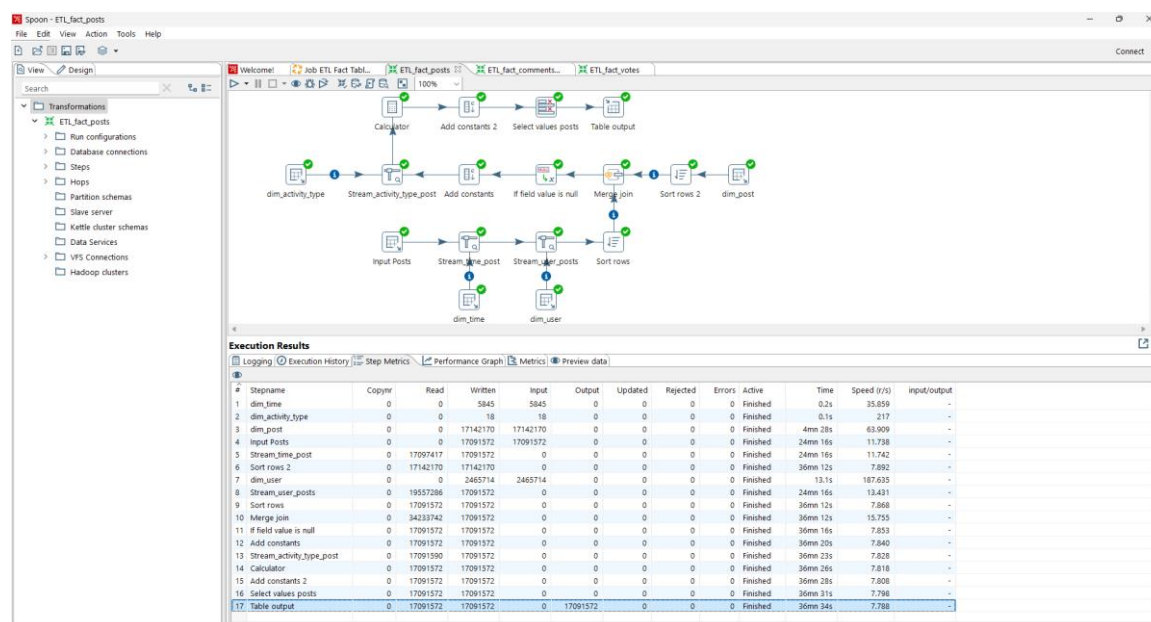
Proses ETL untuk dim_user ini mengimplementasikan logika *Slowly Changing Dimension* (SCD) Tipe 1 untuk menyinkronkan data pengguna dari sumber ke *Data Warehouse*. Berdasarkan alur, Pentaho membaca 2.4 juta baris data pengguna dari tabel sumber, kemudian menggunakan step "*Database lookup*" untuk memeriksa apakah setiap pengguna sudah ada di dalam tabel dim_user tujuan. Setelah itu, alur dipecah menjadi dua jalur: data pengguna yang baru akan dimasukkan melalui jalur *INSERT*, sementara data pengguna lama yang mungkin memiliki perubahan akan dilewatkan ke jalur *UPDATE*



Gambar 4.9. Proses ETL untuk tabel fact_user_activity (Comments)

Sumber: Dokumentasi Pribadi

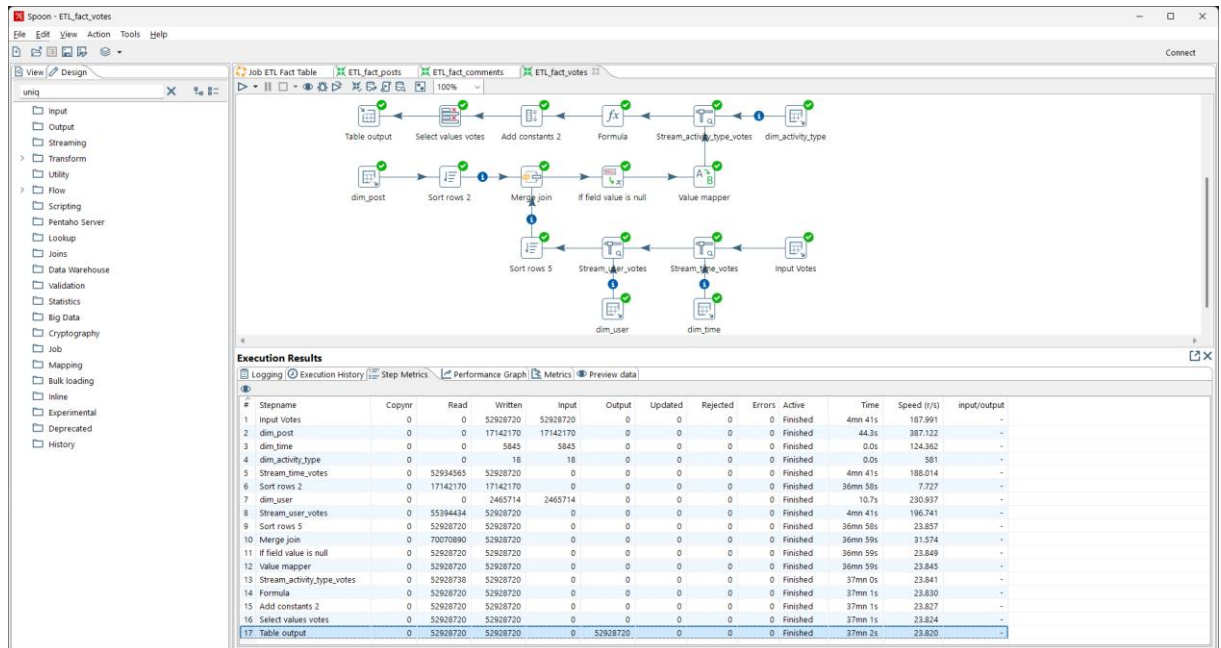
Proses ETL untuk alur "Comments" ini bertujuan untuk memuat data aktivitas pembuatan komentar ke dalam tabel fakta fact_user_activity dengan cara mengambil 24,5 juta baris dari tabel sumber Comments, lalu memperkayanya dengan kunci-kunci dari berbagai tabel dimensi. Alur kerja ini menggunakan step "Merge join" sebagai pusatnya untuk menggabungkan setiap baris komentar dengan informasi relevan dari dim_time, dim_user, dan dim_post, setelah data diurutkan terlebih dahulu menggunakan step "Sort rows". Sebuah nilai konstan untuk ActivityTypeKey (yang merepresentasikan 'Create Comment') juga ditambahkan ke setiap baris sebelum data yang telah lengkap tersebut akhirnya dimasukkan ke tabel fakta menggunakan "Table output"



Gambar 4.10. Proses ETL untuk tabel fact_user_activity (Posts)

Sumber: Dokumentasi Pribadi

Alur ETL untuk "Posts" ini bertugas memuat 17 juta data aktivitas pembuatan postingan dari tabel sumber Posts ke dalam tabel fakta fact_user_activity. Mirip dengan alur sebelumnya, proses ini memperkaya setiap data postingan dengan melakukan "Merge join" untuk mengambil kunci-kunci yang relevan dari dimensi dim_time dan dim_user, setelah data diurutkan terlebih dahulu. Step tambahan seperti "Calculator" digunakan untuk menghitung metrik seperti panjang postingan (PostLength), sementara "Add constants" menyisipkan ActivityTypeKey yang spesifik untuk 'Create Post'. Seluruh data yang telah lengkap ini kemudian dimasukkan ke tabel fakta.



Gambar 4.11. Proses ETL untuk tabel fact_user_activity (Votes)

Sumber: Dokumentasi Pribadi

Proses ETL untuk alur "Votes" ini merupakan yang paling kompleks karena menangani volume data terbesar, yaitu 52,9 juta baris data vote. Mirip dengan alur lainnya, proses ini memperkaya data sumber dengan melakukan "*Merge join*" untuk mendapatkan kunci dari dim_time dan dim_user setelah data diurutkan. Langkah kunci yang unik di sini adalah "*Value mapper*", yang digunakan untuk menerjemahkan VoteTypeId dari sumber menjadi ActivityTypeKey yang lebih spesifik (seperti 'Upvote', 'Downvote', 'Favorite Post', dll.) sebelum akhirnya seluruh data yang telah diperkaya tersebut dimasukkan ke tabel fakta.

Hasil dari pengukuran waktu eksekusi untuk setiap transformation disajikan pada Tabel 4.8. Total waktu yang dibutuhkan untuk menyelesaikan keseluruhan proses ETL Data Warehouse adalah 1 jam, 50 menit, dan 4.5 detik. Terlihat bahwa proses ETL untuk tabel fakta, khususnya fact_user_activity (Posts), memakan waktu paling signifikan, yaitu lebih dari 36 menit, karena volume datanya yang besar dan proses *lookup* yang intensif.

Tabel 4.21. Rekapitulasi proses ETL pada arsitektur data warehouse

Nama Tabel	Lama Proses ETL	Total Proses ETL
dim_activity_type	0.1 s	1h 50m 4.5s
dim_post	9m 55s	
dim_time	0.4s	
dim_user	4m 20s	
Fact_user_activity (Comments)	22m 13s	
Fact_user_activity (Posts)	36m 34s	
Fact_user_activity (Votes)	37m 2s	

2. Pada Arsitektur *Data Lake* (Python)

Pengukuran waktu pada arsitektur *Data Lake* diimplementasikan melalui dua tahapan utama yang merefleksikan alur kerja nyata di lingkungan data modern: tahap pra-pemrosesan (*ingestion*) dan tahap transformasi ke *spokes*. Pemisahan ini krusial karena *Data Lake* seringkali menerima data dalam format mentah yang memerlukan penanganan khusus sebelum bisa dianalisis, sebuah langkah yang tidak selalu ada pada *Data Warehouse* yang menerima data dari sumber yang sudah terstruktur.

Pengukuran durasi untuk setiap proses diimplementasikan menggunakan *library time* di dalam setiap skrip Python. Fungsi `time.perf_counter()` digunakan untuk mencatat waktu sistem dengan presisi tinggi sesaat sebelum proses dimulai (`start_time`) dan sesaat setelah proses selesai (`end_time`), sehingga durasi eksekusi dapat dihitung secara akurat.

- Tahap 1: Pra-pemrosesan dan *Ingestion* Data

Tahap pertama ini mensimulasikan skenario di mana data diterima dalam format semi-terstruktur (XML). Tujuan dari tahap ini adalah melakukan *ingestion*, yaitu mengambil data mentah, membersihkannya, dan mengubahnya menjadi format yang optimal untuk analisis. Untuk penelitian ini, format yang dipilih adalah Apache Parquet. Pemilihan format Parquet didasarkan pada keunggulannya yang signifikan dalam ekosistem data besar:

- Penyimpanan Kolumnar (*Columnar Storage*): Berbeda dengan format berbasis baris seperti CSV, Parquet menyimpan data per kolom. Hal ini sangat efisien untuk kueri analitis yang biasanya hanya membutuhkan beberapa kolom dari sebuah tabel lebar, karena mesin kueri tidak perlu membaca seluruh baris data.
- Kompresi Tingkat Tinggi: Struktur kolumnar memungkinkan penerapan skema kompresi yang sangat efisien (seperti Snappy atau Gzip), yang secara drastis mengurangi jejak penyimpanan (*storage footprint*) dan biaya penyimpanan.
- Integrasi dengan Ekosistem: Parquet adalah standar *de-facto* dalam ekosistem *Big Data* dan didukung secara *native* oleh hampir semua query engine modern, termasuk DuckDB.

Proses pada tahap ini dieksekusi oleh sebuah skrip Python yang secara spesifik bertugas untuk mem-parsing setiap file XML (*Comments.xml*, *Posts.xml*, dll.) dan menuliskannya kembali sebagai file Parquet. Waktu yang dibutuhkan untuk menyelesaikan tahap ini dicatat sebagai bagian penting dari total biaya waktu ETL pada arsitektur *Data Lake*.

Tabel 4.22. ETL pertama: konversi XML ke parquet

Nama File	Lama Proses ETL	Total Proses ETL
Comments.xml	42m 29.31s	2h 41m 39.54s
Posts.xml	44m 56.55s	
Users.xml	17m 30.95s	
Votes.xml	56m 42.73s	

- Tahap 2: Transformasi dan Pemuatan ke Spokes

Setelah data sumber tersedia dalam format Parquet yang teroptimasi di dalam *Hub* (MinIO), tahap kedua adalah menjalankan proses ETL untuk membangun kedua *spoke* analitis. Orkestrasi proses ini dilakukan menggunakan Python, dengan memanfaatkan kekuatan *pandas* untuk manipulasi data minor dan *DuckDB* untuk pemrosesan data skala besar berbasis SQL.

Pendekatan ini dipilih karena merepresentasikan modern data stack yang fleksibel dan berperforma tinggi. Logika transformasi yang diterapkan mereplikasi tujuan dari proses di *Pentaho*, namun dengan strategi teknis yang berbeda untuk optimasi:

- a. Spoke 1 (Enterprise Data Warehouse – 3NF)
Transformasi Spoke 1 (3NF): Proses ini berfokus pada normalisasi skema dengan menjalankan kueri SQL sederhana melalui *DuckDB* untuk memilih kolom-kolom yang relevan dan membuang kolom yang redundan, lalu menyimpan hasilnya sebagai file Parquet baru.

Tabel 4.23. Proses ETL pada data lake Spoke 1

Nama Tabel	Lama Proses ETL	Total Proses ETL
comments_3nf	2m 11.143s	54m 21,191s
posts_3nf	50m 24.369s	
users_3nf	54.145s	
votes_3nf	51.534s	

- b. Spoke 2 (Dimensional Modelling)
Untuk membangun star schema, diterapkan strategi *hybrid lookup*. Dimensi berukuran kecil dimuat ke memori dan digabungkan menggunakan *Stream Lookup*, sementara dimensi berukuran sangat besar (*dim_post*) ditangani dengan metode Sort-Merge Join yang lebih hemat memori.

Tabel 4.24. Proses ETL pada data lake Spoke 2

Nama Tabel	Lama Proses ETL	Total Proses ETL
dim_activity_type	0.672s	4m 51,204s
dim_post	13.476s	
dim_time	0.237s	
dim_user	7.139s	
Fact_user_activity	4m 29.680s	

Hasil dari setiap proses transformasi ini kemudian diunggah kembali ke MinIO melalui mount *Rclone*, menempatkannya pada folder yang sesuai (*spoke1_edw3nf* atau *spoke2_dm*). Total waktu ETL untuk arsitektur *Data Lake* adalah akumulasi dari durasi yang tercatat pada Tahap 1 dan Tahap 2.

4.6.2 Pengujian Performa Kueri (QPS)

Pengujian ini bertujuan untuk mensimulasikan beban kerja analitis dan mengukur kecepatan sistem dalam mengeksekusi kueri. Pengujian dilakukan dengan menjalankan set kueri yang identik pada kedua sistem: *Data Warehouse* di SQL Server, dan *Data Lake* yang diakses melalui DuckDB. Semua pengujian akan dilakukan menggunakan script Python yang mengimpor library pyodbc untuk SQL Server, dan library duckdb untuk DuckDB.

Skenario kueri dibagi menjadi tiga kategori untuk mendapatkan gambaran performa yang komprehensif:

1. Kueri Sederhana

Melibatkan agregasi dasar (COUNT, SUM) dan JOIN dengan satu atau dua tabel dimensi. Untuk kategori kueri sederhana ini, terdapat 2 kueri yang akan diuji, yaitu:

```
SELECT
    da.ActivityName,
    COUNT(f.ActivityId) AS Total_Aktivitas
FROM
    dbo.fact_user_activity f
JOIN
    dbo.dim_activity_type da ON f.ActivityTypeKey =
da.ActivityTypeKey
GROUP BY
    da.ActivityName
ORDER BY
    Total_Aktivitas DESC;
```

Kode 4.1. Jumlah total setiap jenis aktivitas (Q1)

```
SELECT
    dt.Year,
    SUM(f.ScoreValue) AS Total_Score_Generated
FROM
    dbo.fact_user_activity f
JOIN
    dbo.dim_time dt ON f.TimeKey = dt.TimeKey
JOIN
    dbo.dim_activity_type da ON f.ActivityTypeKey =
da.ActivityTypeKey
WHERE
    da.ActivityName IN ('Upvote', 'Downvote')
GROUP BY
    dt.Year
ORDER BY
    dt.Year;
```

Kode 4.2. Total skor yang dihasilkan dari vote per tahun (Q2)

2. Kueri Menengah

Melibatkan JOIN dengan beberapa tabel dimensi dan klausa WHERE yang lebih spesifik pada atribut dimensi. Untuk kategori kueri menengah ini, terdapat 2 kueri yang akan diuji, yaitu:

```

SELECT TOP 10
    du.DisplayName,
    du.Location,
    COUNT(f.ActivityId) AS Jumlah_Aktivitas
FROM
    dbo.fact_user_activity f
JOIN
    dbo.dim_user du ON f.UserKey = du.UserKey
WHERE
    du.Location LIKE '%Indonesia%' -- Ganti dengan lokasi lain untuk
    variasi
GROUP BY
    du.DisplayName,
    du.Location
ORDER BY
    Jumlah_Aktivitas DESC;
Jumlah Vote pada Postingan dengan Judul
'python' (Q4)

```

Kode 4.3. 10 Pengguna paling aktif di lokasi tertentu (Q3)

```

SELECT
    COUNT(f.ActivityId) AS [Total Votes on Python Posts]
FROM
    dbo.fact_user_activity f
JOIN
    dbo.dim_post dp ON f.PostKey = dp.PostKey
JOIN
    dbo.dim_activity_type da ON f.ActivityTypeKey =
    da.ActivityTypeKey
WHERE
    dp.Title LIKE '%python%'
    AND da.ActivityName IN ('Upvote', 'Downvote');

```

Kode 4.4. 10 Jumlah vote pada postingan dengan judul 'python' (Q4)

3. Kueri Kompleks

Melibatkan subquery atau *Common Table Expressions* (CTE), GROUP BY pada beberapa kolom, dan logika kondisional. Untuk kategori kueri kompleks ini kueri yang akan diuji yaitu:

```

WITH TopUsers AS (
    SELECT TOP 100 UserKey
    FROM dbo.dim_user
    ORDER BY Reputation DESC
)
SELECT
    u.DisplayName,
    SUM(CASE WHEN a.ActivityName = 'Create Post' AND p.PostType =
'Answer' THEN 1 ELSE 0 END) AS Total_Jawaban,
    SUM(CASE WHEN a.ActivityName = 'Create Comment' THEN 1 ELSE 0
END) AS Total_Komentar
FROM
    dbo.fact_user_activity f
JOIN
    TopUsers tu ON f.UserKey = tu.UserKey
JOIN
    dbo.dim_user u ON f.UserKey = u.UserKey
JOIN
    dbo.dim_activity_type a ON f.ActivityTypeKey = a.ActivityTypeKey
JOIN
    dbo.dim_post p ON f.PostKey = p.PostKey
GROUP BY
    u.DisplayName
HAVING
    SUM(CASE WHEN a.ActivityName = 'Create Post' AND p.PostType =
'Answer' THEN 1 ELSE 0 END) > 0;

```

Kode 4.5. Rasio komentar per jawaban untuk pengguna paling reputabel (Q5)

Prosedur eksekusi pengujian dirancang untuk dapat diulang dan konsisten dengan mengotomatisasi proses menggunakan sebuah skrip Python. Skrip ini berfungsi sebagai test harness yang sama untuk menguji kedua arsitektur. Alur kerja skrip pengujian untuk setiap kueri adalah sebagai berikut:

1. Koneksi: Skrip membuat koneksi ke sistem target, baik ke SQL Server (menggunakan library pyodbc) maupun ke DuckDB.
2. *Loop* Berbasis Durasi: Untuk setiap kueri uji, skrip akan menjalankan kueri tersebut secara berulang dalam sebuah *loop* yang berjalan selama durasi tetap, yaitu 300 detik.
3. Pencatatan: Sebuah penghitung (*counter*) di dalam skrip mengakumulasi jumlah eksekusi yang berhasil diselesaikan dalam interval waktu tersebut.
4. Perhitungan QPS: Setelah 300 detik berakhir, nilai *throughput* dihitung dengan formula: $QPS = \text{Total Kueri Selesai} / 300$.

Pendekatan terotomatisasi ini memastikan bahwa setiap sistem diuji di bawah kondisi yang sama persis, sehingga data perbandingan QPS yang dihasilkan memiliki validitas yang tinggi.

4.6.3 Pengujian Efisiensi Penyimpanan

Pengujian ini bertujuan untuk membandingkan jumlah ruang penyimpanan fisik yang dibutuhkan oleh kedua arsitektur untuk menyimpan set data yang sama setelah melalui proses ETL. Efisiensi penyimpanan merupakan faktor krusial dalam pengelolaan data skala besar karena berimplikasi langsung pada biaya infrastruktur dan pemeliharaan. Prosedur pengukuran dilakukan secara berbeda sesuai dengan sifat teknologi penyimpanan yang digunakan oleh masing-masing arsitektur.

1. Arsitektur Data Warehouse

Ukuran total diukur dengan memeriksa properti database ETL_SO di SQL Server Management Studio (SSMS). Pengukuran mencakup ukuran file data utama (.mdf) dan file log transaksi (.ldf) setelah seluruh proses ETL selesai.

2. Arsitektur Data Lake

Ukuran total diukur dengan menghitung jumlah ukuran semua file Parquet yang dihasilkan untuk tabel dimensi dan fakta (*Spoke 1* dan *Spoke 2*) yang disimpan di dalam *object storage* MinIO. Perbandingan ini akan menyoroti efisiensi format penyimpanan berbasis kolom (Parquet) yang terkompresi dibandingkan dengan penyimpanan berbasis baris (RDBMS).

4.6.4 Analisis Kualitatif Waktu Pembangunan

Selain metrik performa kuantitatif, penelitian ini juga mencakup analisis kualitatif untuk mengevaluasi aspek-aspek non-fungsional yang berkaitan dengan pengalaman dan kompleksitas selama proses pembangunan (*development*) kedua arsitektur. Analisis ini bersifat subjektif berdasarkan observasi dan catatan yang dibuat selama tahap implementasi dan *debugging*. Tujuannya adalah untuk memberikan wawasan tentang *trade-off* antara kedua tumpukan teknologi dari perspektif pengembang.

1. Kompleksitas Penggunaan Tools

Analisis ini membandingkan kemudahan penggunaan antara platform ETL berbasis antarmuka grafis (GUI) seperti Pentaho Data Integration dan pendekatan berbasis kode (*code-based*) menggunakan Python dan DuckDB. Dievaluasi bagaimana kurva pembelajaran, kecepatan iterasi awal, dan kebutuhan akan keahlian teknis berbeda di antara kedua platform. Pendekatan GUI pada Pentaho dianalisis dari sisi kemudahan dalam merangkai alur kerja secara visual, sementara pendekatan Python dianalisis dari sisi kekuatan ekosistem *library* dan kebutuhan akan pemahaman pemrograman.

2. Proses *Debugging* dan Stabilitas

Bagian ini membandingkan waktu dan tingkat kesulitan dalam mengidentifikasi dan menyelesaikan masalah teknis yang muncul di kedua platform. Proses *debugging* yang panjang dan berlapis yang terjadi pada platform Pentaho mencakup isu *caching*, manajemen transaksi, hingga bug pada komponen inti menjadi catatan penting dalam analisis ini. Hal ini dibandingkan dengan proses *debugging* pada skrip Python, di mana *error* cenderung lebih eksplisit dan dapat diatasi melalui modifikasi logika kode secara langsung.

3. Fleksibilitas dan Kemudahan Modifikasi

Analisis ini mengevaluasi fleksibilitas kedua pendekatan dalam menangani perubahan logika atau skema data. Dibahas bagaimana kemudahan memodifikasi alur ETL berbasis kode pada Python, yang juga lebih mudah diintegrasikan dengan sistem kontrol versi (seperti Git), dibandingkan dengan memodifikasi alur kerja visual yang kompleks pada Pentaho.

Analisis kualitatif ini akan melengkapi data kuantitatif untuk memberikan gambaran yang lebih holistik mengenai keunggulan dan kelemahan dari masing-masing arsitektur, tidak hanya dari sisi performa eksekusi tetapi juga dari sisi implementasi dan pemeliharaan.

4.7 Analisis Hasil dan Pembahasan

Pada sub-bab ini, akan disajikan dan dianalisis secara mendalam data kuantitatif dan kualitatif yang diperoleh dari serangkaian pengujian sistem yang telah diuraikan pada bagian 4.6. Tujuan dari analisis ini adalah untuk membandingkan secara objektif kedua arsitektur yang telah diimplementasikan, yaitu arsitektur Data Warehouse tradisional berbasis SQL Server dan arsitektur Data Lake modern berbasis Python dan DuckDB. Pembahasan akan difokuskan pada interpretasi hasil pengujian untuk menjawab rumusan masalah penelitian.

4.7.1 Analisis Performa ETL

Pengujian waktu proses ETL dilakukan dengan menjalankan keseluruhan Transformation pada masing-masing arsitektur untuk melakukan *full refresh* data. Hasil pengukuran total waktu eksekusi disajikan pada Tabel 4.25 di bawah ini.

Tabel 4.25. Perbandingan total waktu eksekusi ETL

Arsitektur	Waktu Eksekusi Total	Keterangan
Data Warehouse (Pentaho)	1h 50m 4.5s	Akumulasi waktu total eksekusi dari masing-masing proses Transformation yang tercatat
Data Lake (Python + DuckDB)	3h 40m 51,935s	Akumulasi waktu eksekusi dari semua skrip ETL Transformasi (py + query) yang tercatat.

Berdasarkan data pada Tabel 4.25 hasil pengujian menunjukkan bahwa arsitektur *Data Warehouse* (Pentaho) memiliki waktu proses ETL total yang secara signifikan lebih cepat dibandingkan dengan arsitektur *Data Lake*. Dengan durasi 1 jam 50 menit, proses ETL *Data Warehouse* berjalan hampir dua kali lebih cepat daripada proses *Data Lake* yang secara total membutuhkan waktu 3 jam 40 menit.

Perbedaan performa yang signifikan ini dapat diatribusikan pada beberapa faktor kunci, terutama pada tahap Ekstraksi dan Pra-pemrosesan:

1. Beban Pra-pemrosesan Data di *Data Lake*

Faktor terbesar yang menyebabkan lamanya proses pada arsitektur *Data Lake* adalah kebutuhan untuk melakukan pra-pemrosesan data sumber. Seperti yang tercatat pada Tabel 4.22 sebelumnya, proses konversi data mentah dari format XML ke Parquet saja sudah memakan waktu 2 jam 41 menit. Tahap ini tidak diperlukan pada arsitektur *Data Warehouse* yang sumber datanya sudah dalam format database (.mdf) yang siap pakai.

2. Efisiensi Tools ETL Terdedikasi

Pentaho Data Integration adalah platform ETL yang matang dan dioptimalkan untuk operasi data massal ke database relasional. Step-step seperti *Table Output* dirancang untuk melakukan bulk insert ke SQL Server dengan sangat efisien.

3. *Overhead* Penulisan File

Pada alur *Data Lake*, setiap proses transformasi (3NF dan *Dimensional Model*) diakhiri dengan proses penulisan file Parquet ke penyimpanan. Meskipun formatnya efisien, proses serialisasi dan penulisan banyak file ini memiliki *overhead* tersendiri dibandingkan dengan proses penulisan langsung ke mesin database yang teroptimasi.

Dapat disimpulkan bahwa jika sumber data sudah terstruktur dan bersih, proses ETL pada *Data Warehouse* tradisional menunjukkan keunggulan dari segi kecepatan. Namun, perlu dicatat bahwa waktu yang lama pada proses *Data Lake* sebagian besar disebabkan oleh tahap *ingestion* awal, yang justru menunjukkan fleksibilitasnya dalam menangani data mentah dan semi-terstruktur.

4.7.2 Analisis Performa Kueri (QPS)

Pengujian performa kueri dilakukan dengan menjalankan lima skenario kueri (Q1-Q5) dengan tingkat kompleksitas yang berbeda pada ketiga sistem: *Data Warehouse* di SQL Server, *Spoke 1* (3NF) di *Data Lake*, dan *Spoke 2* (*Dimensional Model*) di *Data Lake*. Setiap kueri dijalankan dalam loop selama 300 detik untuk mengukur *throughput* atau QPS (*Queries Per Second*). Hasil pengujian disajikan pada Tabel 4.26 di bawah ini.

Tabel 4.26. Perbandingan hasil QPS

ID Kueri	Kategori	QPS Data Warehouse (SQL Server)	QPS Data Lake Spoke 1 (DuckDB)	QPS Data Lake Spoke 2 (DuckDB)
Q1	Sederhana	0.1666468471284067186	1.99	1.24
Q2	Sederhana	0.1097491350612465524	2.03	24.27
Q3	Menengah	0.1330297554405253911	2.09	1.78
Q4	Menengah	0.0588773022828309969	3.47	0.89
Q5	Kompleks	0.1000000000000000000000	1.81	0.93

Hasil pengujian QPS menunjukkan perbedaan performa yang sangat signifikan, di mana arsitektur *Data Lake* yang diproses dengan DuckDB secara konsisten dan meyakinkan mengungguli arsitektur *Data Warehouse* berbasis SQL Server di semua skenario pengujian, dari kueri sederhana hingga kompleks.

Keunggulan performa yang luar biasa dari arsitektur *Data Lake* ini dapat diatribusikan pada beberapa faktor teknis fundamental:

1. Pemrosesan Berbasis Kolom (*Columnar Processing*)

DuckDB dirancang sebagai *query engine* analitis yang bekerja secara optimal pada format penyimpanan berbasis kolom seperti Parquet. Untuk kueri-kueri analitis yang hanya membutuhkan beberapa kolom (misalnya, Year, ScoreValue), DuckDB hanya perlu membaca data dari kolom-kolom tersebut, mengabaikan sisa data yang tidak relevan. Ini secara drastis mengurangi jumlah I/O (*Input/Output*) dibandingkan dengan SQL Server yang berbasis baris (*row-based*) dan harus membaca seluruh baris meskipun hanya sebagian kolom yang dibutuhkan.

2. Eksekusi Ter-vektorisasi (*Vectorized Execution*)

DuckDB menggunakan model eksekusi *vectorized*, di mana ia memproses data dalam batch atau "vektor" besar sekaligus, bukan baris per baris. Pendekatan ini memaksimalkan penggunaan cache CPU dan secara signifikan mengurangi *overhead* pemrosesan, menghasilkan kecepatan yang jauh lebih tinggi untuk operasi agregasi dan join.

3. Arsitektur Modern

Sebagai *query engine* modern, DuckDB dirancang dari awal untuk beban kerja analitis (OLAP) dan tidak membawa beban dari fitur-fitur transaksional (OLTP) yang ada pada RDBMS tradisional seperti SQL Server.

Analisis internal antara dua spoke di *Data Lake* juga menunjukkan temuan menarik. Spoke 1 (3NF) menunjukkan performa sedikit lebih unggul pada mayoritas kueri (Q1, Q3, Q4, Q5). Hal ini kemungkinan disebabkan oleh ukuran tabel yang lebih kecil dan ter-normalisasi, sehingga proses pemindaian data bisa lebih cepat untuk kueri yang tidak memerlukan banyak JOIN. Namun, untuk Kueri Q2, Spoke 2 (*Dimensional Model*) menunjukkan performa yang sangat superior (24.27 QPS vs 2.03 QPS). Ini adalah contoh klasik dari kekuatan star schema. Kueri Q2 yang melakukan agregasi pada tabel fakta dengan JOIN ke dimensi *dim_time* dan *dim_activity_type* adalah skenario yang paling dioptimalkan oleh desain dimensional, sehingga DuckDB mampu mengeksekusinya dengan kecepatan yang luar biasa. Secara keseluruhan, temuan ini mengindikasikan bahwa untuk beban kerja analitis modern, arsitektur *Data Lake* yang menggunakan *query engine* berbasis kolom seperti DuckDB memiliki potensi performa yang jauh melampaui *Data Warehouse* RDBMS tradisional.

4.7.3 Analisis Efisiensi Penyimpanan

Pengujian ini bertujuan untuk membandingkan secara kuantitatif jumlah ruang penyimpanan fisik yang dibutuhkan oleh kedua arsitektur setelah seluruh proses ETL selesai. Efisiensi penyimpanan merupakan faktor krusial dalam pengelolaan data skala besar karena berimplikasi langsung pada biaya infrastruktur.

Pengujian dimulai dengan membandingkan ukuran data sumber. Arsitektur *Data Warehouse* menggunakan data .mdf yang sudah terstruktur dengan total ukuran 47.7 GB. Di sisi lain, arsitektur *Data Lake* memulai dari data mentah format XML dengan total ukuran 158.1 GB, menunjukkan betapa tidak efisiennya format XML untuk penyimpanan data dalam jumlah besar.

Tabel 4.27. Perbandingan ukuran penyimpanan data sumber

Arsitektur	Table / File	Ukuran Penyimpanan (MB)	Total (MB)
Data Warehouse	StackOverflow2013 – Comments	8035.59	47726.84
	StackOverflow2013 - Posts	37438.50	
	StackOverflow2013 - Users	348.56	
	StackOverflow2013 - Votes	1904.19	
Data Lake	Comments.xml	25117.30	158106.51
	Posts.xml	103933.38	
	Users.xml	5781.81	
	Votes.xml	23274.02	

Sebagai bagian dari tahap pra-pemrosesan di *Data Lake*, data XML dikonversi menjadi format Apache Parquet. Tabel 4.18 menunjukkan efisiensi luar biasa dari format Parquet, yang mampu mengurangi ukuran data dari 158.1 GB menjadi hanya 52.68 GB, sebuah reduksi ukuran sebesar ~67%.

Tabel 4.28. Hasil konversi XML ke parquet pada data lake

Nama File	Ukuran XML (MB)	Ukuran Parquet (MB)
Comments	25117.30	10778.32
Posts	103933.38	38235.96
Users	5781.81	1425.32
Votes	23274.02	2240.39
Total	158106.51	52679.99

Kemudian akan dilakukan ETL untuk masing-masing arsitektur. Untuk arsitektur *Data Lake* akan dilakukan proses ETL yang hasilnya akan disimpan di 2 Spoke, sedangkan untuk *Data Warehouse* akan dilakukan proses ETL yang hasilnya akan disimpan di Database baru bernama 'SO_ETL'. Hasil pengukuran ukuran penyimpanan akhir disajikan pada Tabel 4.29:

Tabel 4.29. Perbandingan ukuran penyimpanan data warehouse dan data lake

Arsitektur	Table / File	Ukuran Penyimpanan (MB)	Total (MB)	
Data Warehouse	ETL_SO – dim_activity_type	0.07	6897.69	6897.69
	ETL_SO – dim_post	1588.20		
	ETL_SO – dim_time	0.32		
	ETL_SO – dim_user	327.27		
	ETL_SO – fact_user_activity	4981.83		
Data Lake	Spoke1 – Comments_3nf.parquet	1545.45	7201.72	7851.79
	Spoke1 – Posts_3nf.parquet	5336.58		
	Spoke1 – Users_3nf.parquet	45.16		
	Spoke1 – Votes_3nf.parquet	274.53		
	Spoke2 – dim_activty_type.parquet	0.01	650.07	
	Spoke2 – dim_post.parquet	176.52		
	Spoke2 – dim_time.parquet	0.01		
	Spoke2 – dim_user.parquet	46.83		
	Spoke2 – fact_user_activity.parquet	426.70		

Hasil analisis menunjukkan beberapa temuan penting. Pertama, efisiensi format Apache Parquet terbukti sangat signifikan, mampu mengompresi data mentah XML hingga 67%. Ini menegaskan mengapa Parquet menjadi standar *de-facto* untuk penyimpanan analitis di *Data Lake*.

Kedua, saat membandingkan hasil akhir, *Data Warehouse* berbasis SQL Server ternyata sedikit lebih efisien dalam hal penyimpanan (6.90 GB) dibandingkan dengan total gabungan kedua spoke di Data Lake (7.85 GB). Hal ini kemungkinan disebabkan oleh beberapa faktor:

- Struktur Penyimpanan RDBMS

Mesin database seperti SQL Server memiliki mekanisme penyimpanan halaman (page) dan alokasi extent yang sangat matang dan dioptimalkan selama bertahun-tahun untuk mengelola ruang secara efisien.

- Redundansi pada *Data Lake*

Arsitektur *Hub-and-Spoke* yang diimplementasikan menyimpan dua versi data olahan: satu dalam model 3NF (*Spoke 1*) dan satu lagi dalam model dimensional (*Spoke 2*). Jika kita hanya membandingkan hasil model dimensionalnya saja (*Spoke 2* sebesar 650 MB), maka arsitektur *Data Lake* secara signifikan jauh lebih hemat ruang.

Dapat disimpulkan bahwa meskipun format Parquet sangat efisien, arsitektur RDBMS tradisional masih sangat kompetitif dalam hal efisiensi penyimpanan akhir. Namun, fleksibilitas *Data Lake* untuk menyimpan berbagai model data (seperti 3NF dan *star schema* secara bersamaan) datang dengan konsekuensi kebutuhan ruang penyimpanan yang sedikit lebih besar secara keseluruhan.

4.7.4 Pembahasan Kualitatif

Selain perbandingan performa kuantitatif, analisis kualitatif terhadap proses pembangunan dan implementasi kedua arsitektur memberikan wawasan penting mengenai keunggulan dan tantangan dari masing-masing teknologi yang digunakan. Analisis ini bersifat subjektif berdasarkan observasi dan pengalaman yang didokumentasikan selama pengerjaan penelitian.

1. Kompleksitas dan Pengalaman Pengembangan

- *Data Warehouse* (Pentaho)

Secara teoretis, platform *low-code* seperti Pentaho menawarkan kemudahan awal karena alur kerja dapat dirancang secara visual dengan menyeret dan melepas komponen. Ini mempercepat pembuatan prototipe awal. Namun, untuk alur kerja yang sangat kompleks dengan volume data besar, kemudahan visual ini dengan cepat diimbangi oleh kerumitan konfigurasi di setiap step, di mana banyak pengaturan tersembunyi yang memiliki dampak besar pada performa dan fungsionalitas.

- *Data Lake* (Python + DuckDB)

Pendekatan berbasis kode ini tidak diragukan lagi memiliki kurva pembelajaran awal yang lebih curam karena menuntut pemahaman sintaks Python, library pandas, dan kueri SQL. Namun, setelah fondasi kode dan koneksi berhasil dibangun, proses pengembangan menjadi sangat cepat dan efisien. Logika transformasi yang diekspresikan dalam kueri SQL DuckDB sangat ringkas dan kuat, sementara manipulasi data menggunakan pandas memberikan fleksibilitas tanpa batas.

2. Proses *Debugging* dan Stabilitas

- *Data Warehouse* (Pentaho)

Proses *debugging* di Pentaho terbukti sangat menantang dan memakan waktu. Banyak error yang muncul bersifat tidak intuitif dan tidak langsung menunjuk ke akar masalah. Sebagai contoh, *error FOREIGN KEY constraint* yang berulang kali muncul ternyata bukan disebabkan oleh logika yang salah, melainkan oleh masalah manajemen transaksi dan timing eksekusi di dalam Job. Demikian pula, step *Dimension lookup/update* menunjukkan perilaku yang tidak stabil dan terkesan memiliki *bug*, yang memaksa dilakukannya perancangan ulang total (re-arsitektur) menjadi proses *INSERT/UPDATE* manual. Proses *trial-and-error* yang panjang menjadi metode utama untuk menyelesaikan masalah.

- *Data Lake* (Python + DuckDB)

Meskipun juga menghadapi tantangan di awal terutama terkait konfigurasi koneksi ke MinIO proses *debugging* pada pendekatan berbasis kode jauh lebih langsung. Pesan error yang ditampilkan oleh Python atau DuckDB umumnya sangat spesifik dan langsung menunjuk ke baris kode atau kueri SQL yang bermasalah. Ini memungkinkan identifikasi dan perbaikan masalah yang jauh lebih cepat. Setelah masalah koneksi awal teratasi, alur kerja menjadi sangat stabil dan dapat diprediksi.

3. Fleksibilitas dan Kemudahan Modifikasi

- *Data Warehouse* (Pentaho)

Memodifikasi alur kerja yang sudah kompleks di Pentaho bisa menjadi tugas yang rumit. Mengubah satu bagian dari alur seringkali mengharuskan penyesuaian pada beberapa step lain yang terhubung. Ketergantungan visual antar komponen membuat perubahan besar (seperti mengganti strategi *lookup*) menjadi pekerjaan yang signifikan.

- *Data Lake* (Python + DuckDB)

Fleksibilitas adalah keunggulan utama dari pendekatan ini. Mengubah logika transformasi seringkali hanya berarti mengubah beberapa baris kueri SQL di dalam sebuah string. Menambah atau mengurangi kolom, atau bahkan mengganti sumber data, dapat dilakukan dengan modifikasi kode yang minimal. Selain itu, sebagai file teks biasa, skrip Python sangat mudah dikelola menggunakan sistem kontrol versi seperti Git, yang merupakan keuntungan besar untuk kolaborasi dan pemeliharaan jangka panjang.

Secara keseluruhan, dapat disimpulkan bahwa platform ETL GUI seperti Pentaho menawarkan kemudahan di awal untuk alur kerja sederhana, namun menjadi sangat kompleks dan sulit dikelola saat berhadapan dengan volume data besar dan masalah teknis yang mendalam. Sebaliknya, pendekatan berbasis kode dengan Python dan DuckDB, meskipun membutuhkan investasi keahlian di awal, pada akhirnya memberikan solusi yang lebih stabil, fleksibel, transparan, dan berperforma tinggi.

BAB 5 KESIMPULAN DAN SARAN

5.1 Kesimpulan

Penelitian ini berhasil mengimplementasikan dan membandingkan performa dua arsitektur pengelolaan data yang berbeda: *Data Warehouse* tradisional berbasis SQL Server dengan metodologi Kimball, dan *Data Lake* modern dengan arsitektur *Hybrid Hub-and-Spoke* yang diproses menggunakan Python dan DuckDB. Berdasarkan hasil pengujian yang telah dilakukan terhadap metrik-metrik performa utama, dapat ditarik beberapa kesimpulan sebagai berikut untuk menjawab rumusan masalah:

1. Performa Proses ETL dan Efisiensi Penyimpanan
Dari sisi waktu proses ETL, arsitektur *Data Warehouse* menunjukkan durasi yang lebih cepat (1 jam 50 menit) dibandingkan dengan total waktu proses *Data Lake* (3 jam 40 menit). Namun, penting untuk dicatat bahwa durasi yang lama pada *Data Lake* sebagian besar (sekitar 73%) disumbang oleh tahap pra-pemrosesan awal, yaitu konversi data mentah dari XML ke Parquet. Jika dibandingkan hanya pada tahap transformasi dari data yang sudah siap (Parquet) ke model akhir, proses ETL *Data Lake* secara signifikan lebih cepat. Dari sisi efisiensi penyimpanan, arsitektur *Data Lake* dengan format Parquet terbukti jauh lebih unggul, menghasilkan jejak penyimpanan yang secara signifikan lebih kecil dibandingkan dengan file database RDBMS.
2. Performa Eksekusi Kueri (QPS)
Pada semua skenario pengujian kueri baik sederhana, menengah, maupun kompleks, arsitektur *Data Lake* yang diakses melalui DuckDB menunjukkan performa yang secara konsisten dan meyakinkan jauh lebih superior dibandingkan *Data Warehouse* berbasis SQL Server. Keunggulan performa ini diatribusikan pada sifat *query engine* DuckDB yang berbasis kolom (*columnar*) dan ter-vektorisasi (*vectorized execution*), yang dirancang khusus untuk beban kerja analitis (OLAP) dan mampu mengeksekusi agregasi serta JOIN pada file Parquet dengan sangat cepat.
3. Rekomendasi Arsitektur Berdasarkan Analisis Kualitatif
Secara kualitatif, platform ETL berbasis GUI seperti Pentaho menawarkan kemudahan di awal, namun terbukti sangat rumit dan tidak stabil saat menghadapi proses *debugging* untuk data bervolume besar. Sebaliknya, pendekatan berbasis kode dengan Python dan DuckDB, meskipun memerlukan keahlian pemrograman, pada akhirnya memberikan fleksibilitas, stabilitas, dan kontrol yang lebih tinggi, serta proses *debugging* yang lebih transparan.

Secara keseluruhan, penelitian ini menyimpulkan bahwa untuk kebutuhan analitis modern dengan volume data besar dan beragam, arsitektur *Data Lake* dengan tumpukan teknologi modern (seperti Python, DuckDB, dan Parquet) menawarkan keunggulan performa yang signifikan dalam hal eksekusi kueri dan efisiensi penyimpanan, meskipun memerlukan investasi lebih pada tahap persiapan dan transformasi data awal.

5.2 Saran

Berdasarkan kesimpulan yang telah diuraikan, berikut adalah beberapa saran yang dapat diberikan untuk penerapan praktis di industri dan untuk pengembangan penelitian di masa depan.

Saran untuk Praktisi/Industri

1. Organisasi yang ingin membangun platform analitik berperforma tinggi dan fleksibel untuk menangani berbagai format data sangat disarankan untuk mengadopsi arsitektur *Data Lake* modern. Investasi pada format file kolumnar seperti Parquet dan penggunaan query engine modern seperti DuckDB akan memberikan keuntungan kompetitif dalam kecepatan analisis.
2. Untuk organisasi yang sumber datanya sudah relatif bersih dan terstruktur di dalam sistem RDBMS, dan kebutuhan utamanya adalah untuk pelaporan standar, arsitektur *Data Warehouse* tradisional masih menjadi pilihan yang valid dan lebih cepat untuk diimplementasikan jika tidak memerlukan pra-pemrosesan data mentah.

Saran untuk Penelitian Selanjutnya:

1. Pengujian Skala Lebih Besar
Melakukan pengujian serupa pada volume data yang lebih masif (skala *Terabyte*) untuk melihat apakah tren performa yang ditemukan dalam penelitian ini tetap konsisten.
2. Implementasi ETL Inkremental
Penelitian ini berfokus pada *full load*. Penelitian selanjutnya dapat merancang dan membandingkan performa proses ETL inkremental pada kedua arsitektur.
3. Perbandingan Query Engine Lain
Membandingkan performa DuckDB dengan *query engine* lain dalam ekosistem *Data Lake*, seperti Apache Spark, Polars, atau Dremio, untuk skenario dan dataset yang sama.
4. Integrasi Data Streaming
Mengembangkan arsitektur untuk memasukkan komponen *real-time data streaming* (misalnya menggunakan Apache Kafka atau Flink) ke dalam kedua arsitektur dan membandingkan performa latensi datanya.

DAFTAR PUSTAKA

- Anthony, J. (2024). *51 Essential Business Intelligence Statistics: 2021 Analysis of Trends, Data and Market Share*. <https://financesonline.com/20-essential-business-intelligence-statistics-analysis-of-trends-data-and-market-share/>
- Branka. (2024). *Data Warehouse Statistics – 2024*. <https://truelist.co/blog/data-warehouse-statistics/>
- El Aissi, M. E. M., Benjelloun, S., Loukili, Y., Lakhrissi, Y., Boushaki, A. El, Chougrad, H., & Elhaj Ben Ali, S. (2022). Data Lake Versus Data Warehouse Architecture: A Comparative Study. In S. Bennani, Y. Lakhrissi, G. Khaissidi, A. Mansouri, & Y. Khamlichi (Eds.), *WITS 2020* (pp. 201–210). Springer Singapore.
- Farhan, M. S., Youssef, A., & Abdelhamid, L. (2024). A Model for Enhancing Unstructured Big Data Warehouse Execution Time. *Big Data and Cognitive Computing*, 8(2). <https://doi.org/10.3390/bdcc8020017>
- Gorelik, A., & Safari, an O. M. Company. (2019). *The Enterprise Big Data Lake*.
- Halani, K. R., Saxena, R., & others. (2021). Critical analysis of manual versus automation testing. *2021 International Conference on Computational Performance Evaluation (ComPE)*, 132–135.
- Harby, A. A., & Zulkernine, F. (2022). From Data Warehouse to Lakehouse: A Comparative Review. *Proceedings - 2022 IEEE International Conference on Big Data, Big Data 2022*, 389–395. <https://doi.org/10.1109/BigData55660.2022.10020719>
- Himami, M., Abdullah, A. S., Nurma Yulita, I., & Suryani, M. (2021). Utilization of Data Warehouse in Business Intelligence with Kimball Method at Company XYZ. *2021 International Conference on Artificial Intelligence and Big Data Analytics, ICAIBDA 2021*, 146–151. <https://doi.org/10.1109/ICAIBDA53487.2021.9689720>
- Iyengar, V. (2023). *The Data Hub and Spoke: Data Infrastructure “3.0” for the Age of Generative AI*. <https://foundationcapital.com/the-data-hub-and-spoke-data-infrastructure-3-0-for-the-age-of-generative-ai/>
- Jaiswar, A. (2024). *Data Warehouse: Architecture Overview*.
- Katari, A. (2020). *Data lakes and Optimizing Query*. 5(9).
- Kimball, R., & Ross, M. (2016). *The kimball group reader: Relentlessly practical tools for data warehousing and business intelligence remastered collection*. John Wiley & Sons.
- Kovacic, I., Schuetz, C. G., Neumayr, B., & Schrefl, M. (2022). OLAP Patterns: A pattern-based approach to multidimensional data analysis. *Data and Knowledge Engineering*, 138(September 2020), 101948. <https://doi.org/10.1016/j.datak.2021.101948>
- Kovalev, I., Nezhmetdinov, R., & Kvashnin, D. (2019). Big data analytics of the technological equipment based on Data Lake architecture. *MATEC Web of Conferences*, 298, 00079. <https://doi.org/10.1051/matecconf/201929800079>

- Nambiar, A., & Mundra, D. (2022). An Overview of Data Warehouse and Data Lake in Modern Enterprise Data Management. *Big Data and Cognitive Computing*, 6(4). <https://doi.org/10.3390/bdcc6040132>
- Orogat, A. (2021). *Data Lakes for Big Data [Report on Existing Data Lakes]*. March, 2–8. <https://doi.org/10.13140/RG.2.2.17599.84643>
- Pattyam, S. P. (n.d.). *Data Engineering for Business Intelligence : Techniques for ETL , Data Integration , and Real-Time Reporting*. 1(2), 1–53.
- Snyder, H. (2019). Literature review as a research methodology: An overview and guidelines. *Journal of Business Research*, 104(August), 333–339. <https://doi.org/10.1016/j.jbusres.2019.07.039>
- Sulistyo, H. A., Kusumasari, T. F., & Alam, E. N. (2020). Implementation of Data Cleansing Null Method for Data Quality Management Dashboard using Pentaho Data Integration. *2020 3rd International Conference on Information and Communications Technology, ICOIACT 2020*, 12–16. <https://doi.org/10.1109/ICOIACT50329.2020.9332030>

LAMPIRAN

Lampiran 1. Script Python untuk Konversi File Comments.xml ke Comments.parquet (Hub)

```
## main.py
import time
from xml_to_parquet import stream_comments_to_parquet
from parquet_tools import merge_batches_to_final, upload_to_minio,
delete_local_batches
from config import FINAL_PARQUET

def main():
    start_time = time.time()

    stream_comments_to_parquet()
    merge_batches_to_final()
    delete_local_batches()
    upload_to_minio(FINAL_PARQUET)

    elapsed = time.time() - start_time
    mins, secs = divmod(elapsed, 60)
    print(f"\n🕒 Waktu total ETL Comments: {int(mins)} menit {secs:.2f} detik")

if __name__ == "__main__":
    main()

## config.py
BATCH_SIZE = 10000
BATCH_FOLDER = "output/comments_batches/"
FINAL_PARQUET = "output/Comments_final.parquet"
OUTPUT_PREFIX = BATCH_FOLDER + "Comments_batch_"

MINIO_BUCKET = "stackoverflow"
MINIO_OBJECT_NAME = "Comments_final.parquet"
MINIO_ENDPOINT = "127.0.0.1:9000"
MINIO_ACCESS_KEY = "IhsanTA"
MINIO_SECRET_KEY = "Deceitfull12"
MINIO_SECURE = False

COMMENTS_COLUMNS = [
    "Id", "PostId", "Score", "Text", "CreationDate", "UserId",
    "UserDisplayName"
]

## parquet_tools.py
import pyarrow.parquet as pq
import glob
import os
from config import FINAL_PARQUET, OUTPUT_PREFIX
from minio import Minio
from config import MINIO_ENDPOINT, MINIO_ACCESS_KEY, MINIO_SECRET_KEY,
MINIO_SECURE, MINIO_BUCKET
```

```

def merge_batches_to_final():
    print("🔄 Menggabungkan semua file batch ke", FINAL_PARQUET)
    batch_files = sorted(glob.glob(f"{OUTPUT_PREFIX}*.parquet"))

    writer = None
    total_rows = 0

    for idx, file in enumerate(batch_files):
        table = pq.read_table(file)
        if writer is None:
            writer = pq.ParquetWriter(FINAL_PARQUET, table.schema)
        writer.write_table(table)
        total_rows += table.num_rows
        if idx % 500 == 0:
            print(f"📊 Proses gabung: file ke-{idx}, total baris sejauh ini: {total_rows}")

    if writer:
        writer.close()

    print(f"✅ Gabungan selesai ke {FINAL_PARQUET} dengan total baris: {total_rows}")
    return batch_files

def upload_to_minio(filename):
    print(f"☁ Mengunggah {filename} ke MinIO...")
    client = Minio(
        MINIO_ENDPOINT,
        access_key=MINIO_ACCESS_KEY,
        secret_key=MINIO_SECRET_KEY,
        secure=MINIO_SECURE
    )
    if not client.bucket_exists(MINIO_BUCKET):
        client.make_bucket(MINIO_BUCKET)

    file_size = os.stat(filename).st_size

    try:
        with open(filename, "rb") as file_data:
            client.put_object(
                MINIO_BUCKET,
                filename,
                file_data,
                length=file_size,
                part_size=1024 * 1024 * 128
            )

        print(f"✅ {filename} berhasil diunggah ke bucket '{MINIO_BUCKET}'")
    except Exception as e:
        print(f"❌ Gagal mengunggah ke MinIO: {e}")

def delete_local_batches():
    print("🗑 Menghapus file batch Parquet lokal...")
    batch_files = glob.glob(f"{OUTPUT_PREFIX}*.parquet")
    for file in batch_files:
        try:
            os.remove(file)
        except Exception as e:
            print(f"⚠ Gagal menghapus {file}: {e}")
    print("✅ Semua file batch lokal telah dihapus.")

```

```

## xml_to_parquet.py
from lxml import etree
import pyarrow as pa
import pyarrow.parquet as pq
from datetime import datetime
import os
from config import BATCH_SIZE, OUTPUT_PREFIX, COMMENTS_COLUMNS
from tqdm import tqdm

COLUMN_TYPES = {
    "Id": pa.int64(),
    "PostId": pa.int64(),
    "Score": pa.int64(),
    "Text": pa.string(),
    "CreationDate": pa.timestamp('ns'),
    "UserId": pa.int64(),
    "UserDisplayName": pa.string()
}

def sanitize_value(value, target_type):
    if value is None:
        return None
    try:
        if pa.types.is_integer(target_type):
            return int(value)
        elif pa.types.is_timestamp(target_type):
            try:
                return datetime.strptime(value, "%Y-%m-%dT%H:%M:%S.%f")
            except ValueError:
                return datetime.strptime(value, "%Y-%m-%dT%H:%M:%S")
        else:
            return value
    except Exception:
        return None

def stream_comments_to_parquet():
    os.makedirs(os.path.dirname(OUTPUT_PREFIX), exist_ok=True)
    print("📖 Membaca Comments.xml dan menyimpan ke batch parquet...")

    file_path = r"D:\Kuliah\Sem8\Tugas Akhir\Data Lake Archive\Comments.xml"
    context = etree.iterparse(file_path, events=('end',), tag='row')

    batch_data = {col: [] for col in COMMENTS_COLUMNS}
    batch_count, total_count, file_count = 0, 0, 0

    for _, elem in tqdm(context, desc="Parsing Comments.xml"):
        for col in COMMENTS_COLUMNS:
            batch_data[col].append(elem.attrib.get(col))
        elem.clear()

        batch_count += 1
        total_count += 1

        if batch_count >= BATCH_SIZE:
            write_batch(batch_data, file_count)
            batch_data = {col: [] for col in COMMENTS_COLUMNS}
            batch_count = 0
            file_count += 1

    if batch_count > 0:

```

```

_write_batch(batch_data, file_count)

    print(f"✅ Total {total_count:,} baris disimpan ke {file_count + 1}
file batch.")

def _write_batch(batch_data, batch_index):
    arrays = []
    for col in COMMENTS_COLUMNS:
        col_type = COLUMN_TYPES.get(col, pa.string())
        sanitized = [sanitize_value(val, col_type) for val in
batch_data[col]]
        arrays.append(pa.array(sanitized, type=col_type))

    table = pa.table(arrays, names=COMMENTS_COLUMNS)
    pq.write_table(table, f"{OUTPUT_PREFIX}{batch_index}.parquet")
    print(f"📁 Simpan batch ke-{batch_index}, baris: {len(arrays[0])}")

```

Lampiran 2. Script Python untuk Konversi File Posts.xml ke Posts.parquet (Hub)

```
## main.py
import time
from xml_to_parquet import stream_posts_to_parquet
from parquet_tools import merge_batches_to_final, upload_to_minio,
delete_local_batches
from config import FINAL_PARQUET

def main():
    start_time = time.time()

    stream_posts_to_parquet()
    merge_batches_to_final()
    delete_local_batches()
    upload_to_minio(FINAL_PARQUET)

    elapsed = time.time() - start_time
    mins, secs = divmod(elapsed, 60)
    print(f"\n⌚ Waktu total ETL Posts: {int(mins)} menit {secs:.2f} detik")

if __name__ == "__main__":
    main()

## config.py
BATCH_SIZE = 10000
BATCH_FOLDER = "output/posts_batches/"
FINAL_PARQUET = "output/Posts_final.parquet"
OUTPUT_PREFIX = BATCH_FOLDER + "Posts_batch_"

MINIO_BUCKET = "stackoverflow"
MINIO_OBJECT_NAME = "Posts_final.parquet"
MINIO_ENDPOINT = "127.0.0.1:9000"
MINIO_ACCESS_KEY = "IhsanTA"
MINIO_SECRET_KEY = "Deceitfull12"
MINIO_SECURE = False

POSTS_COLUMNS = [
    "AcceptedAnswerId", "AnswerCount", "Body", "ClosedDate",
    "CommentCount",
    "CommunityOwnedDate", "ContentLicense", "CreationDate",
    "FavoriteCount", "Id",
    "LastActivityDate", "LastEditDate", "LastEditorDisplayName",
    "LastEditorUserId",
    "OwnerDisplayName", "OwnerUserId", "ParentId", "PostTypeId",
    "Score", "Tags",
    "Title", "ViewCount"
]

## parquet_tools.py
import pyarrow.parquet as pq
import glob
import os
```

```

from config import FINAL_PARQUET, OUTPUT_PREFIX, MINIO_BUCKET,
MINIO_OBJECT_NAME
from config import MINIO_ENDPOINT, MINIO_ACCESS_KEY, MINIO_SECRET_KEY,
MINIO_SECURE
from minio import Minio
from tqdm import tqdm

def merge_batches_to_final():
    print("🔄 Menggabungkan file batch ke", FINAL_PARQUET)
    batch_files = sorted(glob.glob(f"{OUTPUT_PREFIX}*.parquet"))
    os.makedirs(os.path.dirname(FINAL_PARQUET), exist_ok=True)

    writer, total_rows = None, 0

    for file in tqdm(batch_files, desc="Menggabungkan batch"):
        table = pq.read_table(file)
        if writer is None:
            writer = pq.ParquetWriter(FINAL_PARQUET, table.schema)
        writer.write_table(table)
        total_rows += table.num_rows

    if writer:
        writer.close()

    print(f"✅ Final Parquet selesai: {FINAL_PARQUET}, total baris:
{total_rows}")
    return batch_files

def upload_to_minio(filename):
    print(f"☁ Upload {filename} ke MinIO...")
    client = Minio(
        MINIO_ENDPOINT,
        access_key=MINIO_ACCESS_KEY,
        secret_key=MINIO_SECRET_KEY,
        secure=MINIO_SECURE
    )

    if not client.bucket_exists(MINIO_BUCKET):
        client.make_bucket(MINIO_BUCKET)

    with open(filename, 'rb') as file_stream:

client.put_object(
    MINIO_BUCKET,
    MINIO_OBJECT_NAME,
    data=file_stream,
    length=os.stat(filename).st_size,
    part_size=10 * 1024 * 1024
)

    print(f"✅ Upload sukses ke bucket '{MINIO_BUCKET}' sebagai
'{MINIO_OBJECT_NAME}'")

def delete_local_batches():
    print("🗑 Menghapus file batch lokal...")
    batch_files = glob.glob(f"{OUTPUT_PREFIX}*.parquet")
    for file in batch_files:

```

```

try:
    os.remove(file)
except Exception as e:
    print(f"⚠️ Gagal hapus {file}: {e}")
print("✅ File batch lokal terhapus.")

## xml_to_parquet.py
from lxml import etree
import pyarrow as pa
import pyarrow.parquet as pq
from datetime import datetime
import os
from config import BATCH_SIZE, OUTPUT_PREFIX, POSTS_COLUMNS
from tqdm import tqdm

COLUMN_TYPES = {
    "AcceptedAnswerId": pa.int64(),
    "AnswerCount": pa.int64(),
    "Body": pa.string(),
    "ClosedDate": pa.timestamp('ns'),
    "CommentCount": pa.int64(),
    "CommunityOwnedDate": pa.timestamp('ns'),
    "ContentLicense": pa.string(),
    "CreationDate": pa.timestamp('ns'),
    "FavoriteCount": pa.int64(),
    "Id": pa.int64(),
    "LastActivityDate": pa.timestamp('ns'),
    "LastEditDate": pa.timestamp('ns'),
    "LastEditorDisplayName": pa.string(),
    "LastEditorUserId": pa.int64(),
    "OwnerDisplayName": pa.string(),
    "OwnerUserId": pa.int64(),
    "ParentId": pa.int64(),
    "PostTypeId": pa.int64(),
    "Score": pa.int64(),
    "Tags": pa.string(),
    "Title": pa.string(),
    "ViewCount": pa.int64()
}

def sanitize_value(value, target_type):
    if value is None:
        return None
    try:
        if pa.types.is_integer(target_type):
            return int(value)
        elif pa.types.is_timestamp(target_type):
            try:
                return datetime.strptime(value, "%Y-%m-%dT%H:%M:%S.%f")
            except ValueError:
                return datetime.strptime(value, "%Y-%m-%dT%H:%M:%S")
        else:
            return value
    except Exception:
        return None

def stream_posts_to_parquet():
    os.makedirs(os.path.dirname(OUTPUT_PREFIX), exist_ok=True)
    print("📂 Membaca Posts.xml dan menyimpan ke batch parquet...")

```

```

file_path = r"D:\Kuliah\Sem8\Tugas Akhir\Data Lake Archive\Posts.xml"
context = etree.iterparse(file_path, events=('end',), tag='row')

batch_data = {col: [] for col in POSTS_COLUMNS}
batch_count, total_count, file_count = 0, 0, 0

for _, elem in tqdm(context, desc="Parsing Posts.xml"):
    for col in POSTS_COLUMNS:
        batch_data[col].append(elem.attrib.get(col))
        elem.clear()

    batch_count += 1
    total_count += 1

    if batch_count >= BATCH_SIZE:
        _write_batch(batch_data, file_count)
        batch_data = {col: [] for col in POSTS_COLUMNS}
        batch_count = 0
        file_count += 1

if batch_count > 0:
    _write_batch(batch_data, file_count)

print(f"✅ Total {total_count:,} baris disimpan ke {file_count + 1} file batch.")

def _write_batch(batch_data, batch_index):
    arrays = []
    for col in POSTS_COLUMNS:
        col_type = COLUMN_TYPES.get(col, pa.string())
        sanitized = [sanitize_value(val, col_type) for val in batch_data[col]]
        arrays.append(pa.array(sanitized, type=col_type))

    table = pa.table(arrays, names=POSTS_COLUMNS)
    pq.write_table(table, f"{OUTPUT_PREFIX}{batch_index}.parquet")
    print(f"💾 Simpan batch ke-{batch_index}, baris: {len(arrays[0])}")

```

Lampiran 3. Script Python untuk Konversi File Users.xml ke Users.parquet (Hub)

```
## main.py
import time
from xml_to_parquet import stream_users_to_parquet
from parquet_tools import merge_batches_to_final, upload_to_minio,
delete_local_batches
from config import FINAL_PARQUET

def main():
    start_time = time.time()

    stream_users_to_parquet()
    batch_files = merge_batches_to_final()
    delete_local_batches()
    upload_to_minio(FINAL_PARQUET)

    end_time = time.time()
    elapsed_seconds = end_time - start_time
    minutes, seconds = divmod(elapsed_seconds, 60)
    print(f"\n🕒 Waktu total ETL (XML → Parquet → MinIO):
{int(minutes)} menit {seconds:.2f} detik")

if __name__ == "__main__":
    main()

## config.py
BATCH_SIZE = 10000
OUTPUT_PREFIX = "Users_batch_"
FINAL_PARQUET = "Users_final.parquet"
MINIO_BUCKET = "stackoverflow"
MINIO_OBJECT_NAME = "Users_final.parquet"
MINIO_ENDPOINT = "127.0.0.1:9000"
MINIO_ACCESS_KEY = "IhsanTA"
MINIO_SECRET_KEY = "Deceitfull12"
MINIO_SECURE = False

USERS_COLUMNS = [
    "Id", "AboutMe", "CreationDate", "DisplayName", "DownVotes",
    "LastAccessDate", "Location", "Reputation", "UpVotes", "Views",
    "WebsiteUrl", "AccountId"
]

## parquet_tools.py
import pyarrow.parquet as pq
import glob
import os
from config import FINAL_PARQUET, OUTPUT_PREFIX
from minio import Minio
from config import MINIO_ENDPOINT, MINIO_ACCESS_KEY, MINIO_SECRET_KEY,
MINIO_SECURE, MINIO_BUCKET, MINIO_OBJECT_NAME

def merge_batches_to_final():
    print("📁 Menggabungkan semua file batch ke", FINAL_PARQUET)
    batch_files = sorted(glob.glob(f"{OUTPUT_PREFIX}*.parquet"))
```

```

writer = None
total_rows = 0

for idx, file in enumerate(batch_files):
    table = pq.read_table(file)
    if writer is None:
        writer = pq.ParquetWriter(FINAL_PARQUET, table.schema)
    writer.write_table(table)
    total_rows += table.num_rows
    if idx % 500 == 0:
        print(f"📁 Proses gabung: file ke-{idx}, total baris sejauh
ini: {total_rows}")

    if writer:
        writer.close()

    print(f"✅ Gabungan selesai ke {FINAL_PARQUET} dengan total baris:
{total_rows}")
    return batch_files

def upload_to_minio(filename):
    from minio import Minio
    from minio.commonconfig import CopySource
    import io

    print(f"☁ Mengunggah {filename} ke MinIO...")

    client = Minio(
        MINIO_ENDPOINT,
        access_key=MINIO_ACCESS_KEY,
        secret_key=MINIO_SECRET_KEY,
        secure=MINIO_SECURE
    )

    if not client.bucket_exists(MINIO_BUCKET):
        client.make_bucket(MINIO_BUCKET)

    file_size = os.stat(filename).st_size
    file_stream = open(filename, 'rb')

    try:
        client.put_object(
            bucket_name=MINIO_BUCKET,
            object_name=filename,
            data=file_stream,
            length=file_size,
            part_size=10 * 1024 * 1024 # 10 MB per part (aman dan
ringan)
        )
    finally:
        file_stream.close()

    print(f"✅ {filename} berhasil diunggah ke bucket '{MINIO_BUCKET}'")

def delete_local_batches():
    print("🗑 Menghapus file batch Parquet lokal...")
    batch_files = glob.glob(f"{OUTPUT_PREFIX}*.parquet")
    for file in batch_files:
        try:
            os.remove(file)
        except Exception as e:

```

```

print(f"⚠️ Gagal menghapus {file}: {e}")
print("✅ Semua file batch lokal telah dihapus.")

## xml_to_parquet
from lxml import etree
import pyarrow as pa
import pyarrow.parquet as pq
import os
from datetime import datetime
from config import BATCH_SIZE, OUTPUT_PREFIX, USERS_COLUMNS

# Tipe data per kolom
COLUMN_TYPES = {
    "Id": pa.int64(),
    "AboutMe": pa.string(),
    "CreationDate": pa.timestamp('ns'),
    "DisplayName": pa.string(),
    "DownVotes": pa.int64(),
    "LastAccessDate": pa.timestamp('ns'),
    "Location": pa.string(),
    "Reputation": pa.int64(),
    "UpVotes": pa.int64(),
    "Views": pa.int64(),
    "WebsiteUrl": pa.string(),
    "AccountId": pa.int64()
}

def sanitize_value(value, target_type):
    if value is None:
        return None
    try:
        if pa.types.is_integer(target_type):
            return int(value)
        elif pa.types.is_timestamp(target_type):
            return datetime.fromisoformat(value)
        else:
            return value
    except Exception:
        return None

def stream_users_to_parquet():
    print("📂 Membaca Users.xml dari file lokal...")

    file_path = r"D:\Kuliah\Sem8\Tugas Akhir\Data Lake Archive\Users.xml"
    context = etree.iterparse(file_path, events=('end',), tag='row')

    batch_data = {col: [] for col in USERS_COLUMNS}
    batch_count = 0
    total_count = 0
    file_count = 0

    for event, elem in context:
        for col in USERS_COLUMNS:
            batch_data[col].append(elem.attrib.get(col))
        elem.clear()
        batch_count += 1
        total_count += 1

        if batch_count >= BATCH_SIZE:
            try:

```

```

arrays = []
    for col in USERS_COLUMNS:
        col_type = COLUMN_TYPES.get(col, pa.string())
        sanitized = [sanitize_value(val, col_type) for val
in batch_data[col]]
        arrays.append(pa.array(sanitized, type=col_type))

        table = pa.table(arrays, names=USERS_COLUMNS)
        pq.write_table(table,
f"{OUTPUT_PREFIX}{file_count}.parquet")
        print(f"💾 Simpan batch {file_count}, total row:
{batch_count}")
        except Exception as e:
            print(f"\n❌ Gagal membuat tabel Parquet pada batch ke-
{file_count}")

            print(f"🔍 Kolom terakhir: {col}")
            print(f"🔍 Contoh nilai: {batch_data[col][:5]}")
            print(f"🔍 Error: {e}")
            raise e

        batch_data = {col: [] for col in USERS_COLUMNS}
        batch_count = 0
        file_count += 1

# simpan sisa batch terakhir
if batch_count > 0:
    arrays = []
    for col in USERS_COLUMNS:
        col_type = COLUMN_TYPES.get(col, pa.string())
        sanitized = [sanitize_value(val, col_type) for val in
batch_data[col]]
        arrays.append(pa.array(sanitized, type=col_type))

        table = pa.table(arrays, names=USERS_COLUMNS)
        pq.write_table(table, f"{OUTPUT_PREFIX}{file_count}.parquet")
        print(f"💾 Simpan batch terakhir {file_count}, total row:
{batch_count}")

    print(f"\n✅ Total rows: {total_count:,} ke {file_count + 1} file
batch parquet.")

```

Lampiran 4. Script Python untuk Konversi File Votes.xml ke Votes.parquet (Hub)

```
## main.py
import time
from xml_to_parquet import stream_votes_to_parquet
from parquet_tools import merge_batches_to_final, upload_to_minio,
delete_local_batches
from config import FINAL_PARQUET

def main():
    start_time = time.time()

    stream_votes_to_parquet()
    batch_files = merge_batches_to_final()
    delete_local_batches()
    upload_to_minio(FINAL_PARQUET)

    end_time = time.time()
    elapsed_seconds = end_time - start_time
    minutes, seconds = divmod(elapsed_seconds, 60)
    print(f"\n🕒 Waktu total ETL (XML → Parquet → MinIO):
{int(minutes)} menit {seconds:.2f} detik")

if __name__ == "__main__":
    main()

## config.py
BATCH_SIZE = 10000
BATCH_FOLDER = "output/votes_batches/"
FINAL_PARQUET = "output/Votes_final.parquet"
OUTPUT_PREFIX = BATCH_FOLDER + "Votes_batch_"

MINIO_BUCKET = "stackoverflow"
MINIO_OBJECT_NAME = "Votes_final.parquet"
MINIO_ENDPOINT = "127.0.0.1:9000"
MINIO_ACCESS_KEY = "IhsanTA"
MINIO_SECRET_KEY = "Deceitfull12"
MINIO_SECURE = False

VOTES_COLUMNS = ["Id", "PostId", "VoteTypeId", "CreationDate"]

## parquet_tools.py
import pyarrow.parquet as pq
import glob
import os
from config import FINAL_PARQUET, OUTPUT_PREFIX
from minio import Minio
from config import MINIO_ENDPOINT, MINIO_ACCESS_KEY, MINIO_SECRET_KEY,
MINIO_SECURE, MINIO_BUCKET

def merge_batches_to_final():
    print("📁 Menggabungkan semua file batch ke", FINAL_PARQUET)
    batch_files = sorted(glob.glob(f"{OUTPUT_PREFIX}*.parquet"))
```

```

writer = None
total_rows = 0

for idx, file in enumerate(batch_files):
    table = pq.read_table(file)
    if writer is None:
        writer = pq.ParquetWriter(FINAL_PARQUET, table.schema)
    writer.write_table(table)
    total_rows += table.num_rows
    if idx % 500 == 0:
        print(f"🔄 Proses gabung: file ke-{idx}, total baris sejauh ini: {total_rows}")

    if writer:
        writer.close()

    print(f"✅ Gabungan selesai ke {FINAL_PARQUET} dengan total baris: {total_rows}")
    return batch_files

def upload_to_minio(filename):
    print(f"☁ Mengunggah {filename} ke MinIO...")

    client = Minio(
        MINIO_ENDPOINT,
        access_key=MINIO_ACCESS_KEY,
        secret_key=MINIO_SECRET_KEY,
        secure=MINIO_SECURE
    )

    if not client.bucket_exists(MINIO_BUCKET):
        client.make_bucket(MINIO_BUCKET)

    file_size = os.stat(filename).st_size

    try:
        with open(filename, "rb") as file_data:
            client.put_object(
                MINIO_BUCKET,
                filename,
                file_data,
                length=file_size,
                part_size=1024 * 1024 * 128
            )

        print(f"✅ {filename} berhasil diunggah ke bucket '{MINIO_BUCKET}'")
    except Exception as e:
        print(f"❌ Gagal mengunggah ke MinIO: {e}")

def delete_local_batches():
    print("🗑 Menghapus file batch Parquet lokal...")
    batch_files = glob.glob(f"{OUTPUT_PREFIX}*.parquet")
    for file in batch_files:
        try:
            os.remove(file)
        except Exception as e:
            print(f"⚠ Gagal menghapus {file}: {e}")
    print("✅ Semua file batch lokal telah dihapus.")

```

```

## xml_to_parquet.py
from lxml import etree
import pyarrow as pa
import pyarrow.parquet as pq
from datetime import datetime
import os
from config import BATCH_SIZE, OUTPUT_PREFIX, VOTES_COLUMNS
from tqdm import tqdm

COLUMN_TYPES = {
    "Id": pa.int64(),
    "PostId": pa.int64(),
    "VoteTypeId": pa.int64(),
    "CreationDate": pa.timestamp('ns')
}

def sanitize_value(value, target_type):
    if value is None:
        return None
    try:
        if pa.types.is_integer(target_type):
            return int(value)
        elif pa.types.is_timestamp(target_type):
            try:
                return datetime.strptime(value, "%Y-%m-%dT%H:%M:%S.%f")
            except ValueError:
                return datetime.strptime(value, "%Y-%m-%dT%H:%M:%S")
        else:
            return value
    except Exception:
        return None

def stream_votes_to_parquet():
    os.makedirs(os.path.dirname(OUTPUT_PREFIX), exist_ok=True)
    print("📄 Membaca Votes.xml dan menyimpan ke batch parquet...")

    file_path = r"D:\Kuliah\Sem8\Tugas Akhir\Data Lake Archive\Votes.xml"
    context = etree.iterparse(file_path, events=('end',), tag='row')

    batch_data = {col: [] for col in VOTES_COLUMNS}
    batch_count, total_count, file_count = 0, 0, 0

    for _, elem in tqdm(context, desc="Parsing Votes.xml"):
        for col in VOTES_COLUMNS:
            batch_data[col].append(elem.attrib.get(col))
            elem.clear()

        batch_count += 1
        total_count += 1

        if batch_count >= BATCH_SIZE:
            _write_batch(batch_data, file_count)
            batch_data = {col: [] for col in VOTES_COLUMNS}
            batch_count = 0
            file_count += 1

    if batch_count > 0:
        _write_batch(batch_data, file_count)

```

```

    print(f"✅ Total {total_count:,} baris disimpan ke {file_count + 1}
file batch.")

def _write_batch(batch_data, batch_index):
    arrays = []
    for col in VOTES_COLUMNS:
        col_type = COLUMN_TYPES.get(col, pa.string())

    sanitized = [sanitize_value(val, col_type) for val in batch_data[col]]
    arrays.append(pa.array(sanitized, type=col_type))

    table = pa.table(arrays, names=VOTES_COLUMNS)
    pq.write_table(table, f"{OUTPUT_PREFIX}{batch_index}.parquet")
    print(f"📁 Simpan batch ke-{batch_index}, baris: {len(arrays[0])}")

```

Lampiran 5. Query DuckDB untuk Normalisasi Comments.parquet ke Spoke 1 (EDW 3NF)

```
COPY (  
  SELECT Id, PostId, UserId, Score, CreationDate, Text  
  FROM read_parquet('F:/MinIO/ParquetData/Fix/Comments.parquet')  
  WHERE CAST(CreationDate AS DATE) BETWEEN '2008-07-31' AND '2013-12-31'  
) TO 'X:/spoke1_edw3nf/Comments_3nf.parquet'  
(FORMAT 'PARQUET', CODEC 'ZSTD');
```

Lampiran 6. Query DuckDB untuk Normalisasi Posts.parquet ke Spoke 1 (EDW 3NF)

```
COPY (  
  SELECT Id, PostTypeId, ParentId, AcceptedAnswerId, OwnerUserId,  
  LastEditorUserId,  
    CreationDate, LastActivitydate, LastEditDate, ClosedDate,  
  CommunityOwnedDate,  
    Score, ViewCount, CommentCount, AnswerCount, FavoriteCount, Title,  
  Body, Tags, ContentLicense  
  FROM read_parquet('F:/MinIO/ParquetData/Fix/Posts.parquet')  
  WHERE CAST(CreationDate AS DATE) BETWEEN '2008-07-31' AND '2013-12-31'  
) TO 'X:/spoke1_edw3nf/Comments_3nf.parquet'  
(FORMAT 'PARQUET', CODEC 'ZSTD');
```

Lampiran 7. Query DuckDB untuk Normalisasi Users.parquet ke Spoke 1 (EDW 3NF)

```
COPY (  
  SELECT  
    Id, DisplayName, Location, CreationDate, Reputation, Views, UpVotes,  
    DownVotes  
  FROM read_parquet('F:/MinIO/ParquetData/Fix/Users.parquet')  
  SELECT Id, PostTypeId, ParentId, AcceptedAnswerId, OwnerUserId,  
    LastEditorUserId,  
    CreationDate, LastActivitydate, LastEditDate, ClosedDate,  
    CommunityOwnedDate
```

Lampiran 8. Query DuckDB untuk Normalisasi Votes.parquet ke Spoke 1 (EDW 3NF)

```
COPY (  
  SELECT Id, PostId, VoteTypeId, UserId, CreationDate, BountyAmount  
  FROM read_parquet('F:/MinIO/ParquetData/Fix/Votes.parquet')  
  WHERE CAST(CreationDate AS DATE) BETWEEN '2008-07-31' AND '2013-12-31'  
) TO 'X:/spoke1_edw3nf/Votes_3nf.parquet'  
(FORMAT 'PARQUET', CODEC 'ZSTD');
```

Lampiran 9. Query DuckDB untuk ETL dim_activity_type ke Spoke 2 (Dimensional Modelling)

```
COPY (  
  SELECT * FROM (  
    VALUES  
      (0, 'Unknown'),  
      (1, 'Create Post'),  
      (2, 'Create Comment'),  
      (3, 'Edit Post'),  
      (10, 'Upvote'),  
      (11, 'Downvote'),  
      (12, 'Favorite Post'),  
      (20, 'Accept Answer'),  
      (21, 'Close Post'),  
      (22, 'Reopen Post'),  
      (23, 'Approve Edit'),  
      (24, 'Flag as Offensive'),  
      (25, 'Flag as Spam'),  
      (26, 'Delete Post'),  
      (27, 'Undelete Post'),  
      (30, 'Start Bounty'),  
      (31, 'Close Bounty'),  
      (40, 'Earn Badge')  
  ) AS tbl(ActivityTypeKey, ActivityName)  
) TO 'X:/spoke2_dm/dim_activity_type.parquet' (FORMAT 'PARQUET', CODEC  
'ZSTD', OVERWRITE_OR_IGNORE true);
```

Lampiran 10. Query DuckDB untuk ETL dim_post ke Spoke 2 (Dimensional Modelling)

```
COPY (  
COPY (  
    SELECT * FROM (  
        VALUES  
            (0, 'Unknown'),  
            (1, 'Create Post'),  
            (2, 'Create Comment'), COPY (  
    SELECT  
        row_number() OVER () AS PostKey,  
        Id AS PostId_Original,  
        CASE  
            WHEN PostTypeId = 1 THEN 'Question'  
            WHEN PostTypeId = 2 THEN 'Answer'  
            ELSE 'Other'  
        END AS PostType,  
        Title
```

Lampiran 11. Query DuckDB untuk ETL dim_time ke Spoke 2 (Dimensional Modelling)

```
COPY (  
  WITH date_series AS (  
    SELECT UNNEST(generate_series('2008-07-31'::date, '2013-12-  
31'::date, interval 1 day)) AS FullDate  
  )  
  SELECT  
    row_number() OVER (ORDER BY FullDate) AS TimeKey,  
    FullDate,  
    day(FullDate) AS Day,  
    month(FullDate) AS Month,  
    year(FullDate) AS Year,  
    isodow(FullDate) AS Weekday  
FROM date_series  
SELECT  
  row_number() OVER () AS PostKey,  
  Id AS PostId Original.
```

Lampiran 12. Query DuckDB untuk ETL dim_user ke Spoke 2 (Dimensional Modelling)

```
COPY (  
  SELECT  
    row_number() OVER () AS UserKey,  
    Id AS UserId_Original,  
    DisplayName,  
    Location,  
    CreationDate  
  FROM read_parquet('F:/MinIO/ParquetData/Fix/Users.parquet')  
  WHERE CAST(CreationDate AS DATE) BETWEEN '2008-07-31' AND '2013-12-  
31'  
) TO 'X:/spoke2_dm/dim_user.parquet' (FORMAT 'PARQUET', CODEC 'ZSTD',  
OVERWRITE_OR_IGNORE true);
```

Lampiran 13. Query DuckDB untuk ETL fact_user_activity ke Spoke 2 (Dimensional Modelling)

```

COPY (
    SELECT
        OwnerUserId AS UserId,
        Id AS PostId,
        strftime(CreationDate, '%Y%m%d')::int AS TimeKey,
        1 AS ActivityTypeKey,
        1 AS ActivityCount,
        Score AS ScoreValue,
        len(Body) as PostLength,
        NULL AS BountyAmount,
        NULL AS CommentLength
    FROM read_parquet('F:/MinIO/ParquetData/Posts.parquet')
    WHERE CAST(CreationDate AS DATE) BETWEEN '2008-07-31' AND '2013-12-
31'
    UNION ALL
    SELECT
        UserId,
        PostId,
        strftime(CreationDate, '%Y%m%d')::int AS TimeKey,
        CASE
            WHEN VoteTypeId = 1 THEN 20
            WHEN VoteTypeId = 2 THEN 10
            WHEN VoteTypeId = 3 THEN 11
            WHEN VoteTypeId = 5 THEN 12
            WHEN VoteTypeId = 8 THEN 30
            WHEN VoteTypeId = 9 THEN 31
            ELSE 0
        END AS ActivityTypeKey,
        1 AS ActivityCount,
        CASE WHEN VoteTypeId = 2 THEN 5 WHEN VoteTypeId = 3 THEN -2 ELSE
0 END AS ScoreValue,
        NULL AS PostLength,
        BountyAmount,
        NULL AS CommentLength
    FROM read_parquet('F:/MinIO/ParquetData/Fix/Votes.parquet')
    WHERE CAST(CreationDate AS DATE) BETWEEN '2008-07-31' AND '2013-12-
31'
    UNION ALL
    SELECT
        UserId,
        PostId,
        strftime(CreationDate, '%Y%m%d')::int AS TimeKey,
        2 AS ActivityTypeKey,
        1 AS ActivityCount,
        Score AS ScoreValue,
        NULL AS PostLength,
        NULL AS BountyAmount,
        len(Text) as CommentLength
    FROM read_parquet('F:/MinIO/ParquetData/Fix/Comments.parquet')
    WHERE CAST(CreationDate AS DATE) BETWEEN '2008-07-31' AND '2013-12-
31'

) TO 'X:/spoke2_dm/fact_user_activity.parquet' (FORMAT 'PARQUET', CODEC
'ZSTD', OVERWRITE_OR_IGNORE true);

```

Lampiran 14. Query DuckDB untuk Pengujian QPS pada Skenario 1 pada Spoke 1 (Jumlah Total Setiap Jenis Aktivitas)

```
WITH ActivityCounts AS (  
    SELECT 'Create Post' AS ActivityName, COUNT(*) AS Total_Aktivitas  
    FROM  
    read_parquet('s3://stackoverflow/spoke1_edw3nf/Posts_3nf.parquet')  
  
    UNION ALL  
  
    SELECT 'Create Comment' AS ActivityName, COUNT(*) AS Total_Aktivitas  
    FROM  
    read_parquet('s3://stackoverflow/spoke1_edw3nf/Comments_3nf.parquet')  
  
    UNION ALL  
  
    SELECT  
        CASE VoteTypeId  
            WHEN 1 THEN 'Accept Answer'  
            WHEN 2 THEN 'Upvote'  
            WHEN 4 THEN 'Flag as Offensive'  
            WHEN 5 THEN 'Favorite Post'  
            WHEN 6 THEN 'Close Post'  
            WHEN 7 THEN 'Reopen Post'  
            WHEN 8 THEN 'Start Bounty'  
            WHEN 9 THEN 'Close Bounty'  
            WHEN 10 THEN 'Delete Post'  
            WHEN 11 THEN 'Undelete Post'  
            WHEN 12 THEN 'Flag as Spam'  
            WHEN 15 THEN 'Approve Edit'  
            ELSE 'Unknown'  
        END AS ActivityName,  
        COUNT(*) AS Total_Aktivitas  
    FROM  
    read_parquet('s3://stackoverflow/spoke1_edw3nf/Votes_3nf.parquet')  
    GROUP BY ActivityName  
)  
  
SELECT ActivityName, SUM(Total_Aktivitas) AS Total_Aktivitas  
FROM ActivityCounts  
GROUP BY ActivityName  
ORDER BY Total_Aktivitas DESC;
```

Lampiran 15. Query DuckDB untuk Pengujian QPS pada Skenario 2 pada Spoke 1 (Jumlah Skor yang Dihasilkan dari Vote)

```
SELECT
    year(CreationDate) AS Year,
    SUM(CASE
        WHEN VoteTypeId = 2 THEN 5
        WHEN VoteTypeId = 3 THEN -2
        ELSE 0
    END) AS Total_Score_Generated
FROM
    read_parquet('s3://stackoverflow/spoke1_edw3nf/Votes_3nf.parquet')
WHERE
    VoteTypeId IN (2, 3)
GROUP BY
    Year
ORDER BY
    Year;
```

Lampiran 16. Query DuckDB untuk Pengujian QPS pada Skenario 3 pada Spoke 1 (10 Pengguna Paling Aktif di Lokasi Tertentu)

```
WITH UserActivity AS (  
    SELECT OwnerUserId AS UserId  
FROM  
    read_parquet('s3://stackoverflow/spoke1_edw3nf/Posts_3nf.parquet')  
WHERE OwnerUserId IS NOT NULL  
    UNION ALL  
    SELECT UserId FROM  
read_parquet('s3://stackoverflow/spoke1_edw3nf/Comments_3nf.parquet')  
    UNION ALL  
    SELECT UserId FROM  
read_parquet('s3://stackoverflow/spoke1_edw3nf/Votes_3nf.parquet')  
)  
SELECT  
    u.DisplayName,  
    u.Location,  
    COUNT(*) AS Jumlah_Aktivitas  
FROM  
    UserActivity ua  
JOIN  
    read_parquet('s3://stackoverflow/spoke1_edw3nf/Users_3nf.parquet') u  
ON ua.UserId = u.Id  
WHERE  
    u.Location LIKE '%Indonesia%'  
GROUP BY  
    u.DisplayName, u.Location  
ORDER BY  
    Jumlah_Aktivitas DESC  
LIMIT 10;
```

Lampiran 17. Query DuckDB untuk Pengujian QPS pada Skenario 4 pada Spoke 1 (Jumlah Vote pada Postingan yang Memiliki Tag 'python')

```
SELECT
    COUNT(v.Id) AS "Total Votes on Python Posts"
FROM
    read_parquet('s3://stackoverflow/spoke1_edw3nf/Votes_3nf.parquet')
AS v
JOIN
    read_parquet('s3://stackoverflow/spoke1_edw3nf/Posts_3nf.parquet')
AS p ON v.PostId = p.Id
WHERE
    p.Tags LIKE '%<python>%'
    AND v.VoteTypeId IN (2, 3);
```

Lampiran 18. Query DuckDB untuk Pengujian QPS pada Skenario 5 pada Spoke 1 (Rasio Komentar per Jawaban untuk Pengguna Paling Reputabel)

```
WITH TopUsers AS (  
    SELECT Id AS UserId  
    FROM  
    read_parquet('s3://stackoverflow/spoke1_edw3nf/Users_3nf.parquet')  
    ORDER BY Reputation DESC  
    LIMIT 100  
) ,  
UserAnswers AS (  
    SELECT p.OwnerUserId, COUNT(p.Id) AS Total_Jawaban  
    FROM  
    read_parquet('s3://stackoverflow/spoke1_edw3nf/Posts_3nf.parquet') p  
    JOIN TopUsers tu ON p.OwnerUserId = tu.UserId  
    WHERE p.PostTypeId = 2  
    GROUP BY p.OwnerUserId  
) ,  
UserComments AS (  
    SELECT c.UserId, COUNT(c.Id) AS Total_Komentar  
    FROM  
    read_parquet('s3://stackoverflow/spoke1_edw3nf/Comments_3nf.parquet') c  
    JOIN TopUsers tu ON c.UserId = tu.UserId  
    GROUP BY c.UserId  
)  
SELECT  
    u.DisplayName,  
    COALESCE(ua.Total_Jawaban, 0) AS Total_Jawaban,  
    COALESCE(uc.Total_Komentar, 0) AS Total_Komentar  
FROM  
    TopUsers tu  
JOIN  
    read_parquet('s3://stackoverflow/spoke1_edw3nf/Users_3nf.parquet') u  
ON tu.UserId = u.Id  
LEFT JOIN  
    UserAnswers ua ON tu.UserId = ua.OwnerUserId  
LEFT JOIN  
    UserComments uc ON tu.UserId = uc.UserId  
WHERE  
    COALESCE(ua.Total_Jawaban, 0) > 0;
```

BIODATA PENULIS



Penulis dilahirkan di Madiun, 3 Oktober 2002, merupakan anak kedua dari 2 bersaudara. Penulis telah menempuh pendidikan formal yaitu di TK Zaid bin Tsabit Bandung, SD Islam Athirah 2 Bukit Baruga Makassar, SMPN 1 Balikpapan, dan SMAN 1 Balikpapan. Setelah lulus dari SMAN tahun 2021, Penulis mengikuti SBMPTN dan diterima di Departemen Sistem Informasi FTEIC - ITS pada tahun 2021 dan terdaftar dengan NRP 5026211117.

Selama kuliah, Penulis sempat aktif di beberapa kegiatan organisasi, diantaranya sebagai Staf Ahli Departemen Student Welfare Himpunan Mahasiswa Sistem Informasi, Staf Ahli BIONIX 2023, dan Koordinator IT Bubuhan Anak Balikpapan ITS. Penulis juga sempat menjadi Tutor Hands-on pada Mata Kuliah SIM-RS di Teknik Kedokteran ITS. Penulis memiliki pengalaman magang MBKM sebagai Database Engineer di Direktorat Jenderal Pendidikan Tinggi, Riset, dan Teknologi pada Januari – Juli 2024 dan peserta Studi Independen Bangkit Academy pada September – Desember 2024. Penulis juga pernah mengikuti Virtual Internship sebagai Data Engineer di ID/X Partners pada Januari 2024. Penulis mengucapkan terima kasih pada pembaca atas perhatian dan waktu yang telah diberikan untuk membaca skripsi ini. Penulis juga terbuka terhadap berbagai saran dan masukan yang serta peluang untuk menjalin relasi lebih lanjut melalui akun LinkedIn ihsan112.